
u·n·i·q technologies
We are on a mission to address the digital skills gap for 10 Million+ young professionals, train and empower them to forge a career path into future tech



u·n·i·q technologies

Services | technology | development

Data Structures

15 MARCH, 2022

Contents

- Introduction to Data Structures
- Arrays
- Sorting & Searching
- Stack and Queue
- List and Linked List
- Hashing
- Introduction to Tree and binary tree
- Introduction to Graph
- Set

Introduction to Data Structures



Data

Data

- **Data** is a **raw and unorganized** fact that required to be **processed** to **make it meaningful.**
- **Data** can be simple at the same time unorganized unless it is organized.
- Data can be anything like text, image audio, video etc.
- **Example**

  - 4,8,12,16
  - Dog, cat, cow, cockatoo
  - 161.2, 175.3, 166.4, 164.7, 169.3

Information

Information

- **Information** is a set of data which is **processed in a meaningful way** according to the given requirement.
- **Information** is **processed, structured,** or **presented** in each context to make it **meaningful and useful.**
- **Example**

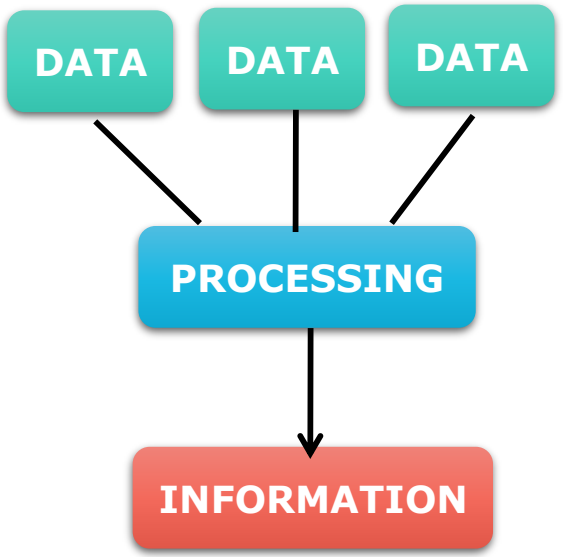
  - 4, 8, 12 and 16 are the first four answers in the 4 x table
  - Dog, cat, cow is a list of household pets
  - 165, 175.2, 186.3, 164.3, 169.3 are the height of 14-year old students.

Note:
"Every information is data, but every data is not information"

Data vs Information

- **Data and information** are similar concepts, but they are not the same thing.
- The main difference between data and information is that,
  - data is a part and
  - information is the whole.





Data vs Information

Parameters	Data	Information
Meaning	Data does not have any specific purpose.	It carries meaning that has been assigned by interpreting data.
Feature	Data is a single unit and is raw. It alone doesn’t have any meaning.	Information is the product and group of data which jointly carry a logical meaning.
Interrelation	Information that is collected	Information that is processed.
Dependency	Data depends upon the sources for collecting data.	Information depends upon data.
Example	Each student's test score is one piece of data.	The average score of a class or of the entire school is information that can be derived from the given data.

Need of organize the data

Data

- Data can be something simple and seemingly random and **useless until it is organized.**



Why should we organize the data?

- For compact storage
- Ability to store large amount of data
- Easy access
- Easy modification



Need of organize the data

Structure

- Way of organizing information, so that it is easier to use
- It's a **suitable container** where set of rules holds data together.



Fruits arranged in organized manner



Oil stored in oil cans

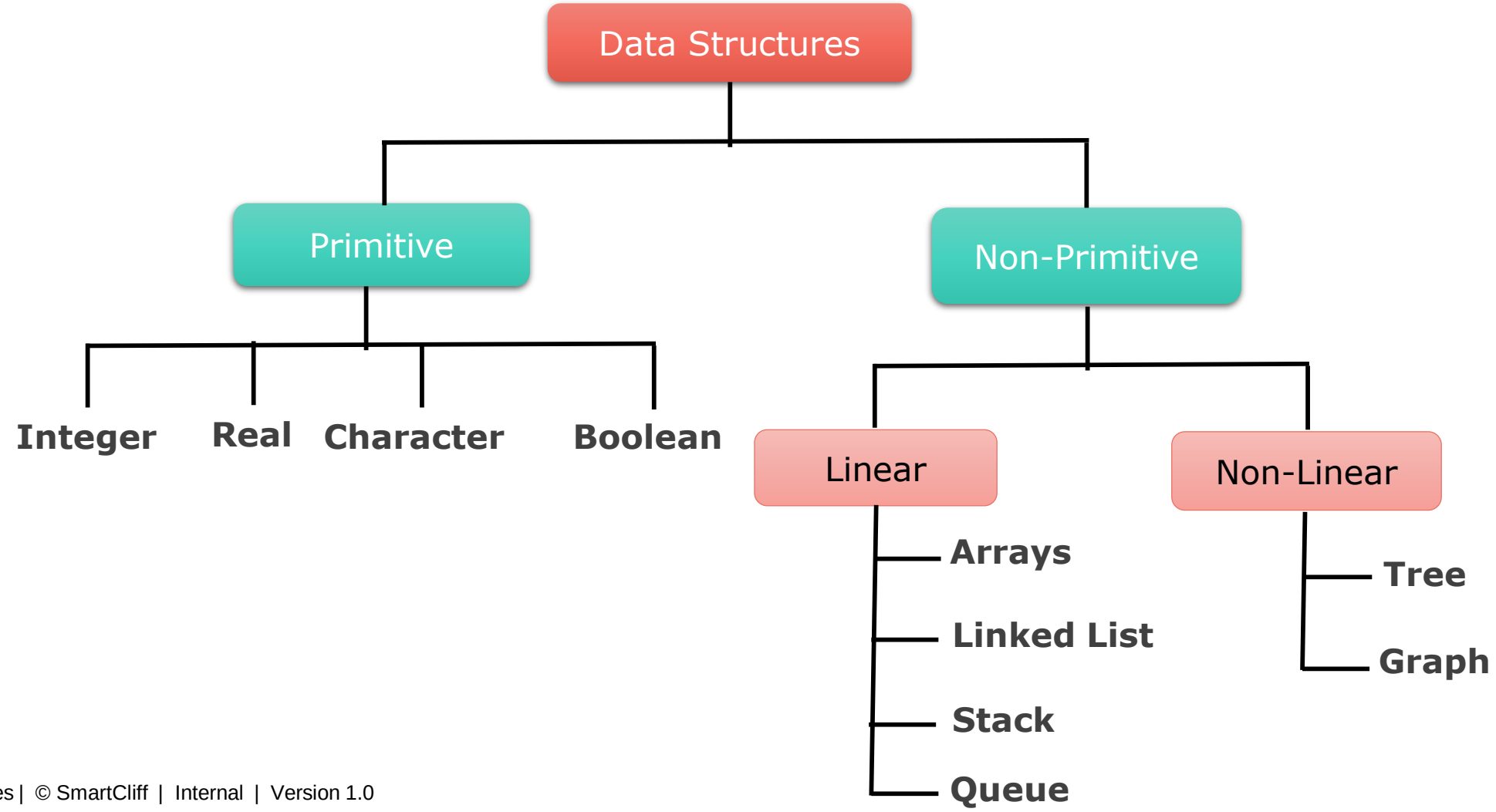


Oil can't be stored in paper bags

What is Data Structures?

- Data structure is a **systematic way to organize data** in computer memory so that it can be accessed and used efficiently.
- In other words, Data Structure is the way of organizing, storing, and retrieving data and their **relationship with each other.**
- There are several kinds of data structures like **Array, stack, queue, linked list etc**.
- Each of them are suitable for **specific type of tasks**.

Classification of Data Structures



Classification of Data Structures

Primitive data structure

- Primary Data Structures or Primitive Data Structures are the **basic data structures that directly operate upon the machine instructions.**
- **Example:** Integer, Floating point numbers, Character constants, String constants, Pointers.

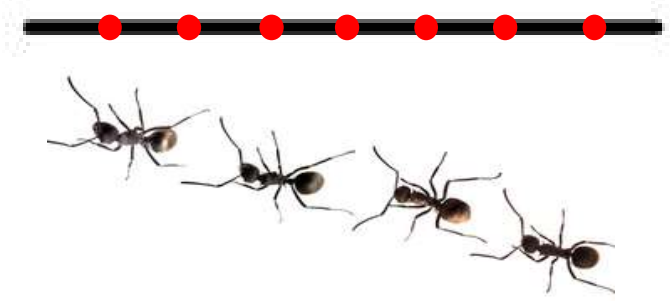
Non-primitive data structure

- Non primitive data structures or Secondary data structures are **derived from one or more primitive data structures.**
- The objective of creating non-primitive data structures is to form sets of **homogeneous or heterogeneous** data elements.
- It is categorized into two parts such **as linear data structure** and **non-linear data structure**.

Introduction to Data Structures

Linear data structure

- Linear data structure is a **sequential type** of data structure, and here sequential means that **all the elements in the memory are stored in a sequential manner**;
- **For example**, element stored after the second element would be the third element, the element stored after the third element would be the fourth element and so on.
- Linear data structures **holding the sequential values** such as **Array, Linked list, Stack, Queue.**

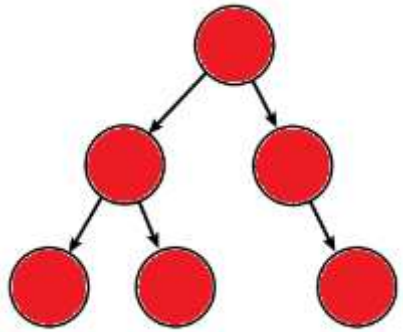


Ant go in line

Introduction to Data Structures

Non-linear data structure

- Non-linear data structure is a kind of **random type** of data structure.
- The non-linear data structures are **Tree** and **Graph.**



Tree



Chess pieces move in random order

ADT

- An **abstract data type** is an **abstraction of a data structure** that provides only the interface to which the data structure must adhere.
- The **interface** does not give any specific details about something should be implemented or in what programming language.
- The definition of ADT only mentions **what operations are to be performed** but **not how these operations will be implemented.**
- It does not specify **how data will be organized in memory** and **what algorithms** will be used for implementing the operations.
- It is called **“abstract”** because it gives an **implementation-independent view**.
- The process of providing only the essentials and hiding the details is known as **abstraction.**

ADT

- **ADT** is a collection of **data** together with a **set of operations** on that data.

ADT = Data + Operations

Definition :

Abstract Data type (ADT) is defined as a **mathematical model** with a **collection of operations** defined on that model.

ADT

For example, If we consider the smart phone.

- We look at the high specifications of the smart phone, such as:
- 128 GB RAM, Snapdragon 8 generation processor
- 6.5 inch LCD screen, Dual camera, Android 11.0

- The above specifications of the smartphone are the data, and we can also perform the following **operations on the smartphone**:
  - **call():** We can call through the smartphone.
  - **text():** We can text a message.
  - **photo():** We can click a photo.
  - **video():** We can also make a video.
- The smartphone is an entity whose **data or specifications and operations** are given above. They are **abstract and the implementation is hidden from user**.

ADT Types

Simple ADTs

- Predefined data types are Simple ADTs.
- **Example**: int, float, double, char, bool
- Consider **Integer ADT**:
- This type of ADT is an integer with **predefined ranges**.
- **Several operations** that can be applied to this data type (addition, subtraction, multiplication, division and so on)

Integer ADT
….. -3,-2,-1,0,1,2,3….

Integer ADT

Data
- Number
- Positive / Negative

Operations
- Addition
- Subtraction
- Multiplication
- Division

ADT Types

Complex ADT:

- While several simple ADTs have been implemented, such as integer, real, character, pointer and so on, and are available for use in most languages, **many useful complex ADTs are not.**
- We need some additional ADTs that should be created and stored in the library of the computer to be used.
- **Example**: List, Set, Graph, Stack, Queue, etc.,

Relation of algorithm, data structure and program

Algorithms + Data Structures = Programs

- A computer program is a **set of instructions** for a computer. These instructions **describe the steps** that the computer must follow to implement a plan.
- An **algorithm is a plan for solving a problem**.
  - A person must **design an algorithm**.
  - A person must translate an algorithm into a computer program.

Relation of algorithm, data structure and program

Algorithms + Data Structures = Programs

- **Data structures and algorithms** are both interrelated and complement each other in programming.
- An algorithm applies to a **particular or right data structure** while solving the problem
  - It helps to drastically improve the performance of an algorithm.
- We can’t detach data structures and algorithms.
- Data Structures are the programmatic way of storing data so that data can be used efficiently.
- Algorithms in most cases need data structures internally to make them work as intended.
- Both data structures and algorithms needed while coding in any programming language.

Relation of algorithm, data structure and program

- As applications are becoming more **complex** and the amount of data is **increasing day by day**, which may cause problems with
  - processing speed,
  - searching data,
  - handling multiple requests etc.
- Data structure provides a way of **organizing**, **managing**, and **storing data efficiently**.
- With the help of data structure, the **data items can be traversed easily**.
- Data structure provides **efficiency**, **reusability** and **abstraction**.
- It plays an important role in **enhancing the performance** of a program,
  - the main function of the program is to store and retrieve the user's data as fast as possible.

Why We Need Data Structures?

Visualize the need of Data Structures in known applications



Search Engine

- When you provide a search term in Google and click on search button, you get the search results in a fraction of second.
- Google handles 3.8 million searches per minute on average across the globe.
- Just Imagine, How many web pages Google goes through before giving you the search results?

Why We Need Data Structures?

Visualize the need of Data Structures in known applications



- There are three basic components such as

1. **Crawler**: To collect new web pages from the net

2. **Index**: To ensure super-quick retrieval of required web pages

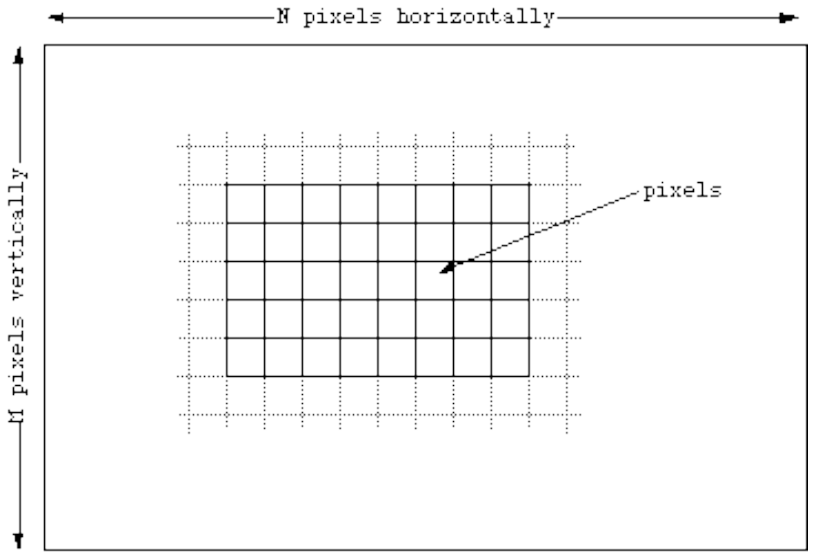
3. **Query Processor**: To provide an easy interface for user to query

- To effectively and efficiently perform searching task crawler can be used **Hash set** data structure.

- The Indexing can be used in **inverted index** and query processing can be used in **regex library**.

Why We Need Data Structures?

Visualize the need of Data Structures in known applications



Multimedia Data Storage

- You need to store an image (1000 X 1000 pixels) as a bitmap.

How it is possible?

- For this scenario, the data should be in organized manner to retrieve image data efficiently.
- Basic element of picture is pixels. These values are stored in a matrix format which consists set of rows and columns. Array data structure is basically used to handle images.

Why We Need Data Structures?

Visualize the need of Data Structures in known applications



Undo / Redo Operation in Automation Tool

- You need to store undo/redo operations in a word processor.

How is it done?

- To implement this task, the system should keep track of recently stored data in a memory.
- **Stack** is a restricted list, used to solve these kind of problem.

Why We Need Data Structures?

Visualize the need of Data Structures in known applications





Social Media

- You have to store social network “feeds".
- You do not know the size, and things may need to be dynamically added.
- When you put a post (comments) in Facebook / Twitter, irrespective of how many friends you are connected to, the post is immediately reflected in all their accounts.
- Facebook users create 3.2 billion likes and comments every day.
- Just Imagine How Facebook / Twitter keeps track of all your friends and update their pages based on your posts

Why We Need Data Structures?

Visualize the need of Data Structures in known applications





Information Retrieval

- To store a set of fixed key words which are referenced very frequently.

Just Imagine, How it is done?

- An ordered list of words is an attractive data structure when implementing autocomplete task.
- **Tries / Prefix tree** is used to solve these kind of problems

Why We Need Data Structures?

Visualize the need of Data Structures in known applications



Printer Spooler

- To implement printer spooler (Software that manages sending jobs to the printer) so that jobs can be printed in the order of their arrival.

Just Imagine, How it is done?

- The printer tasks are usually stored in a memory which uses **queue** data structure.
- **Queue follows the principle of First In First Out (FIFO)**

Why We Need Data Structures?

Visualize the need of Data Structures in known applications



Location Maps

- When you give a source and destination in Google Maps and click on find route, you get several results based on time, current traffic etc. in a fraction of second.
- More than 2 Billion miles is the distance that Google Maps Navigation guides users per year.

  Just think, How Google maps is able to quickly find locations and routes?

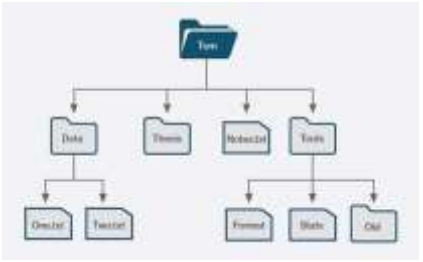
- The location information forms a network, hence **Graph** data structure can be used to keep track of location data.

Why We Need Data Structures?











Key Benefits of Data Structures

1. Every Data Structure permits data to be stored in specific ordered manner.

2. Data Structures consent to efficient data search and retrieval.

3. It allows to manage huge amount of data such as large databases and indexing services such as hash table.

4. Data structures can be used to organize the storage and retrieval of information stored in both main memory and secondary memory.

Quiz



1) A mathematical-model with a collection of operations defined on that model is called

a) Data structures

b) Abstract Data Type

c) Primitive Data Type

d) Algorithm

Answer : Option b)

Quiz



2) Which one of the following is the example for Nonlinear Data Structure?

a) Stack

b) Queue

c) Tree

d) LinkedList

Answer : Option c)

Quiz



3) An ADT is defined to be a mathematical model of a user-defined type along with the collection of all _____ operations on that model

a) Cardinality

b) Assignment

c) Primitive

d) Structured

Answer : Option c)

Quiz



4) Which of the following Linear data structures doesn't hold the sequential values.

a) Array

b) Linkedlist

c) Tree

d) Stack and Queue

Answer : Option c)

Quiz



5) Which of the data structure application used to store a set of fixed key words which are referenced very frequently.

a) Printer Spooler

b) Information Retrieval

c) Social media

d) Automation Tool

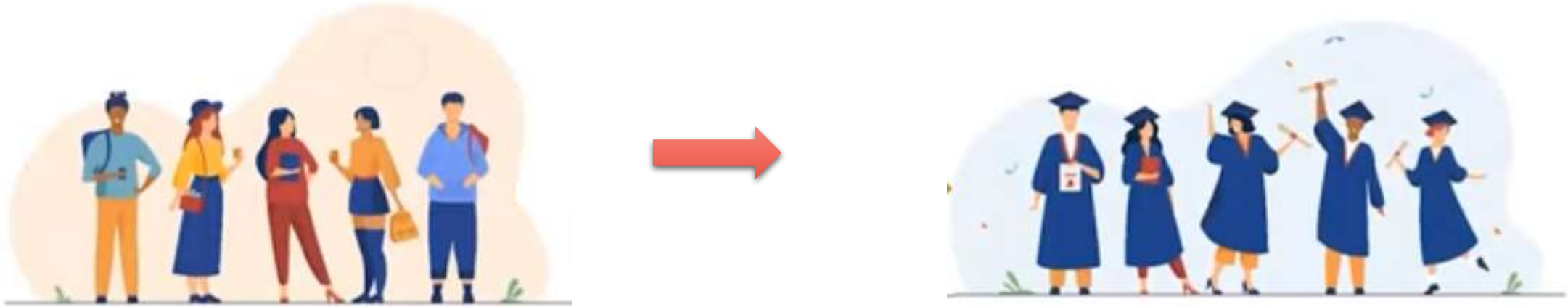
Answer : Option b)

Arrays



Why We Need Array?

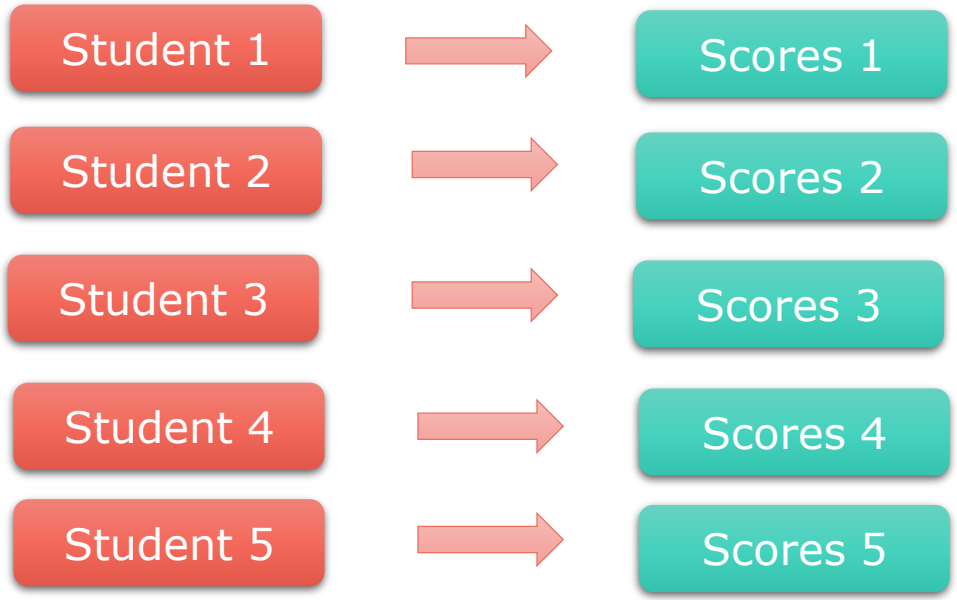
- **Let us imagine** that we were supposed to keep **track of students marks and honor them with degrees**.



Why We Need Array?

- If this was done in the traditional way, then we will declare **individual variables for each student**.

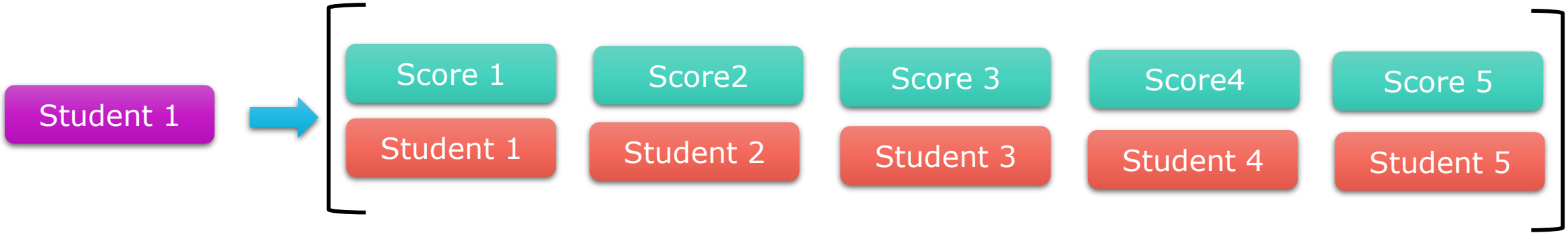




- This process is **time-consuming if the number of students are increase**
- **So, what can we do???**

Why We Need Array?

- What if there is an option where one variable could do the **job of ‘n’ variables**?



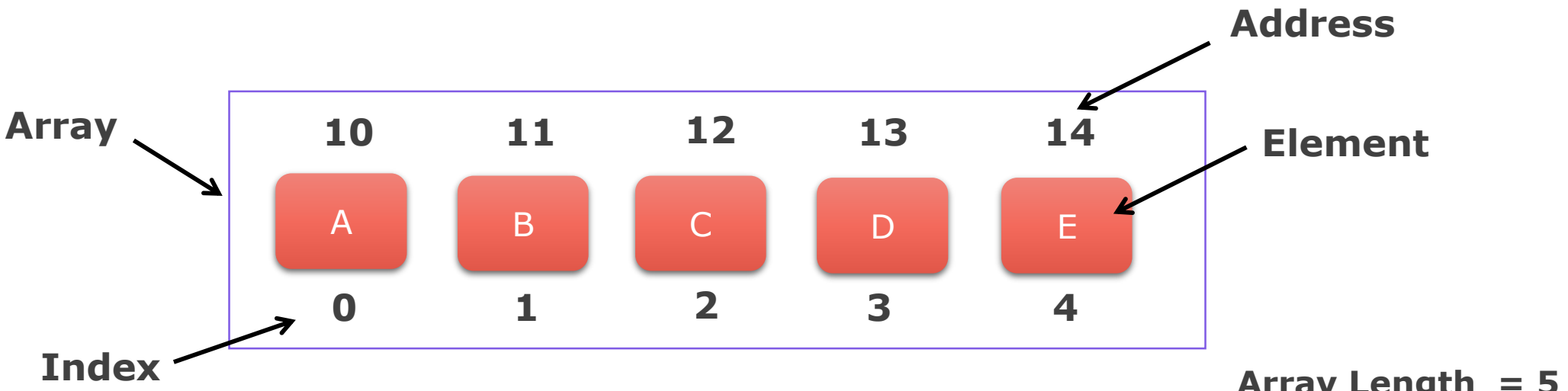
- So , with similar approach, **Array Data Structure were designed.**

What are Arrays?

- An a**rray** is a **linear data structure** that collects **elements of same data type under the common name** and stores them in a **contiguous and adjacent memory locations.**
- Array is the simplest data structure where **each data element can be randomly accessed** by using its **index number.**
- The elements stored in an array can be anything from **primitives types** such as integer, real, character or Boolean.
- Length of the **array is fixed** and index **starts with 0. Can’t access** the elements **beyond the array limit.**

Why We Need Array?

- Let's imagine that we wanted **create array of 5 elements**



- The elements in an array are stored using memory block address and Array index

Array Operations

- The **basic operations** supported by an array:
  - **Traverse** − print all the array elements one by one.
  - **Insertion** − Adds an element at the given index.
  - **Deletion** − Deletes an element at the given index.
  - **Search** − Searches an element using the given index or by the value.
  - **Update** − Updates an element at the given index.
  - **etc.**

Array Operations

Inserting an element at particular index in an array

1. Consider array with 5 indices from 0 to 4





2. Each index is holding a fruit: watermelon, strawberry, mango, banana, guava





3. Suppose we want add pineapple from the basket at the index 2

Array Operations

4. First we need to make index 2 free, so we will shift mango, banana , guava to right side.

5. Add, new index 5 to shift the fruits



6. Shift guava from index 4 to index 5

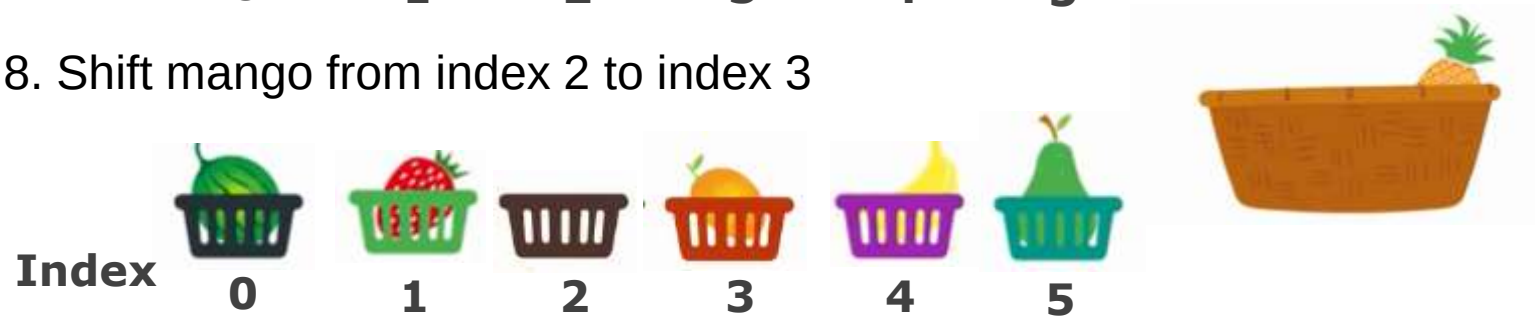


Array Operations

7. Shift banana from index 3 to index 4



8. Shift mango from index 2 to index 3



9. Now, Pineapple in the basket can be inserted in index 2



Array Operations

Remove an element from an array

1. Consider array with 5 indices from 0 to 4



2. Each index is holding a fruit: watermelon, strawberry, mango, banana, guava



3. Suppose we want to remove mango which is at the index 2

Array Operations

4. First we need to find where mango in the array. Once found remove mango from index 2



5. Now, index 2 is free.



6. Shift banana from index 3 to index 2





Array Operations

7. Shift guava from index 4 to index 3



8. Now, index 4 is free and we no need anymo



Array Operations

Searching an element in an array

1. Consider array with 5 indices from 0 to 4



2. Each index is holding a fruit: Watermelon, Strawberry, Mango, Banana, Guava



Array Operations

3. Suppose we need to find whether mango is found in the array. So key = Mango

4. We need to compare the key and elements in the array.

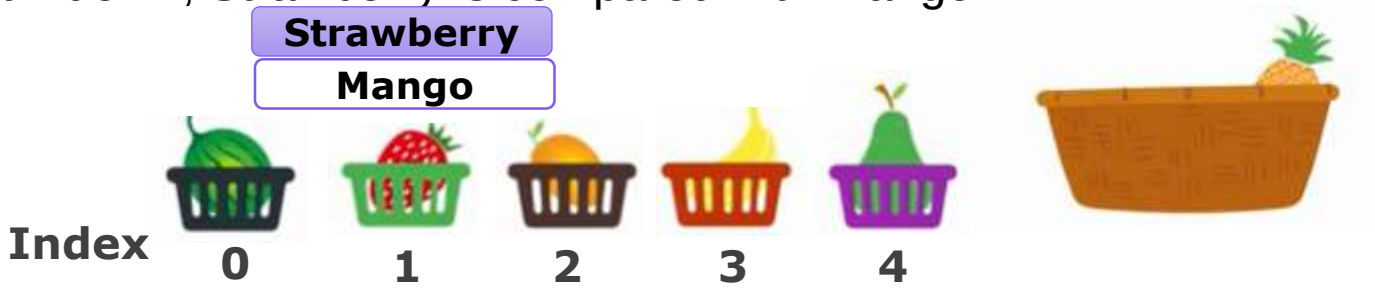
5. First, start from index 0. Watermelon is compared with Mango



6. Its not same so move on to next index 1.

Array Operations

7. At index 1, Strawberry is compared with Mango



6. Its not same so move on to next index 2.

7. At index 2, we found the key element.



Array Operations

Imagine that key element is not found in the array

1. Suppose we need to find whether pineapple is found in the array. So key = Pineapple



2. Start to compare from index 0. Watermelon is compared with Pineapple



Array Operations

3. Its not same so move on to next index 1. At index 1, Strawberry is compared with Pineapple



4. Its not same so move on to next index 2. At index 2, Mango is compared with Pineapple



Array Operations

5. Its not same so move on to next index 3. At index 3, Banana is compared with Pineapple



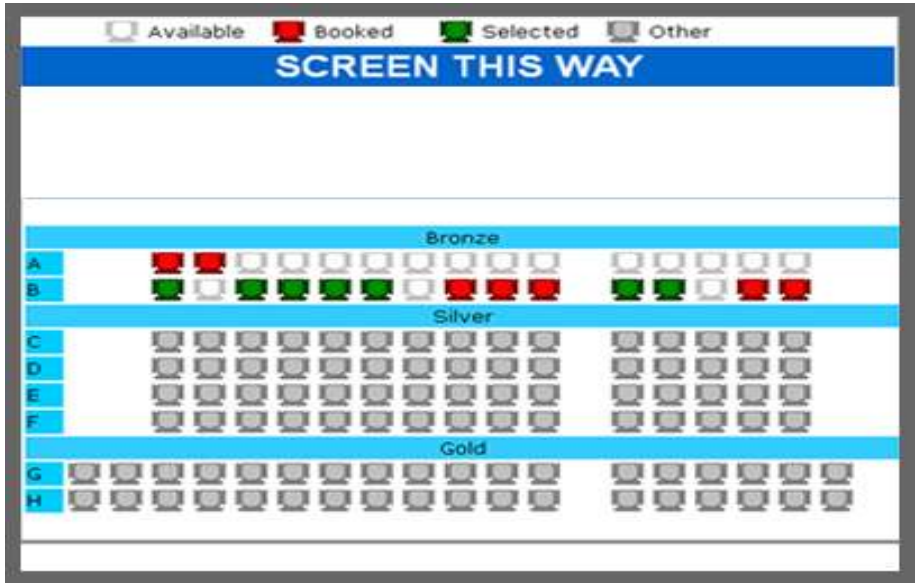
6. Its not same so move on to next index 4. At index 4, Guava is compared with Pineapple



7. Array is completely visited and the key element pineapple is not found

Real Time Applications

Online ticket booking system



- If a user wants to book a seat in C-4, the array becomes seat[C][4] or seat[3][4]

Real Time Applications

Gaming system



- To store the score and arrange them in descending order to clearly make out the rank of each player in the game

Advantages

- Arrays represent multiple data items of the same type using a single name.
- Elements can be accessed randomly by using the index number.
- Arrays allocate memory in **contiguous memory locations** for all its elements.
    - There is no chance of extra memory being allocated in case of arrays.
    - This avoids memory overflow or shortage of memory in arrays.

Disadvantages

- The **number of elements** to be stored in an array should be **known in advance**.
- An array is a **static structure** (which means the array is of fixed size).
    - Once declared the size of the array cannot be modified.
    - The memory which is allocated to it **cannot be increased or decreased**.
- **Insertion and deletion** are quite difficult in an array as the elements are stored in consecutive memory locations and the shifting **operation is costly**.
- Allocating more memory than the requirement leads to **wastage of memory space** and less allocation of memory also leads to a problem.

Quiz



1) Objects in a sequence that have the same type, is called

a) Algorithm

b) Arrays

c) Operators

d) Functions

Answer : Option b)

Quiz



2) How can we describe an array in the best possible way?

a) A data structure that shows a hierarchical behavior

b) Container of objects of similar types

c) Arrays are immutable once initialized

d) Array is not a data structure

Answer : Option b)

Quiz



3) In general, the index of the first element in an array is _____

a) 0

b) -1

c) 1

d) 2

Answer : Option a)

Quiz



4) Elements in an array are accessed _____

a) Randomly

b) Sequentially

c) Exponentially

d) Both a) and b)

Answer : Option d)

Quiz



5) Which of the following is the advantages of using an array?

a) Objects of mixed data types can be stored

b) Elements in an array cannot be sorted

c) Index of first element of an array is 1

d) Easier to store elements of same data type

Answer : Option d)

Sorting



Introduction

- Sorting is the **process of arranging data** into meaningful order so that we can analyze it more effectively.
- Sorting can be done in **ascending** and **descending order.**
- **Arranging items** of the **same kind, class or nature** in some ordered sequence.

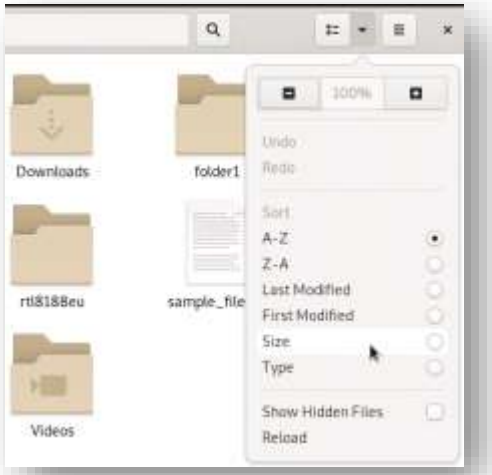
  A = { 3 1 6 2 1 3 4 5 9 0 } A = { 0 1 1 2 3 3 4 5 6 9 }

- Sorting techniques are differentiated by their **efficiency** and **space** requirements.

Introduction

Commonly used Examples

- Words in a dictionary are sorted
- Files in a directory are often listed in sorted order.
- The index of a book is sorted
- Sorting Books in Library
- Sorting Numbers (Sequential)
- etc.







Introduction

Need

- Efficient and quick data access
  - Sorting is the **basic solution** for many **complex problem**.
- Efficient data manipulation

Note:

- In Most organization, more than 25% of computation time is spent on sorting

Introduction

Types of sorting

Internal Sorting	External Sorting
Internal sorting deals with sorting the data stored in the computer's memory.	**External sorting** deals with sorting the data stored in files. External sorting is applied when there is voluminous data that cannot be stored in the memory.
For smaller amount of data	For massive amounts of data
Takes place entirely within main memory	Takes place on data that reside in external memory
Example: Insertion Sort, Bubble Sort, Selection sort, etc.,	**Example**: External Merge Sort

Sorting techniques

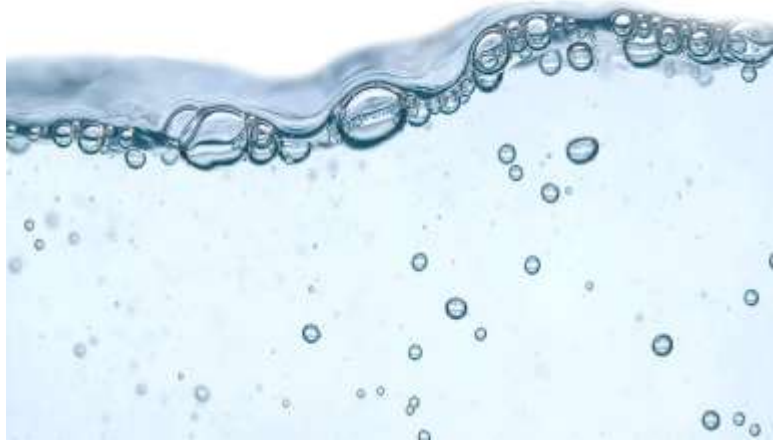
- A **sorting algorithm** is an algorithm that places elements of a list in certain order.
- The most used orders are **numerical order** and **lexicographical order**.

Different Types of Sorting algorithms:

- Bubble Sort
- Insertion Sort
- Selection Sort
- Merge Sort
- etc.

Sorting techniques: Bubble Sort

Idea



- Bubble in water moves upward why?
- When a bubble moves upward the water from above will move downward to fill in the space left by the bubble.
- It is also known as **Exchange Sort/Sinking Sort**

Strategy

- Compare **adjacent elements** and exchange them if they are out of order
  - Comparing the first two elements, the second and third elements, and so on, will move the **largest (or smallest) elements to the end of the array**
  - Repeating this process will eventually sort the array into ascending (or descending) order
- **Bubble up**: After each pass the largest element is bubbled to its position.

Sorting techniques: Bubble Sort

Algorithm

- Compare each element (except the last one) with its neighbor to the right
  - If they are out of order, swap them
  - This puts the largest element at the very end
  - The last element is now in the correct and final place
- Compare each element (except the last two) with its neighbor to the right
  - If they are out of order, swap them
  - This puts the second largest element next to last
  - The last two elements are now in their correct and final places
- Continue as above until you have no unsorted elements on the left

```
BUBBLE_SORT(A, N)
Step 1: Repeat Step 2 For I = 1 to N-1
Step 2: Repeat For J = 1 to N - I
Step 3:          IF A[J] > A[J + 1]
                 SWAP   A[J]   and A[J+1]
          [END OF INNER LOOP]
          [END OF OUTER LOOP]
Step 4: EXIT
```

Sorting techniques: Bubble Sort

Sort the following elements using Bubble sort: 7,9,1,4 and 2

Strategy:

- Traverse a collection of elements
  - Move from the front to the end
  - "Bubble" the largest value to the end using pair-wise comparisons and swapping

	Comparison
	Sorted Elements

Pass 1:

7	9	1	4	2	7 < 9 No Swap
7	9	1	4	2	9 > 1 Swap
7	1	9	4	2	9 > 4 Swap
7	1	4	9	2	9 > 2 Swap
7	1	4	2	9	

Sorting techniques: Bubble Sort

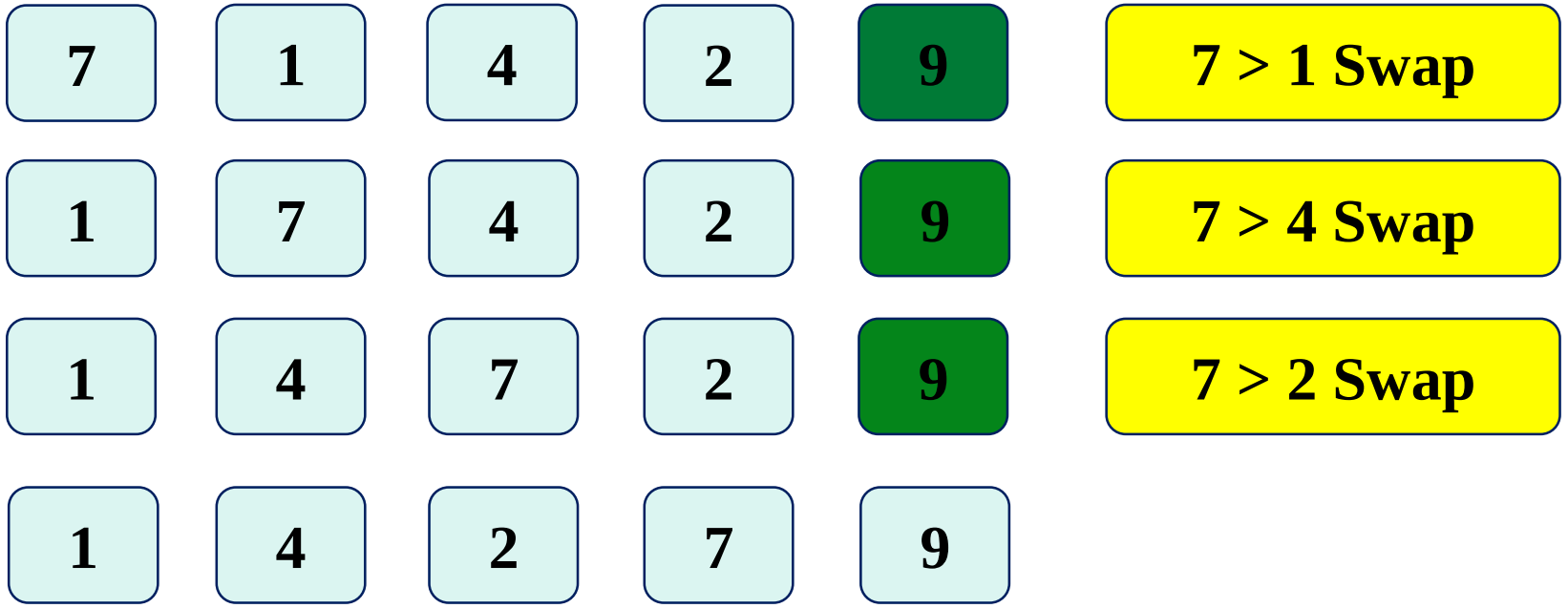
Sort the following elements using Bubble sort: 7,9,1,4 and 2

Strategy:

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping

	Comparison
	Sorted Elements

Pass 2:



Sorting techniques: Bubble Sort

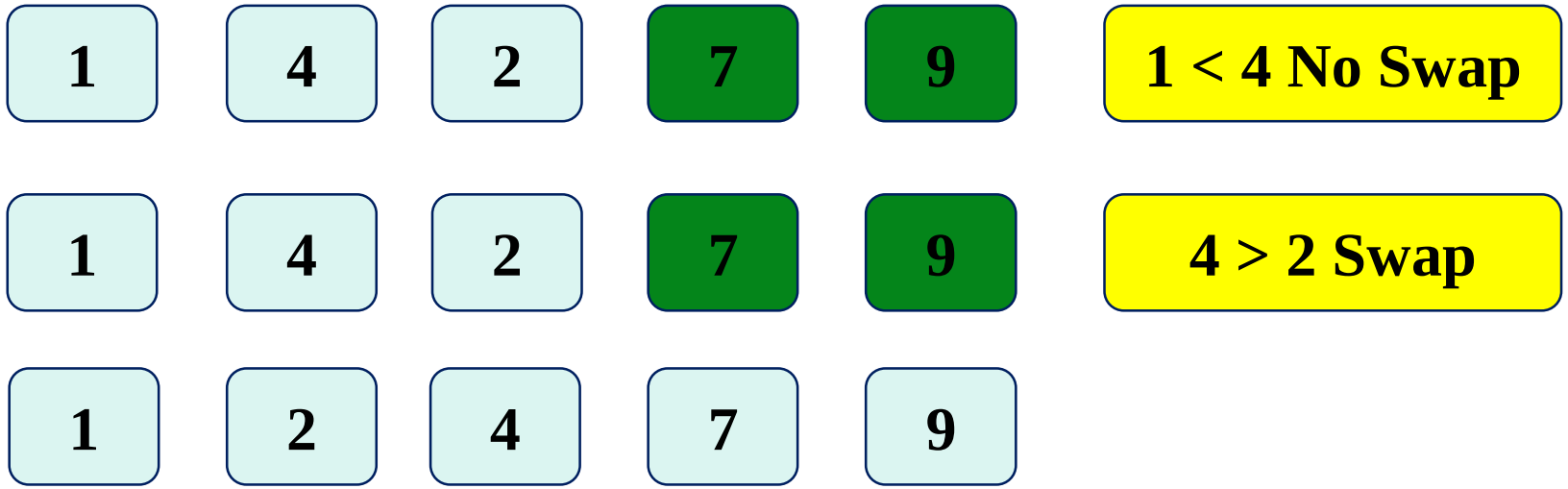
Sort the following elements using Bubble sort: 7,9,1,4 and 2

Strategy:

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping

	Comparison
	Sorted Elements

Pass 3:



Sorting techniques: Bubble Sort

Sort the following elements using Bubble sort: 7,9,1,4 and 2

Strategy:

- Traverse a collection of elements
  - Move from the front to the end
  - “Bubble” the largest value to the end using pair-wise comparisons and swapping

	Comparison
	Sorted Elements

Pass 4:





Sorted List :



Sorting techniques: Bubble Sort

Summary

- Bubble sort takes **$O(n^2)$** time.
- **Bubble sort** starts with first two elements.
- It compares the element to **check which one is greater**.
- This operation continues until the array is **sorted in ascending order**

Sorting techniques: Insertion Sort

Idea: Playing cards

- Start with an empty left hand and the cards facing down on the table.
- Remove one card at a time from the table, and insert it into the correct position in the left hand
- Compare it with each of the cards already in the hand, from right to left
- The cards held in the left hand are sorted
- These cards were originally the top cards of the pile on the table
- This process is repeated until all the cards are in the correct sequence.

Strategy

- Sorting a list by **inserting element** at **appropriate position**.
- The array elements are sequentially compared and then arranged in a **specific order simultaneously.**

Sorting techniques: Insertion Sort

Algorithm

Step 1: In every pass, an element is compared with all its previous elements.

Step 2: If at some point it is found that the element can be inserted at a position then space is created by shifting other elements one position to the right and insert the element at the suitable position.

Step 3: The steps are repeated for all elements in the array.

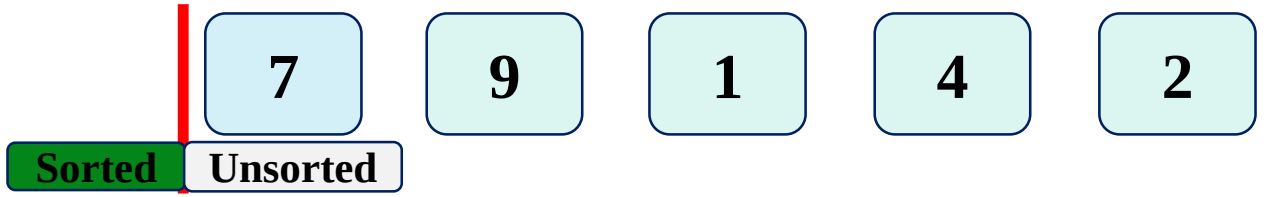
```
INSERTION-SORT (ARR, N)
Step 1: Repeat Steps 2 to 5 for K = 1 to N – 1
Step 2: SET TEMP = ARR[K]
Step 3: SET J = K - 1
Step 4: Repeat while TEMP <= ARR[J]
            SET ARR[J + 1] = ARR[J]
            SET J = J - 1
        [END OF INNER LOOP]
Step 5: SET ARR[J + 1] = TEMP
        [END OF LOOP]
Step 6: EXIT
```

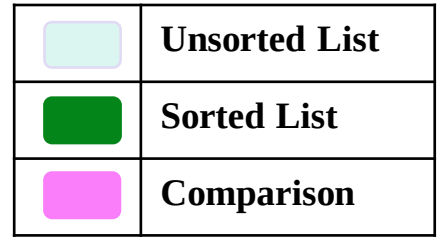
Sorting techniques: Insertion Sort

Sort the following elements using Insertion sort: 7,9,1,4 and 2

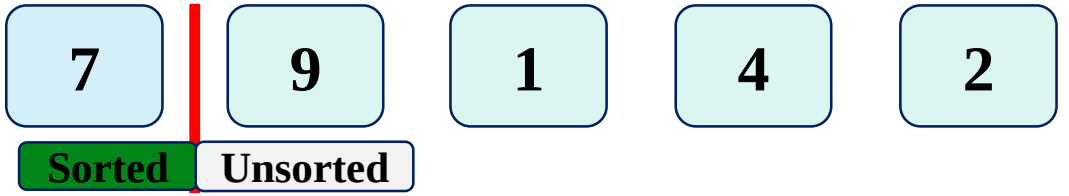
Strategy:

- A[0] is the only element in the sorted set.
- **In Pass 1 :** A[1] will be placed **either before** or **after A[0]**, so that the array A is sorted.
- **In Pass 2 :** A[2] will be placed **either before A[0]**, **in between A[0]** and **A[1]**, or **after A[1].**
- **In Pass N–1:** A[N–1] will be placed in its proper place to keep the array sorted.





Pass 1:



Sorting techniques: Insertion Sort

Pass 2:



Pass 3:



Pass 4:



	Unsorted List
	Sorted List
	Comparison

Sorting techniques: Insertion Sort

Summary

- **Insertion sort** works like the way we sort **playing cards** in our hands.
- It always starts with the **second element as key.**
- The key is compared with the elements ahead of it and is put it in the right place.
- Insertion sort runs in **O(n)** time in its **best case** and runs in **O(n2)** in its **worst and average cases.**

Sorting techniques: Selection Sort

Idea: Playing cards

- Find the smallest card. Swap it with the first card.
- Find the second-smallest card. Swap it with the second card.
- Find the third-smallest card. Swap it with the third card.
- Repeat finding the next-smallest card and swapping it into the correct position until the array is sorted.

Strategy

- The algorithm maintains two subarrays in each array.

  1) The subarray which is already **sorted.**

  2) Remaining subarray which is **unsorted.**

- It **selects** the **smallest element** from an unsorted list **in each iteration** and **places** that element at the **beginning of the unsorted list**.

Sorting techniques: Selection Sort

Algorithm

- The algorithm divides the input list into two parts:
- **A sorted sub list** of items which is built up from left to right at the front (left) of the list and a sub list of the remaining **unsorted items** that occupy the rest of the list.
- The procedures listed below are repeated until the unsorted subarray is empty:
- Choose the smallest element in the unsorted subarray.
- Swap it with the unsorted subarray's leftmost element.
- Unsorted subarray's leftmost element is now a portion (rightmost) of sorted subarray and will no longer be a component of unsorted subarray.

```
SELECTION SORT(ARR, N)
Step 1: Repeat Steps 2 and 3 for K = 1 to N-1
Step 2:    CALL SMALLEST(ARR, K, N, POS)
Step 3:    SWAP A[K] with ARR[POS]
         [END OF LOOP]
Step 4: EXIT

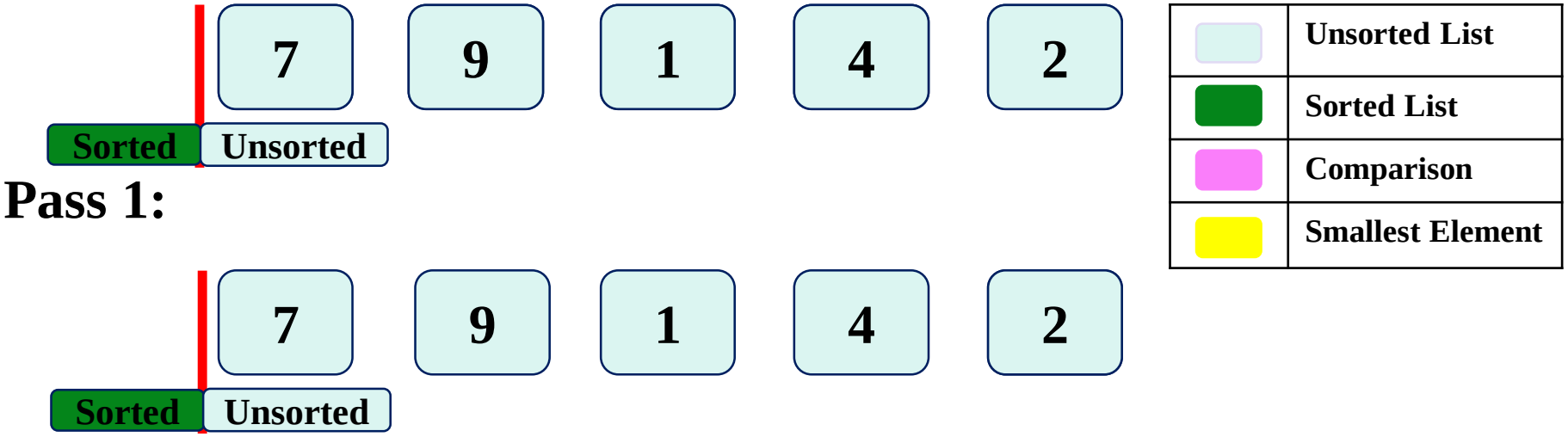
SMALLEST (ARR, K, N, POS)
Step 1: [INITIALIZE] SET SMALL = ARR[K]
Step 2: [INITIALIZE] SET POS = K
Step 3: Repeat for J = K+1 to N
                IF SMALL > ARR[J]
                   SET SMALL = ARR[J]
                   SET POS = J
                [END OF IF]
         [END OF LOOP]
Step 4: RETURN POS
```

Sorting techniques: Selection Sort

Sort the following elements using Selection sort: **7,9,1,4 and 2**

Strategy:

- First find the **smallest value** in the **unsorted array** and place it in the **first** position.
- Find the **second smallest value** in **unsorted array** and place it in the **second** position.
- Repeat until the entire array has been sorted.



Sorting techniques: Selection Sort

Sort the following elements using selection sort: 7,9,1,4,2

Pass 2:



	Unsorted List
	Sorted List
	Comparison
	Smallest Element

Pass 3:



Pass 4:



Sorting techniques: Selection Sort

Summary

- The **smallest element** is found in **first pass,** and it is placed at the first position.
- In **second pass**, smallest element is searched from the rest of the element excluding first element.
- Selection sort keeps doing this, until the array is sorted.
- The average and worst-case complexity of selection sort is **$O(n^2)$**

Sorting techniques: Merge Sort

Idea: Deck of cards

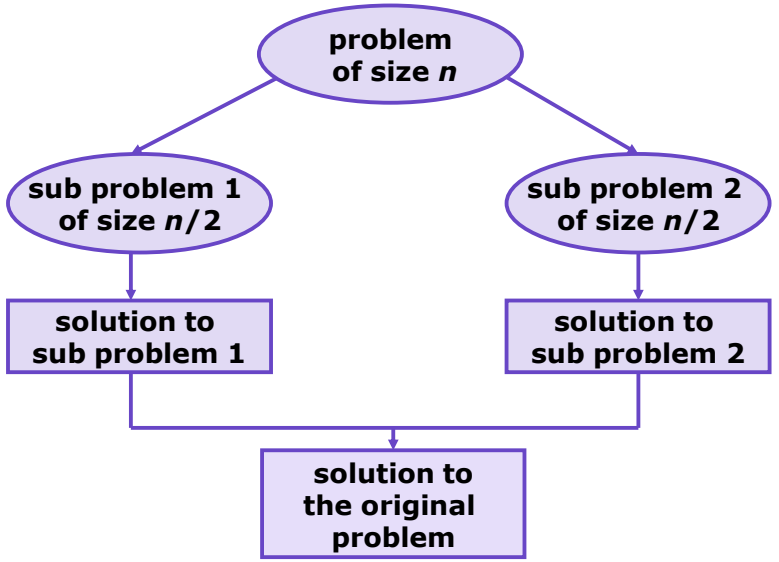
- Form two sub decks with about 10 cards each and sort them so that when they are face up the lowest cards are on top. Place both decks face up in front of you.
- Compare the top card from each deck and choose the lower one. Flip it over and add it to the merged deck.
- Repeat step two until one of the decks is empty.
- Then take the remaining cards and add them to the merged deck.

Sorting techniques: Merge Sort

Strategy

Sort a sequence of **n** elements into non-decreasing order. This algorithm is an example of **divide and conquer approach**.

- **Divide:** Divide the n-element sequence to be sorted into two subsequences of n/2 elements each
- **Conquer:** Sort the two subsequences recursively using merge sort.
- **Combine:** Merge the two sorted subsequences to produce the sorted answer.



Sorting techniques: Merge Sort

Algorithm

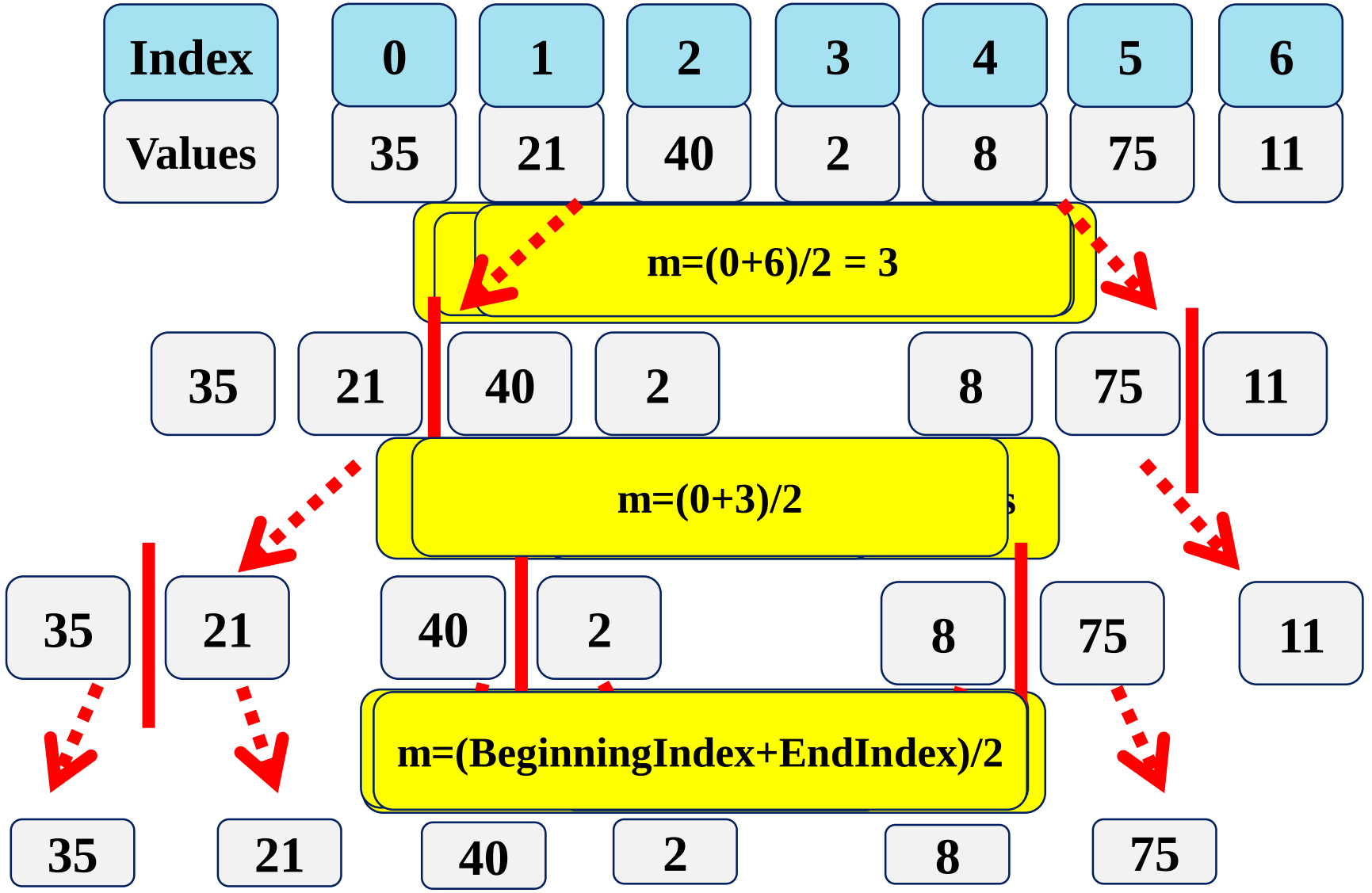
```
MERGE (ARR, BEG, MID, END)
Step 1: [INITIALIZE] SET I = BEG, J = MID + 1, INDEX = 0
Step 2: Repeat while (I <= MID) AND (J<=END)
            IF ARR[I] < ARR[J]
                SET TEMP[INDEX] = ARR[I]
                SET I = I + 1
            ELSE
                SET TEMP[INDEX] = ARR[J]
                SET J = J + 1
            [END OF IF]
            SET INDEX = INDEX + 1
        [END OF LOOP]
Step 3: [Copy the remaining elements of right sub-array]
            IF I > MID
                Repeat while J <= END
                    SET TEMP[INDEX] = ARR[J]
                    SET INDEX = INDEX + 1, SET J = J + 1
[END OF LOOP]
[Copy the remaining elements of left sub-array]
ELSE
    Repeat while I <= MID
        SET TEMP[INDEX] = ARR[I]
        SET INDEX = INDEX + 1, SET I = I + 1
    [END OF LOOP]
[END OF IF]
Step 4: [Copy the contents of TEMP back to ARR]
        SET K= 0
Step 5: Repeat while K < INDEX
        SET ARR[K] = TEMP[K]
        SET K = K + 1
        [END OF LOOP]
Step 6: END
```

Sorting techniques: Merge Sort

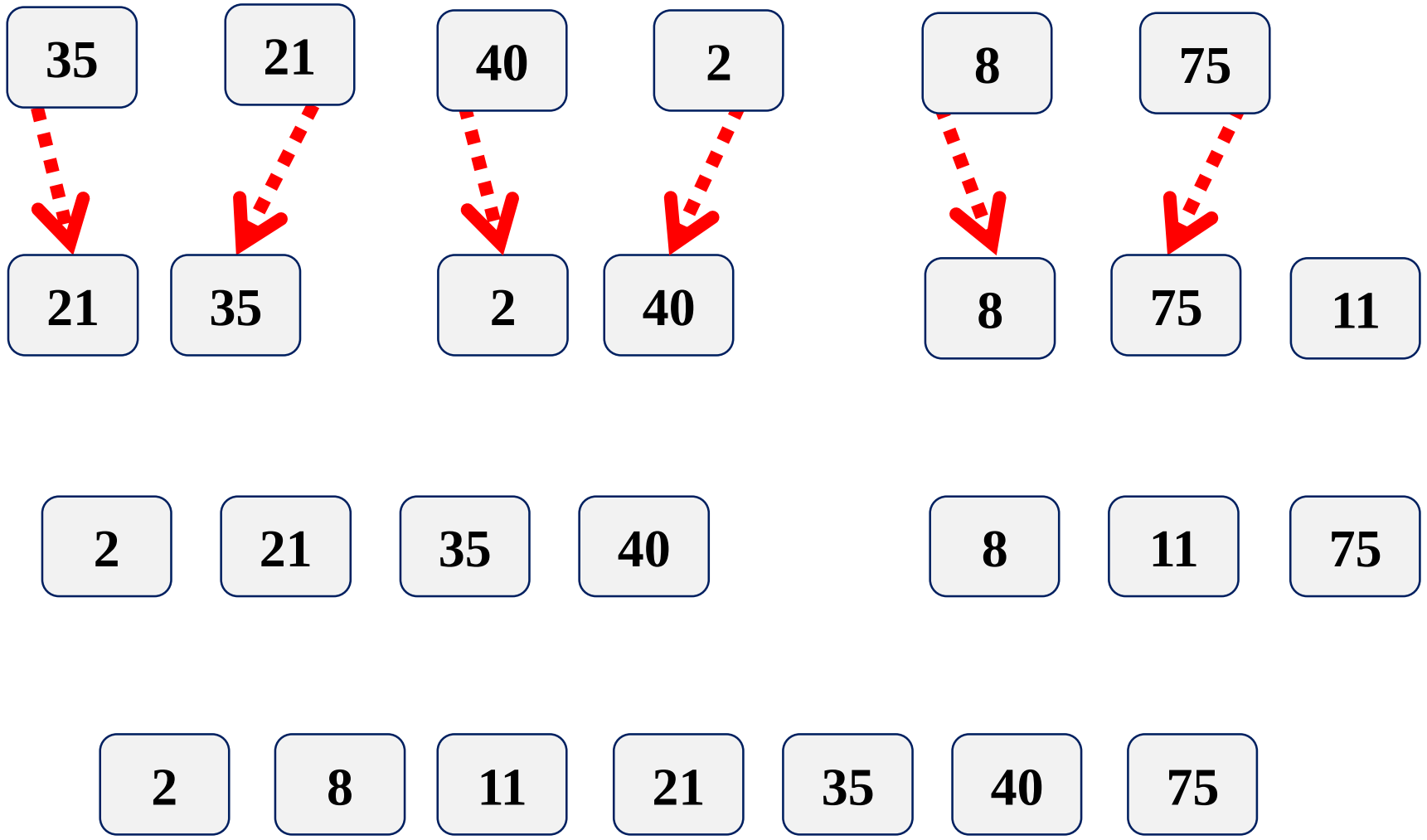
Algorithm

```
MERGE_SORT(ARR, BEG, END)
Step 1: IF BEG < END
                SET MID = (BEG + END)/2
                CALL MERGE_SORT (ARR, BEG, MID)
                CALL MERGE_SORT (ARR, MID + 1, END)
                MERGE (ARR, BEG, MID, END)
            [END OF IF]
Step 2: END
```

Sorting techniques: Merge Sort

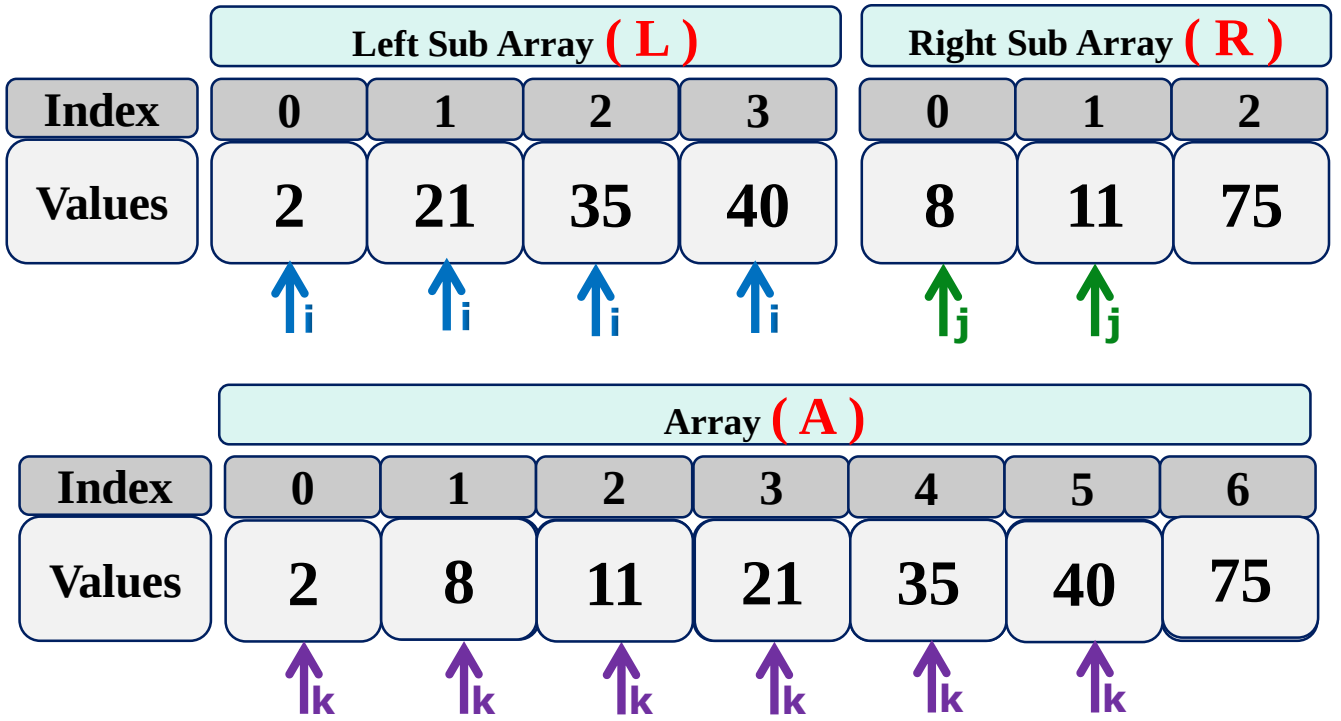


Sorting techniques: Merge Sort



Sorting techniques: Merge Sort

	Comparison
	Value Assigned



++i , ++k

i reached end

Sorting techniques: Merge Sort

Summary

- Merge Sort is a divide and conquer algorithm.
- It works by recursively breaking down a problem into two or more sub-problems of the same or related type, until these become simple enough to be solved directly.
- Merge Sort first divides the array into equal halves and then combines them in a sorted manner.
- Overall time complexity of Merge sort is **O(n logn).**
    - It is more efficient as it is in worst case also the runtime is **O(n logn)**

Quiz



1) If the given input array is sorted or nearly sorted, which of the following algorithm gives the best performance?

a) Insertion sort

b) Bubble sort

c) Selection sort

d) Merge sort

Answer : Option a)

Quiz



2) Which of the following sorting methods would be most suitable for sorting a list which is almost sorted?

a) Insertion sort

b) Bubble sort

c) Selection sort

d) Merge sort

Answer : Option b)

Quiz



3) Consider a situation where swap operation is very costly. Which of the following sorting algorithms should be preferred so that the number of swap operations are minimized in general?

a) Insertion sort

b) Bubble sort

c) Selection sort

d) Merge sort

Answer : Option c)

Quiz



4) Merge sort uses which of the following technique to implement sorting?

a) backtracking

b) greedy algorithm

c) divide and conquer

d) dynamic programming

Answer : Option c)

Quiz



5) Which of the following sorting algorithms maintain two sub-lists, one sorted and one to be sorted?

a) Insertion sort

b) Bubble sort

c) Selection sort

d) Both a) and c)

Answer : Option d)

Searching

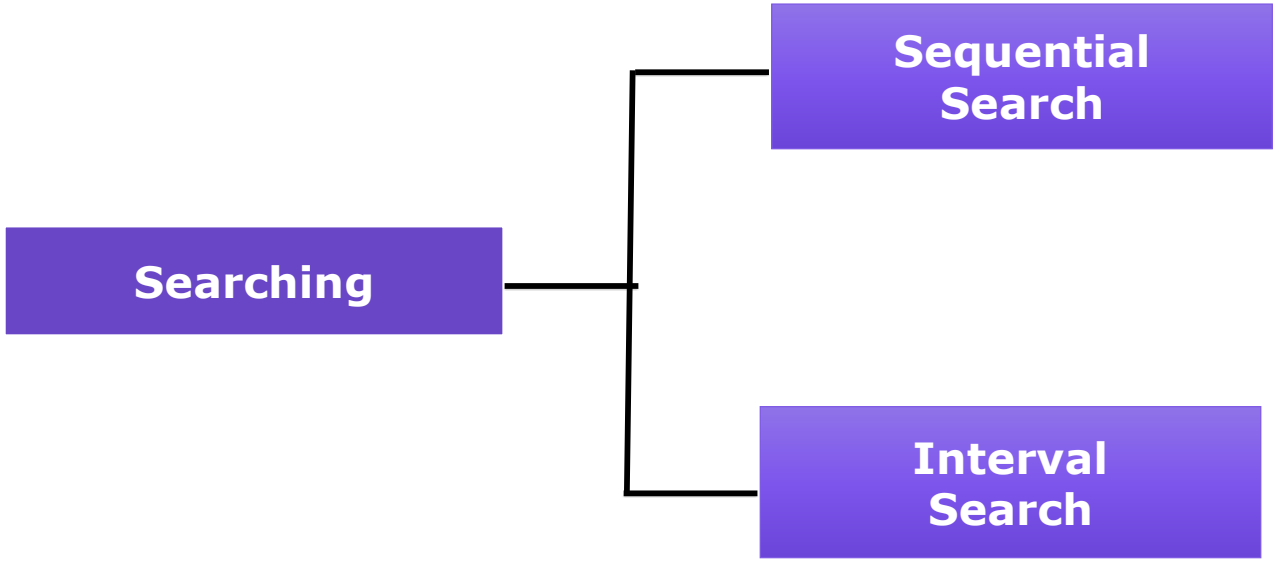


Searching

- **Searching** is the process of **finding a desired element** in set of items. The desired element is called **"target".**
- Any search is said to be **successful or unsuccessful** depending upon whether the element that is being **searched is found or not**.
- For **efficient data manipulation** need fast searching.

Searching Techniques

- There are **two commonly used searching algorithms**



Searching Techniques

Sequential Search

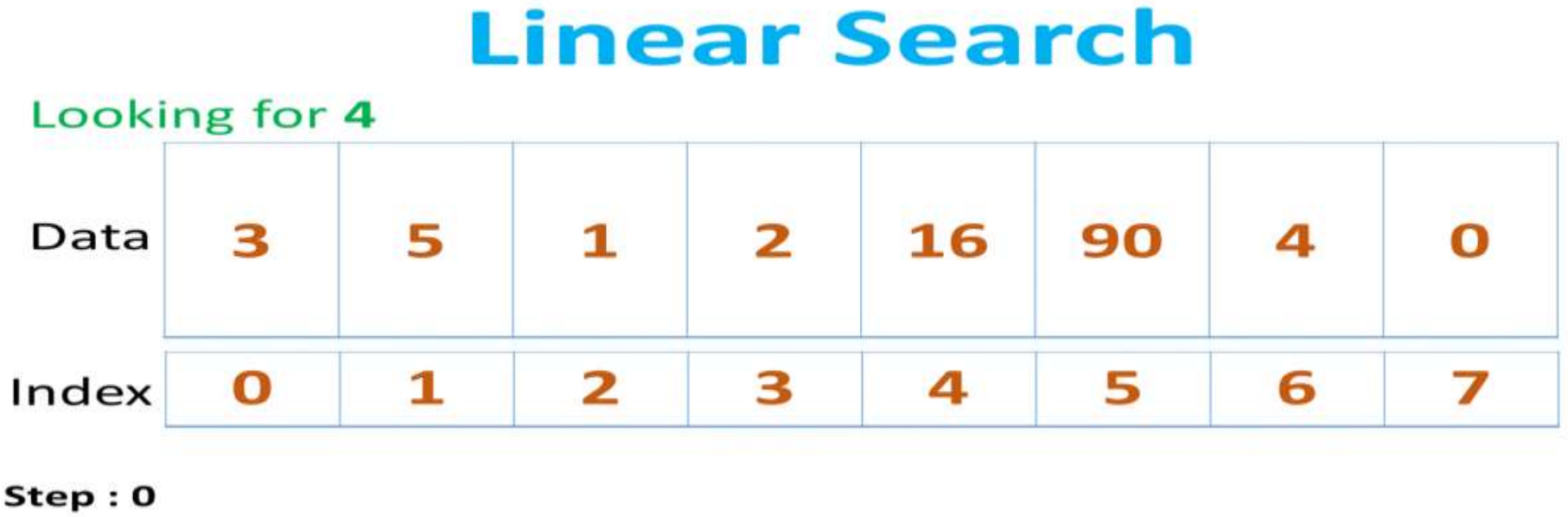
- In this, the list or array is **traversed sequentially,** and every element is checked.
- **For Example:** Linear Search.

Interval Search

- These algorithms are specifically designed for searching in **sorted data-structures**.
- These type of searching algorithms are much **more efficient than Linear Search** as they **repeatedly target the centre** of the search structure and **divide the search space in half.**
- **For Example:** Binary Search.

Searching Techniques: Sequential Search

- This is the **simplest method** for searching.
- In this technique of searching, finding a particular value in a list, that consists of **checking every one of its elements**, one at a time and in sequence, until the desired one is found
- This method can be performed on a **sorted or an unsorted list**.



Searching Techniques: Sequential Search

Algorithm

Step 1: Start the search from the first element and Check the key element with each element of list x.

Step 2: If element is found, return the index position of the key.

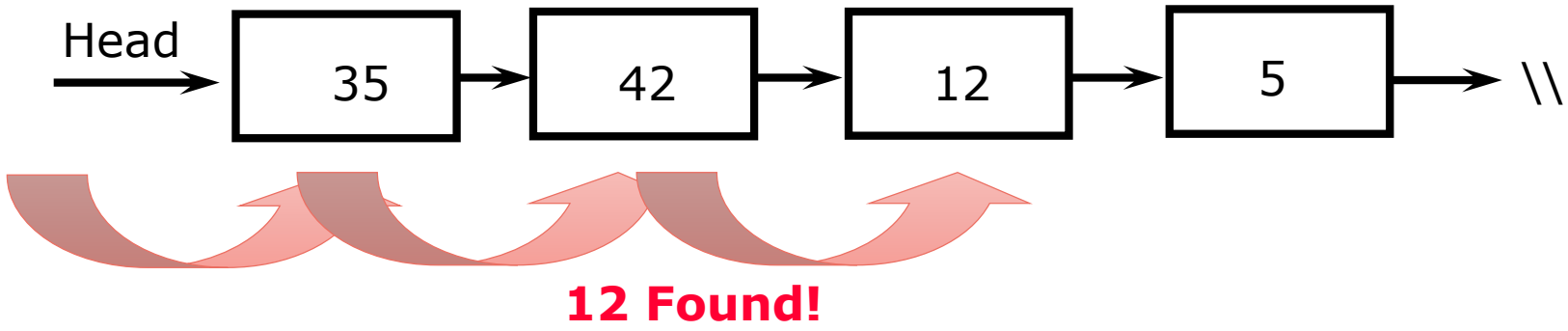
Step 3: If element is not found, print element is not present.

Searching Techniques: Sequential Search

Unsorted list

- Searching in case of unsorted list begins from the 0th element and continues until the element or the end of the list is reached.

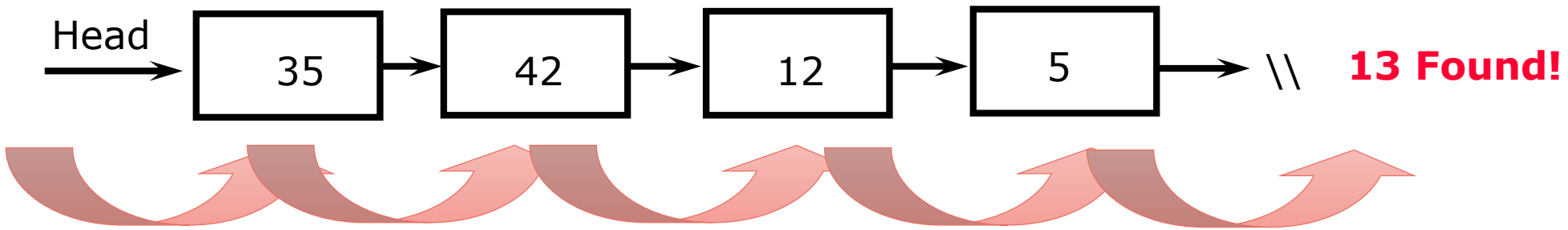
Example: Find 12



Searching Techniques: Sequential Search

Unsorted list

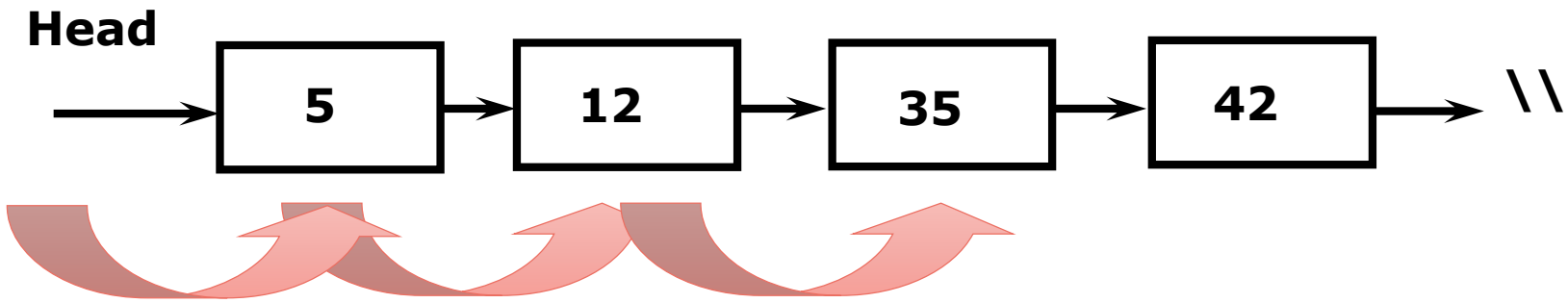
Example: Find 13



Searching Techniques: Sequential Search

Sorted list

Example: Find 13



13 Not Found!

Searching Techniques: Sequential Search

Time Complexity

- In linear search, **best-case complexity is O(1)** where the element is found at the first index.
- **Worst-case complexity is O(n)** where the element is found at the last index or element is not present in the array.

Advantages

- Will perform fast searches of small to medium lists.
- The list does not need to sorted
- Not affected by insertions and deletions.

Disadvantages

- Slow searching of large lists.

Searching Techniques: Binary Search

- Binary search is an **efficient algorithm** for finding an item from a **sorted list of items**.
- This algorithm is also an example of **divide and conquer approach.**
- The most essential thing of this method is it requires the **list to be in sorted order.**

Searching Techniques: Binary Search

Algorithm

Step 1 : Obtain the middle element

Step 2 : If it equals searched value, return position and go to step 4

Step 3 : Other wise, two cases possible.

Step 3.1: If search value < middle element, repeat the procedure for sub list before the middle element.

Step 3.2: If search value > middle element, repeat the procedure for sub list after the middle element.

Step 4 : Stop the algorithm.

Searching Techniques: Binary Search

Example: Binary search to Find(33) in the given list of elements.

Step 1 : Middle=(low+high)/2

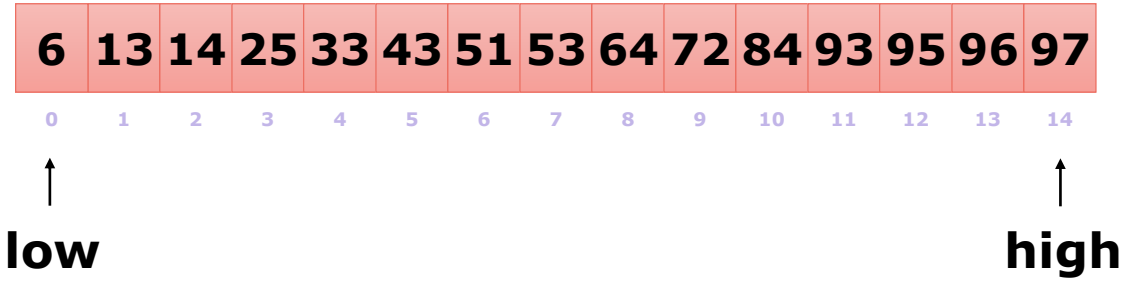
=(0+14)/2

=7

Initial value

low=0

high=n-1

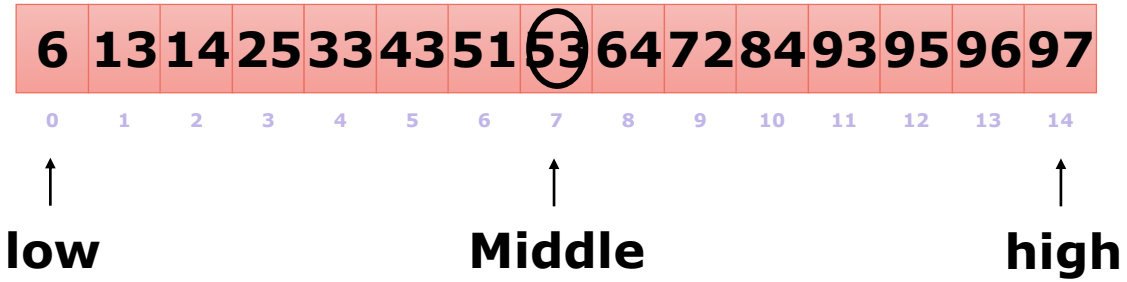


Searching Techniques: Binary Search

Middle=(low+high)/2

=(0+14)/2

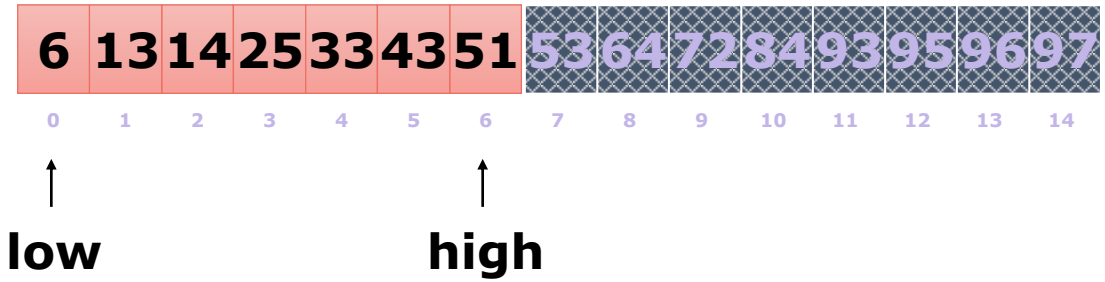
=7



Searching Techniques: Binary Search

Step 2: Binary search for Find(33).

- Middle Element >Find
- Check Lower Half i.e. before middle element
- Change high=middle-1



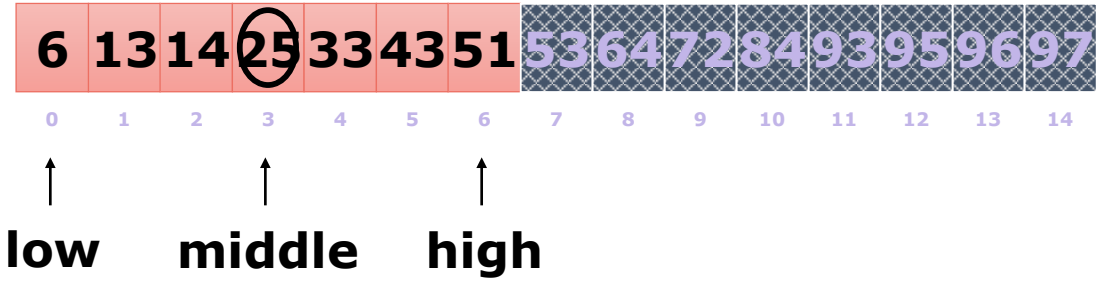
Searching Techniques: Binary Search

Step 3: Binary search for Find(33).

Middle=(low+high)/2

=(0+6)/2

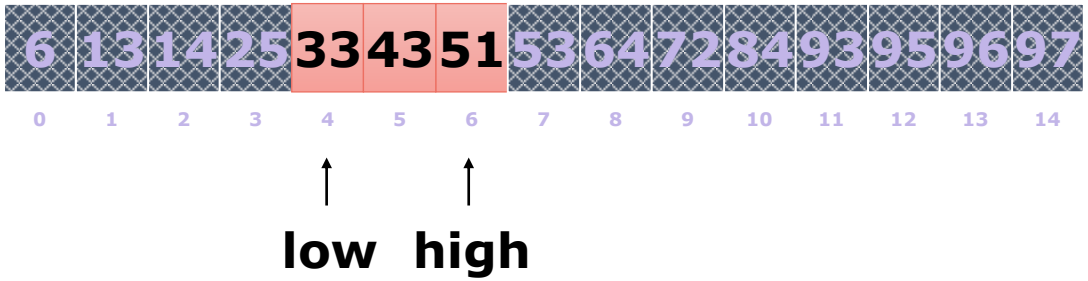
=3



Searching Techniques: Binary Search

Step 4: Binary search for Find(33).

- Middle Element <Find
- Check First Half of top half i.e. after middle element
- Change low=middle+1



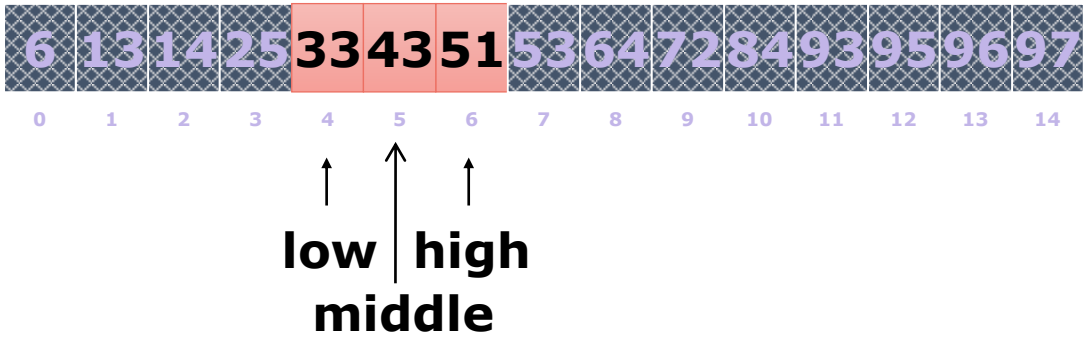
Searching Techniques: Binary Search

Step 5: Binary search for Find(33).

Middle=(low+high)/2

=(4+6)/2

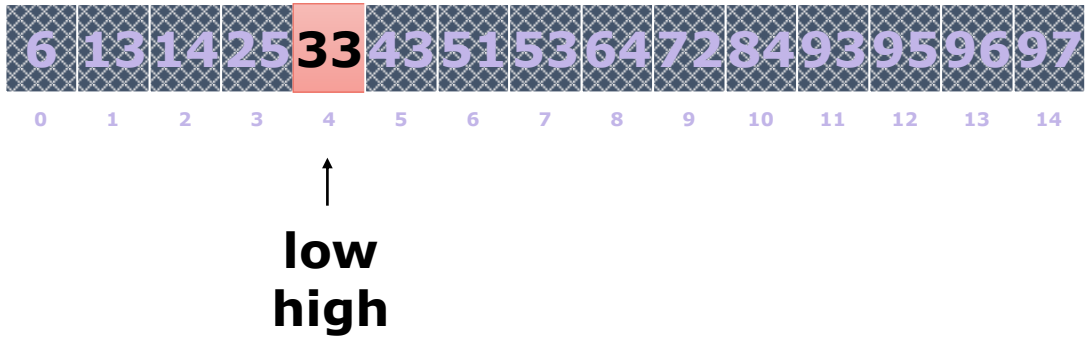
=5



Searching Techniques: Binary Search

Step 6: Binary search for Find(33).

- Middle Element >Find
- Check Lower Half of bottom half i.e. before middle element
- Change high=middle-1



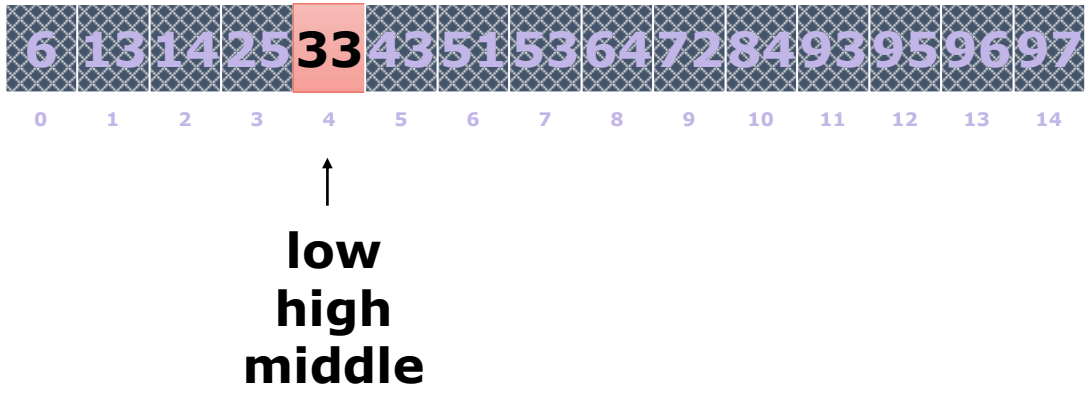
Searching Techniques: Binary Search

Step 7: Binary search for Find(33).

Middle=(low+high)/2

=(4+4)/2

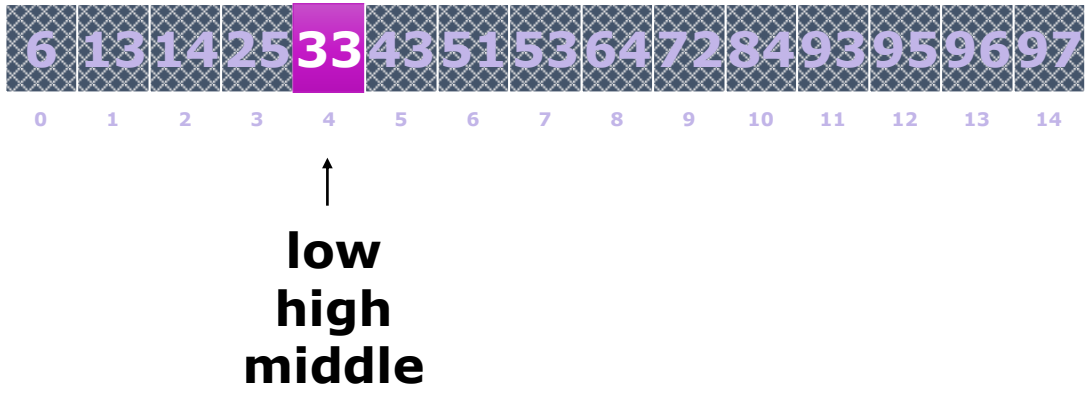
=4



Searching Techniques: Binary Search

Step 8: Binary search for Find(33).

- Middle==Find
- The element 33 is found



Searching Techniques: Binary Search

Time Complexity

- In binary search, **best-case complexity is O(1)** where the element is found at the middle index.
- The **worst-case complexity is O(log2n).**

Advantages

- It **eliminates half of the list** from further searching by using the result of **each comparison**. It indicates whether the element being searched is before or after the current position in the list.
- This information is used to **narrow the search**.
- For **large lists of data**, it works significantly **better** than linear search.

Disadvantages

- It employs **recursive approach** which requires **more stack space**.
- Programming binary search algorithm is **error prone and difficult**.
- The interaction of binary search with memory hierarchy i.e., caching is poor. (because of its random-access nature)

Sequential Search vs Binary Search

BASIS FOR COMPARISON	LINEAR SEARCH	BINARY SEARCH
Time Complexity	O(N)	O($\log_2 N$)
Best case time	First Element O(1)	Center Element O(1)
Prerequisite for an array	No required	Array must be in sorted order
Worst case for N number of elements	N comparisons are required	Can conclude after only $\log_2 N$ comparisons
Can be implemented on	Array and Linked list	Cannot be directly implemented on linked list
Insert operation	Easily inserted at the end of list	Require processing to insert at its proper place to maintain a sorted list.
Algorithm type	Iterative in nature	Divide and conquer in nature
Usefulness	Easy to use and no need for any ordered elements.	Anyhow tricky algorithm and elements should be organized in order.
Lines of Code	Less	More

Quiz



1) In which circumstances is linear searching used?

a) When the list has only a few elements

b) When performing a single search in an unordered list

c) Used all the time

d) When the list has only a few elements and When performing a single search in an unordered list

Answer : Option d)

Quiz



2) What is the best case and worst case complexity of ordered linear search?

a) O(nlogn), O(logn)

b) O(logn), O(nlogn)

c) O(n), O(1)

d) O(1), O(n)

Answer : Option d)

Quiz



3) Which of the following is a disadvantage of linear search?

a) Requires more space

b) Greater time complexities compared to other searching algorithms

c) Not easy to understand

d) Not easy to implement

Answer : Option b)

Quiz



4) Binary Search can be categorized into which of the following?

a) Brute Force technique

b) Divide and conquer

c) Greedy algorithm

d) Dynamic programming

Answer : Option b)

Quiz



5) Which of the following is not a limitation of binary search algorithm?

a) must use a sorted array

b) requirement of sorted array is expensive when a lot of insertion and deletions are needed

c) there must be a mechanism to access middle element directly

d) binary search algorithm is not efficient when the data elements more than 1500.

Answer : Option d)

Stack



Stack

- **Stack** is an **ordered list** of the same type of elements.
- It is a **linear list** where all insertions and deletions are permitted only at one end of the list.
- Stack is a **LIFO (Last In First Out)** structure.
- In a stack, when an element is added, it goes to the top of the stack.



"**Stack** is a collection of **similar data items** in which both insertion and deletion operations are performed based on **LIFO principle**".

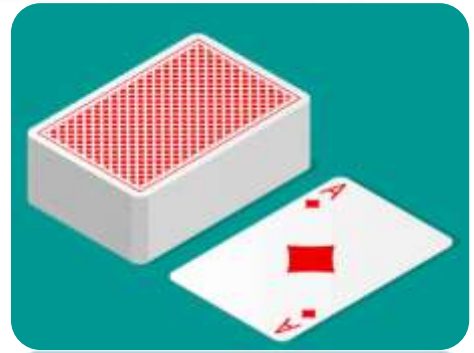


Stack

- When a stack is completely full, it is said to be **Overflow** state and if stack is completely empty, it is said to be **Underflow** state.
- Stack allows operations at **one end** only.
- Stack behaves like a real-life stack, For example,
  - we can remove a **plate or dish** from the top of the stack only
  - while playing a **deck of cards**, we can place or remove a card from top of the stack only.

  Similarly, here also, we can only access the **top element of a stack**.
- According to its LIFO structure, **the element which is inserted last**, **is accessed first.**

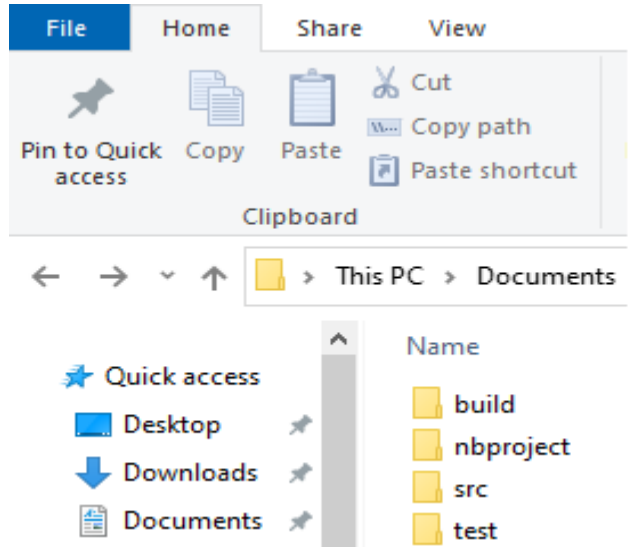




Stack - Real Time Applications

Back/Forward button in browsers and file browsers

- Use the stack data structure to store / maintain data in “First in Last out” or Last in First out principle.



Undo / Redo Operation in Office Automation Tool

- Use the stack data structure to store / maintain last modified data in **“First in Last out” or “Last in First out”** principle.



Stack Operations

- **There are some basic operations performed in a Stack**

1. Push() - Pushing / inserting an element on the stack

2. Pop() - Removing / accessing an element from the stack.

3. Peek() - Retrieval of top (recently stored) element of the stack, without removing it.

4. isEmpty() - Check whether stack is empty

5. isFull() - Check whether stack is full

Stack Operations

Initial values

- **CAPACITY** - Maximum size of Stack
- **TOP** - To represent the index/or address of the top element of the stack in the array. It is this position from where the element will be added or removed.

Example:

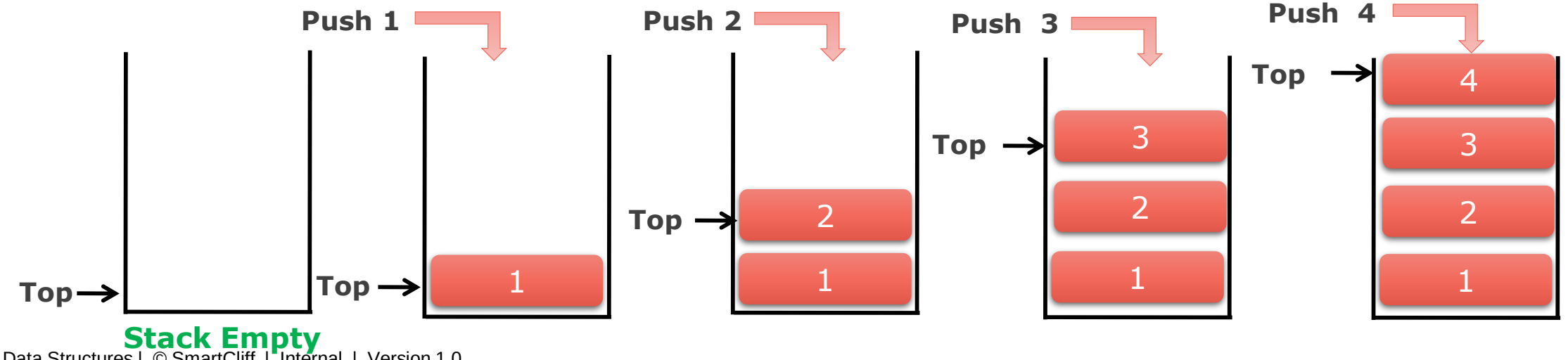
TOP = -1 indicates that the stack is empty

TOP = CAPACITY - 1 indicates that the stack is full

Stack Operations

Push()

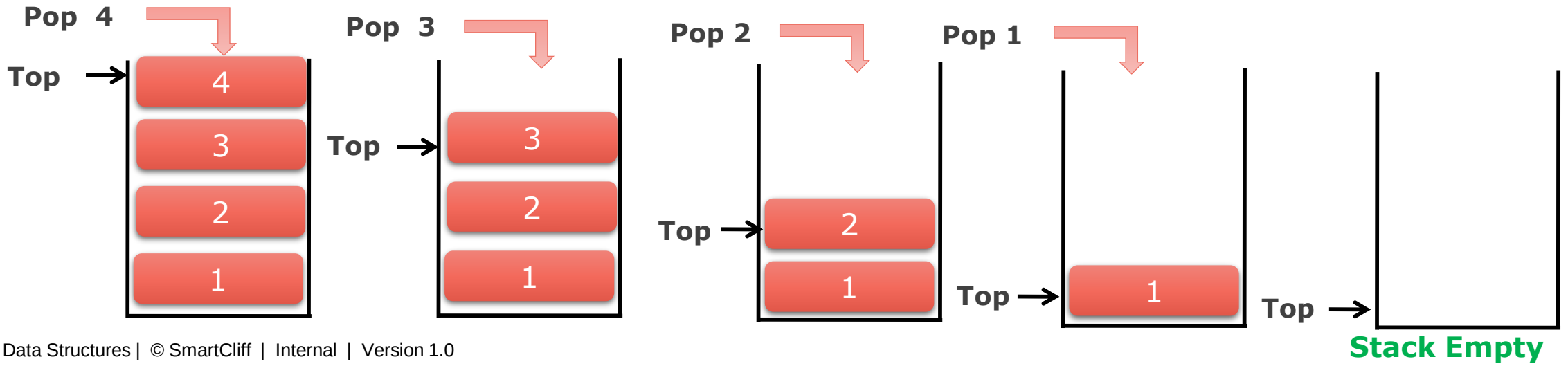
- The **push operation** is used to insert an element into the stack. The new element is added at the topmost position of the stack.
- First check if **TOP==CAPACITY-1**.
  If true, then it means the **stack is full** and no more insertions can further be added, an **OVERFLOW** message is printed.
- If not true, increase **TOP** by 1, then add the element at **TOP** position



Stack Operations

Pop()

- The pop operation is used to **delete the topmost** element from the stack.
- First check if **TOP == -1**.
  If true then it means the **stack is empty** so no more deletions can further be done, an **UNDERFLOW** message is printed.
- If not true, get the value of the top element, decrease **TOP** by one.

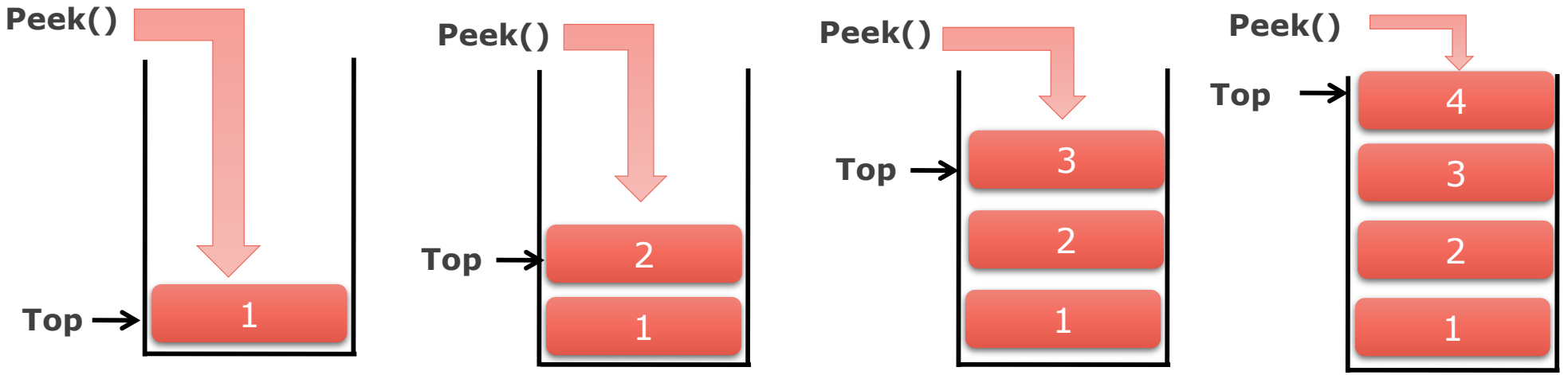


Stack Operations

Peek()

- Peek is an operation that returns the value of the **topmost element** of the stack **without deleting** it from the stack.
- First check if **TOP == -1**.

  If true then it means the **stack is empty**, an **UNDERFLOW** message is printed.
- If not true, get the value of the top element.



Stack Applications

- Function call
- Processing Recursion Call
- Parsing
- Tower of Hanoi Problem
- Expression Evaluation: Evaluation of Postfix / Prefix Expression
- Expression Conversion: Infix to Postfix Conversion / Infix to Prefix Conversion
- Maintain Browser History Etc.,

Quiz



1) Process of Removing element from the stack is called as _____.

a) Pop

b) Push

c) Create

d) Delete

Answer : Option a)

Quiz



2) What does 'stack overflow' refer to?

a) accessing item from an undefined stack

b) adding items to a full stack

c) removing items from an empty stack

d) index out of bounds exception

Answer : Option b)

Quiz



3) In peek operation, delete the topmost element from stack and decrement the TOP position. Say True or False?

a) True

b) False

Answer : Option b)

Quiz



4) User push 1 element in the stack having already five elements and having stack size as 5 then stack becomes _____.

a) User Flow

b) Overflow

c) Crash

d) Underflow

Answer : Option b)

Quiz



5) Consider the characters 'C', 'B', 'D', 'A' are stored in a stack (in that order), and then removed one character at a time, in what order will they be removed?

a) CBDA

b) ADBC

c) ABCD

d) ADCB

Answer : Option b)

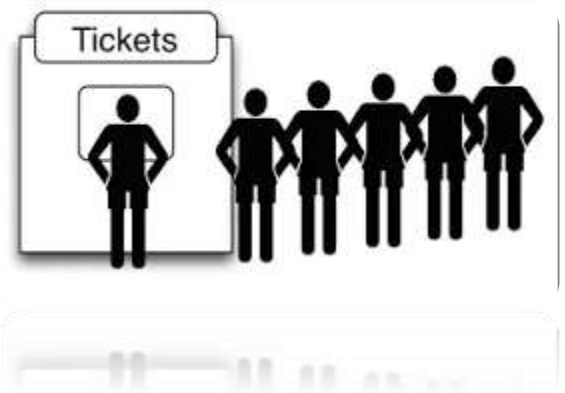
Queue



Queue

- Queue is a **linear data structure** where the first element is **inserted** from one end called **REAR** and **deleted** from the other end called as **FRONT.**
- Front points to the beginning of the queue and Rear points to the end of the queue.
- Queue follows the **FIFO (First - In - First Out)** structure.
- According to its FIFO structure, **element inserted first will also be removed first.**





Queue - Real Time Applications

- **Vehicles in single-lane one-way road**

  Vehicles in single-lane one-way road is one of the best examples where the vehicle enters first, exits first.



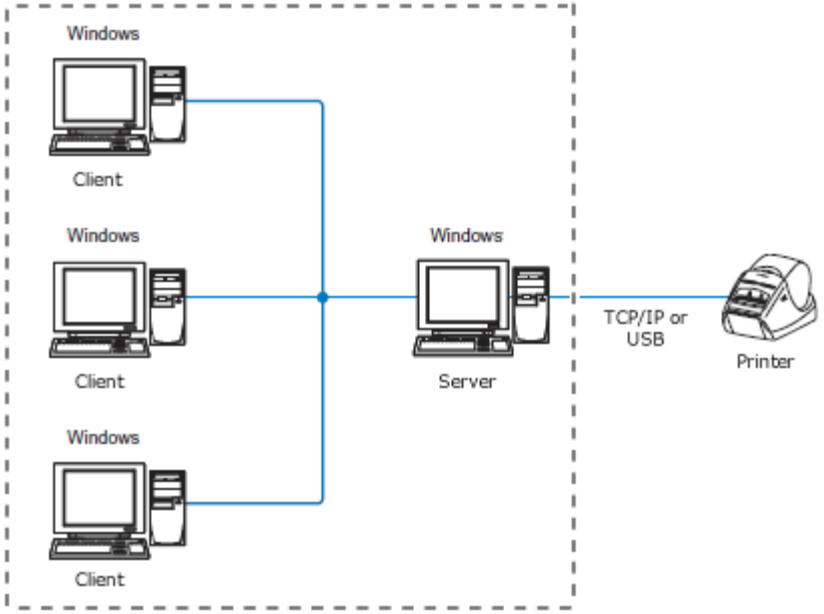
- **Call Centre**

  Queues are employed in call centres to keep clients waiting in line until a service representative is available.



Queue - Real Time Applications

Printer Sharing



- The computer can share that printer with other computers on the same network, Queues are formed in order to printing job in printer.

Queue Operations

- In a queue, one end is always used to **insert data (enqueue)** and the other is used to **delete data (dequeue),** because queue is open at both its ends.

Following are the basic operations performed on a Queue:

- **enqueue()** - operation for adding an element into queue.
- **dequeue()** - the operation for removing an element from queue.
- **Front** - to get the front data item from a queue.
- **Rear** - to get the last item from a queue.

Queue Implementation

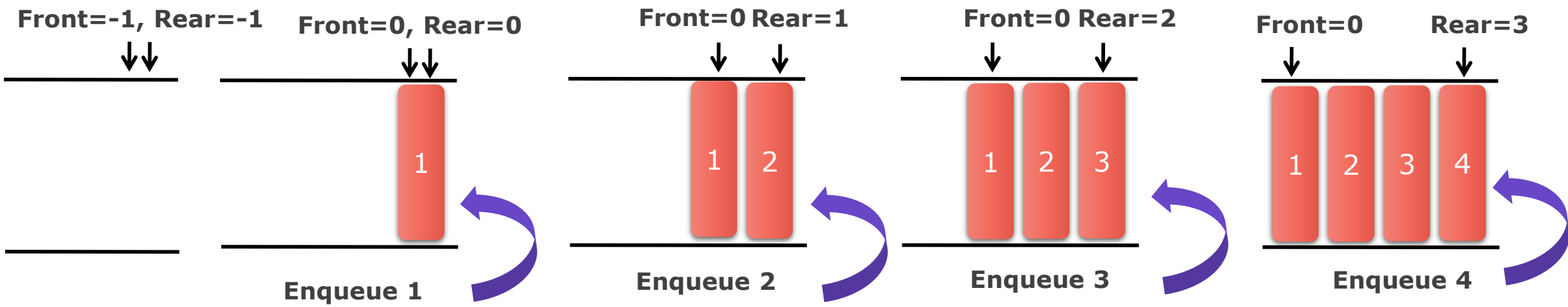
Initial values

- CAPACITY - Maximum size of Queue
- Use a variable front to represent the index/or address of the front element of the queue in the array. Use a variable rear to represent the index/or address of the recently stored element of the queue in the array.
    - Initially, front=rear=-1
    - front = -1 indicates that the queue is empty
    - rear = CAPACITY - 1 indicates that the queue is full

Queue Operations

Enqueue()

- The enqueue operation is used to **insert** an element into the queue.
- The new element is added at **rear position of the queue.**
- First check if **rear==CAPACITY-1**.
  If true, then it means the queue is full and no more insertions can further be added, an **OVERFLOW** message is printed.
- If not true, increase **rear** by 1, then add the element at **rear** position

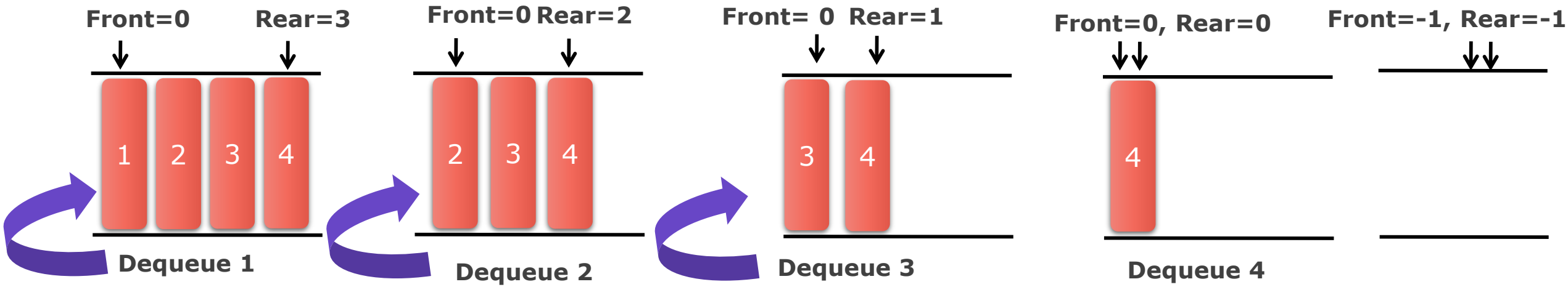


Queue Operations

Dequeue()

- The dequeue operation is used to **delete the element** which is present in the front end of the queue.
- First check if **front == -1**.

  If true then it means the queue is empty so no more deletions can further be done, an **UNDERFLOW** message is printed.
- If not true, get the value of the queue element, increase **front** by one.



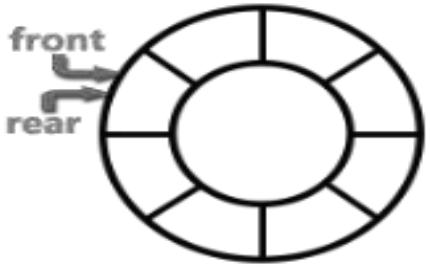
Queue Applications

- **Operating Systems**
  - CPU Scheduling
  - Spooling in Printers
  - Buffer for I/O devices
- **Computer Networks**
  - Queues in routers / switches
- **Serving requests** on a single shared resource, like a printer, CPU task scheduling etc.
- **Handling of interrupts** in real-time systems.

Types of Queue

Circular Queue

- A circular queue is the extended version of a regular queue where the last element is connected to the first element. Thus forming a circle-like structure.
- **Applications**
  - Computer controlled Traffic Signal System.
  - CPU scheduling and Memory management.



Types of Queue

Double Ended Queue (deque)

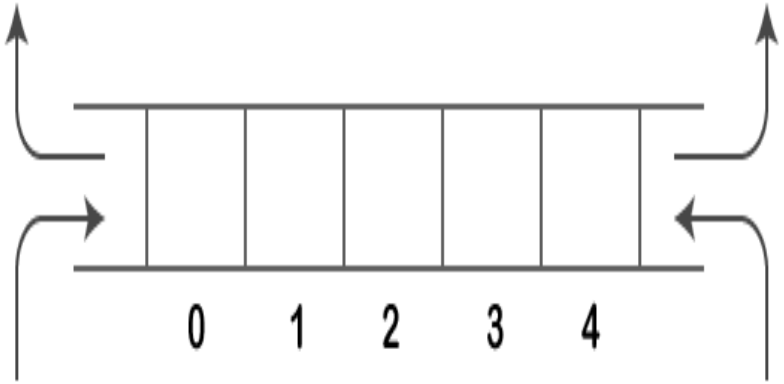
- A deque is also a special type of queue. The **enqueue** and **dequeue** operations are done at **both front and rear.**
- Dequeue can be used both as stack and queue as it allows the insertion and deletion operations on both ends.

Applications

Supports clockwise and anticlockwise rotations which can be useful in certain applications.

Example:

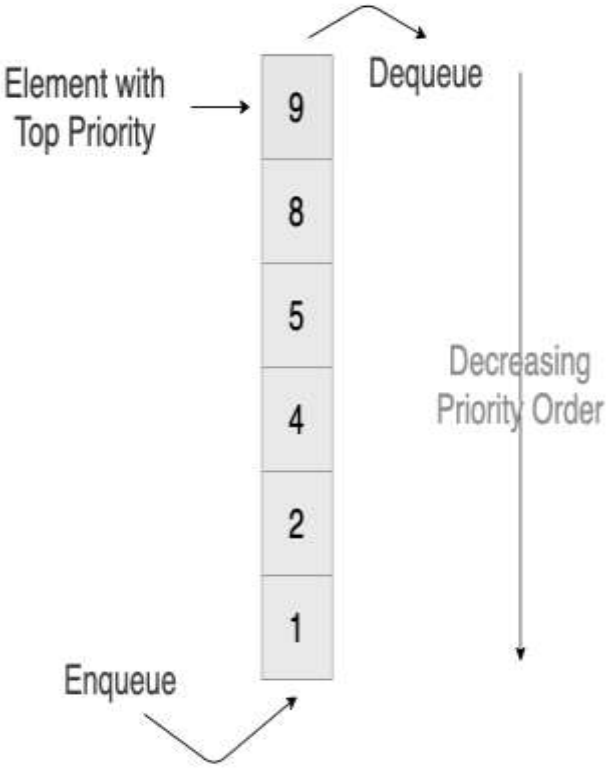
Steal job scheduling algorithm where Deque is used as deletions operation is required at both ends.



Types of Queue

Priority Queue

- A priority queue is a special kind of queue in which every element has a **predefined priority of service**.
- In this queue, the **enqueue operation** takes place at the **rear** in the order of arrival of the element, while the **dequeue operation** is done at the **front based on the priority** of an element.
- An element with a **high priority** will be **dequeued** before an element with a low priority.



Types of Queue

Priority Queue Applications:

- In Operation system (Scheduling algorithm, load balancing, interrupt handling)
- Data compression
- Shortest path and minimum spanning algorithm
- etc.,

Quiz



1) A queue follows the principle of _____

a) FIFO (First In First Out)

b) LIFO (Last In First Out)

c) Ordered Array

d) Linear Tree

Answer : Option a)

Quiz



2) In which data structure, element is inserted at one end called Rear and deleted at other end called Front.

a) Queue

b) Stack

c) Array

d) Tree

Answer : Option a)

Quiz



3) Insertion and Deletion operation in Queue is known as?

a) Push and Pop

b) Enqueue and Dequeue

c) Insert and Delete

d) None

Answer : Option b)

Quiz



4) If the elements "A", "B", "C" and "D" are placed in a queue and are deleted one at a time, in what order will they be removed?

a) ABCD

b) DCBA

c) DCAB

d) BADC

Answer : Option a)

Quiz



5) For the following scenario, choose the suitable data structure from the given list

“Suppose that a supermarket store decided that customers who come first will be served first”

a) Queue

b) Stack

c) Array

d) LinkedList

Answer : Option a)

List and Linked List



List

- The List is an **ordered sequence** of data items called elements. It can have **duplicate elements**.

$$A_1, A_2 \ldots\ldots A_N (N>=0)$$

Here,
A_1 - First Element of the list.
N - Size of the list.
Position of is A_i is i.
A_{i+1} succeeds A_i.
A_{i-1} precedes A_i.

Examples:
Students in Java Class=
[Jack, Raj, Abi, Henry, Rita, …, Vicky]

Days of Week = [S, M, T, W, Th, F, Sa]

Months = [Jan, Feb, Mar, Apr, …, Nov, Dec]

List

Types of List:

- Array List
- Linked List
- Restricted List
  - Stack (**LIFO Order**)
  - Queue (**FIFO Order**)

Common Operations:

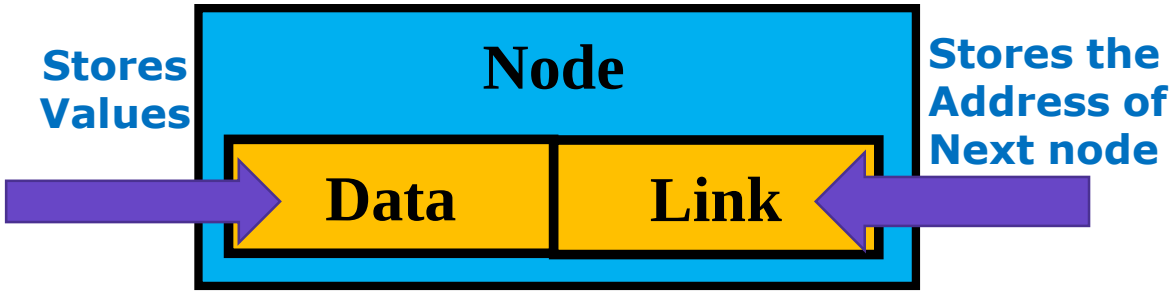
- Creation
- Insertion
- Deletion
- Traversal
- Search
- etc.

List can be **implemented** by using

- Array
- Linked List

Linked List

- It is one kind of **Linear Data Structure.**
- A Linked List consists of **set of nodes** related to each other through reference.
- Each node in a Linked List consists of **data and address of next data.**



Pictorial Representation of Node

Linked List

- Each node consists of its **own data and the address** of the next node and forms a **chain**.
- Next portion of **last node** refers no node. Hence it is set as **null value.**



Pictorial Representation of Linked List

Linked list Need

- **Dynamic Data Structure**

  We store data dynamically based on user demand.

- **Insertion and Deletion**

  Insertion and deletion operation is easier than array.

- **No Memory Waste**

- **Running Time**

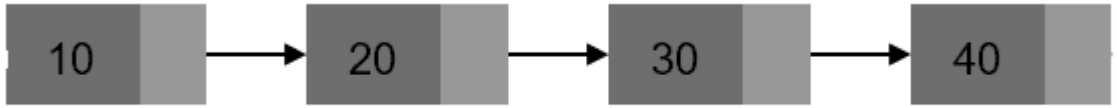
  We can reduce running time while inserting the element at front end and deleting the element at front end.

Array vs Linked List

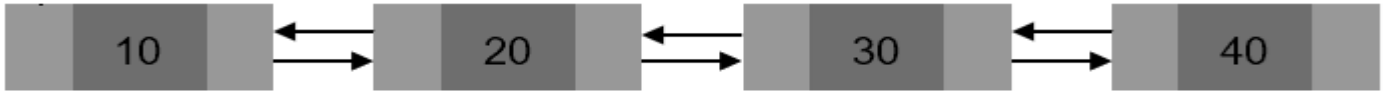
ARRAY	LINKEDLIST
Array holds set of elements of **same type.**	Each entity in a Linked List holds one or more data with address of next entity.
Memory allocation for an array is **contiguous**	Memory allocation for a linked list is not **contiguous**
The size of the Array is **static.**	The size of the LinkedList is **dynamic**.
Accessing the elements in an array is **faster** when compared to LinkedList.	Accessing the elements in an array is **slow** when compared to LinkedList.
Cost of i**nserting** an element at beginning and **deleting** an element at beginning is **more**.	Cost of **inserting** an element at beginning and **deleting** an element at beginning is **less**.

Types of Linked list

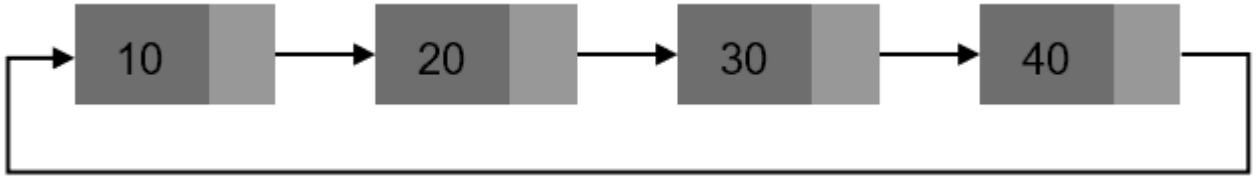
- **Simple Linked List**: Elements are accessed is forward only.



- **Doubly Linked List**: Elements can be accessed forward and backward.



- **Circular Linked List**: Last node contains link of the first node as next and the first node has a link to the last element as previous.



Linked List: Real Time Applications

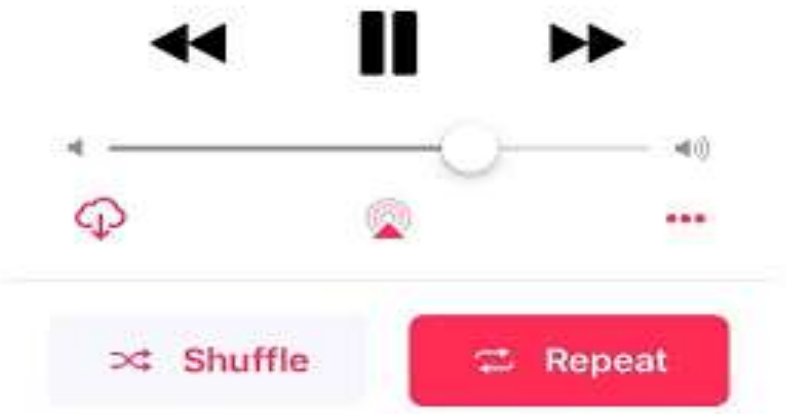
Image viewer



- **Image viewer software** uses a linked list to **view the previous and the next images** using the previous and next buttons.

Linked List: Real Time Applications

The navigation feature of a music player



- The **next and previous buttons** in a music player is used to **navigate to previous and next song** is a fine example for doubly linked list.
- In case of **repeat option** use circular linked list.

Linked List Operations

Basic operations performed on a list:

- **Creation of list**: This operation is used to create a list
- **Insertion /Deletion of data in the list**

  a) At/From the beginning of the list

  b) At/From the end of the list

  c) At/From the specified position in the list
- **Searching of data in the list**: Finding an element in a list
- **Display all data in the list**: Traversing of elements in the list

Singly Linked List – Node Creation

Single linked list is a sequence of elements in which **every element has link to its next element** in the sequence.

- Head is a pointer which always points to the first node of the list, if **Head points to nothing**, it means the **linked list is empty**.
- **Each node** in a list consists of **at least two parts**:
  - data
  - Reference to the next node



Singly Linked List – Node Creation



Steps:

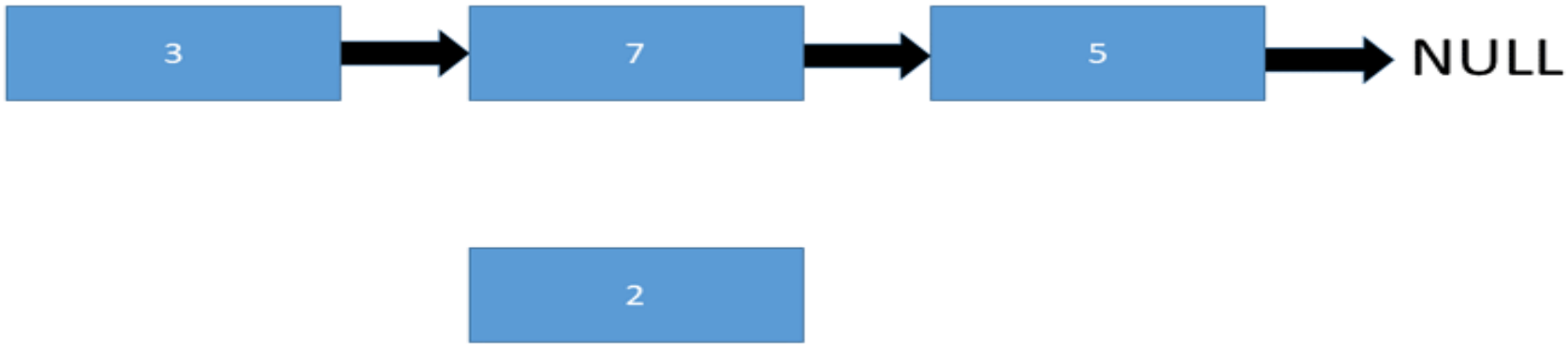
1. Allocate the Node

2. Put in the data

3. Make next of new Node as head

4. Move the head to point to new Node

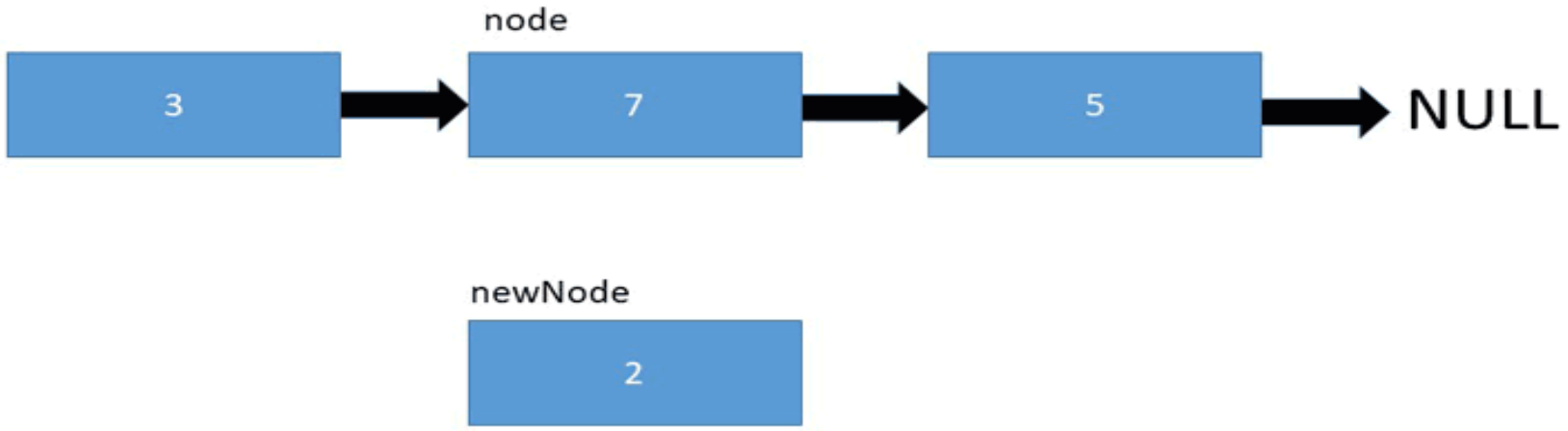
Singly Linked List: Insert at End



Steps:

1. Allocate a new node
2. Insert new element
3. Traverse the list to the last to find out last element
4. Have old last node point to new node
5. Have new

Singly Linked List: Insert at specified position



Steps:

1. Get the value and position of where the new node has to inserted.
2. Allocate a new node
3. Insert new element
4. Make a previous node point to new node and new node point to next node.

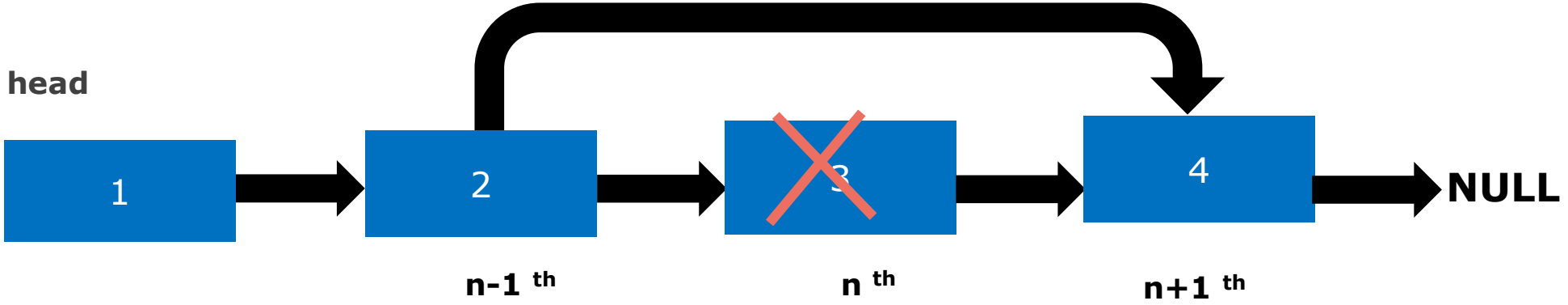
Singly Linked List – Delete at beginning



Steps:

1. Copy the address of first node
2. Move the head to the second node of the linked list
3. Disconnect the connection of first node to second node.
4. Free the memory occupied by the first node.

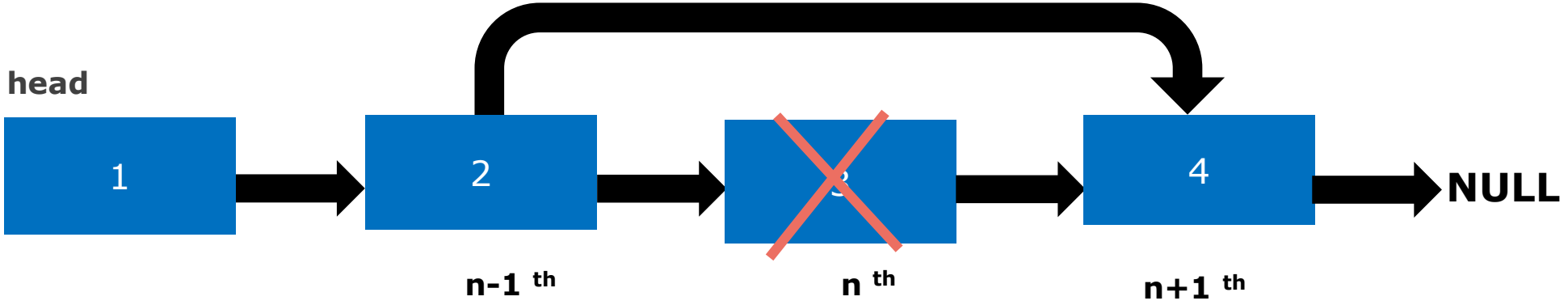
Singly Linked List – Delete at Specified Position



Steps:

1. Traverse to the nth node of the singly linked list and also keep reference of $n-1^{th}$ node
2. Reconnect the $n-1^{th}$ node with the $n+1^{th}$ node
3. Free the memory occupied by the nth node

Singly Linked List – Delete at end



Steps:

1. Traverse to the last node of the linked list keeping track of the second last node
2. If the last node is the head node then make the head node as NULL else disconnect the second last node with the last node
3. Free the memory occupied by the last node.

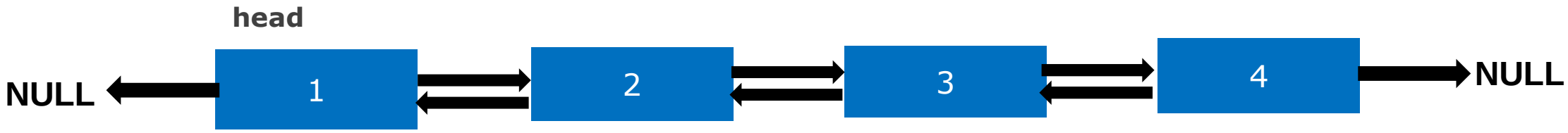
Doubly Linked List

- In a **single linked list**, every node has a link to its next node in the sequence.
  - we can traverse from one node to another node only in one direction
  - we can not traverse back.
- We can solve this kind of problem by using a **double linked list.**
- A double linked list can be defined as,

> Double linked list is a sequence of elements in which **every element has links to its previous element and next element** in the sequence.

Doubly Linked List

- In a double linked list, every node has a link to its previous node and next node.
  - we can **traverse forward by using the next field** and **can traverse backward by using the previous field.**

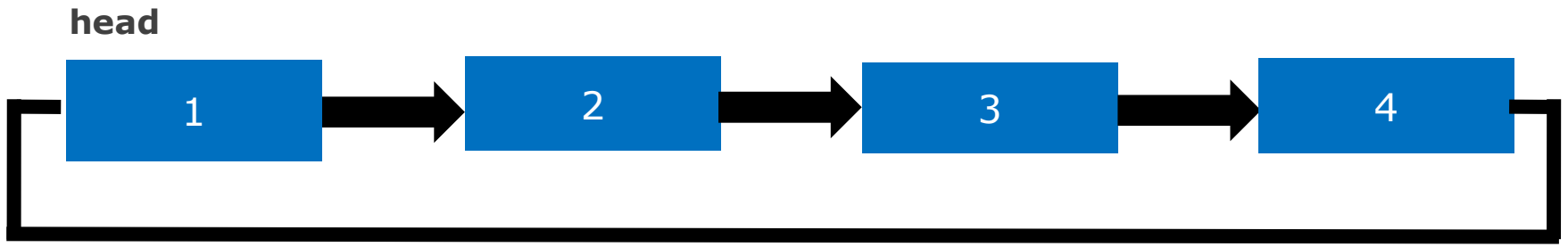


- In double linked list, the first node must be always pointed by head.
- Always the previous field of the first node must be NULL.
- Always the next field of the last node must be NULL.

Circular Linked List

- In single linked list, every node points to its next node in the sequence and the last node points NULL.
- But in circular linked list, every node points to its next node in the sequence but the **last node points to the first node in the list.**

A **circular linked list** is a sequence of elements in which every element has a link to its next element in the sequence and the **last element has a link to the first element**.



Quiz



1) A linear collection of data elements where the linear node is given by means of pointer is called?

a) Linked list

b) Node list

c) Primitive list

Answer : Option a)

Quiz



2) Which of the following is not true about Linked list when compared to an array?

a) Linked lists can change their size any time, the size of array has to be pre-decided

b) It is easy to insert and delete elements in a Linked list

c) Random access is allowed in Linked list

d) Memory allocation for an array is contiguous but the memory allocation for linked list is not contiguous.

Answer : Option c)

Quiz



3) A variant of linked list in which last node of the list points to the first node of the list is?

a) Singly linked list

b) Doubly linked list

c) Circular linked list

d) All the above

Answer : Option c)

Quiz



4) In doubly linked lists, traversal can be performed?

a) Only in forward direction

b) Only in reverse direction

c) In both directions

d) All the above

Answer : Option c)

Quiz



5) A variant of the linked list in which none of the node contains NULL pointer is?

a) Singly linked list

b) Doubly linked list

c) Circular linked list

d) All the above

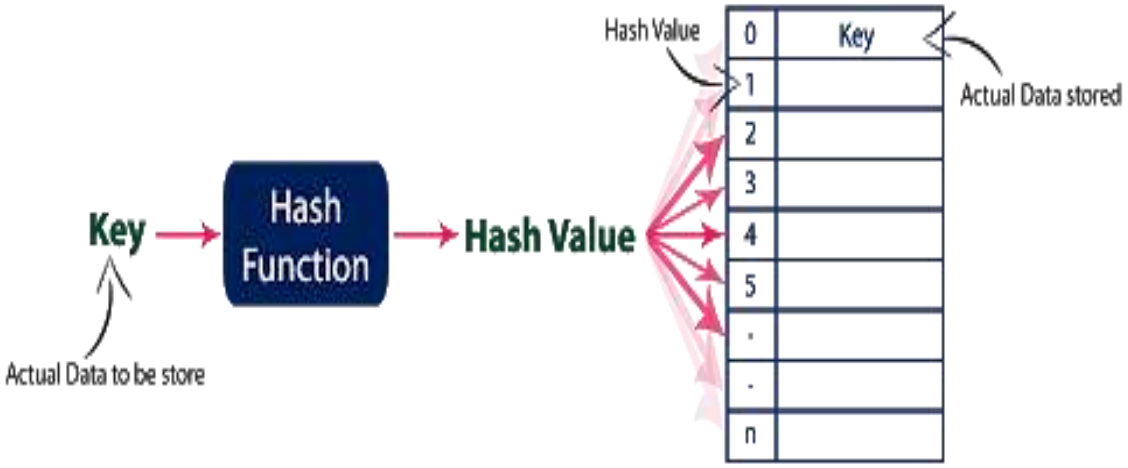
Answer : Option c)

Hashing



Hashing

- Hashing is a **process of mapping keys**, **values into the hash table** by using a **hash function**.
- It is used for storing and retrieving information as quickly as possible.
- In applications if we need operations at constant time, then hashing provides the way.









Need

- The **array & linked list implementation** of data sets requires on the **average of O(n) steps** to execute a single insert, delete or find operations.
- **Hashing** is a technique used for performing insertion, deletion & search operations in **constant average time of O(1).**
- Hashing is applied where the **array size is large**, and **time taken for searching an element**.

Real life application

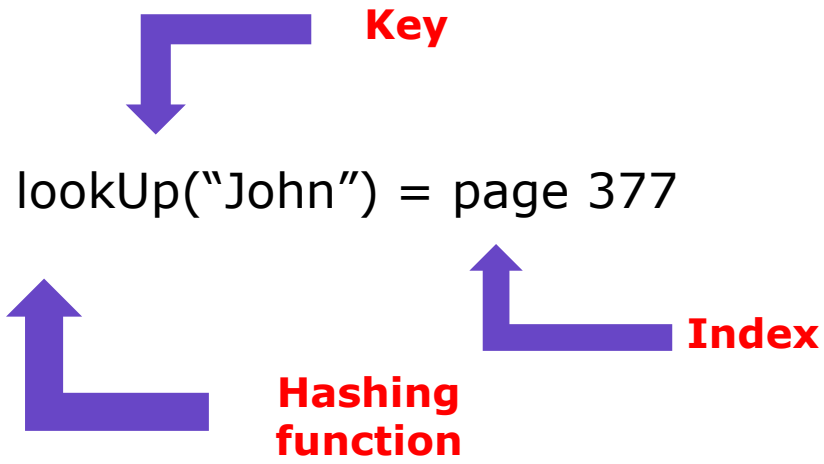
- Today’s world is dealing with millions of dons of data.
- Consider search engines and web pages. There are myriad look ups.
- Lookups are time critical.
- Typical data structures may not be sufficient to handle lookups efficiently with constant time.
- **For Example**: Search a word in dictionary

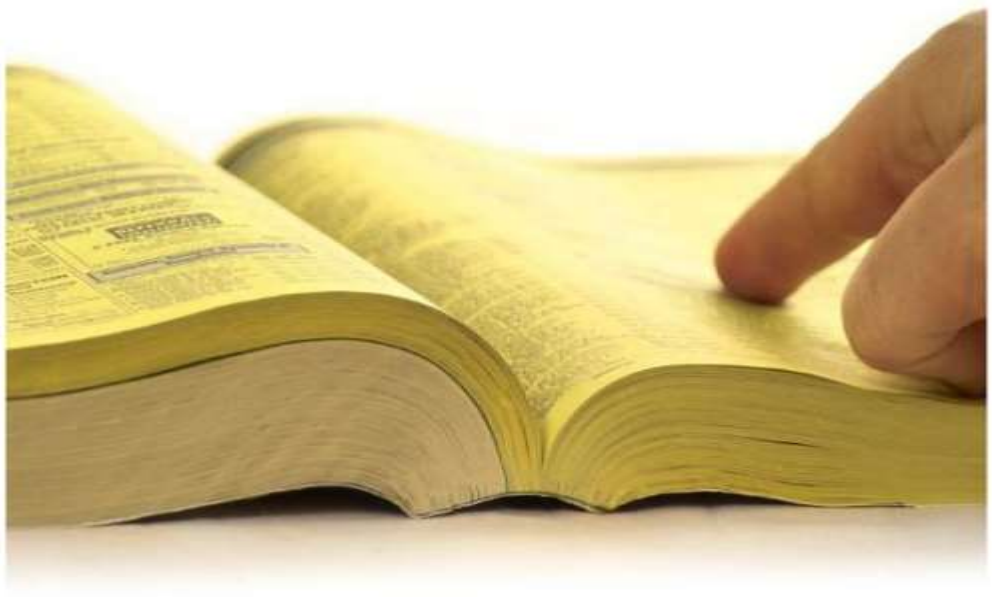




Real life application

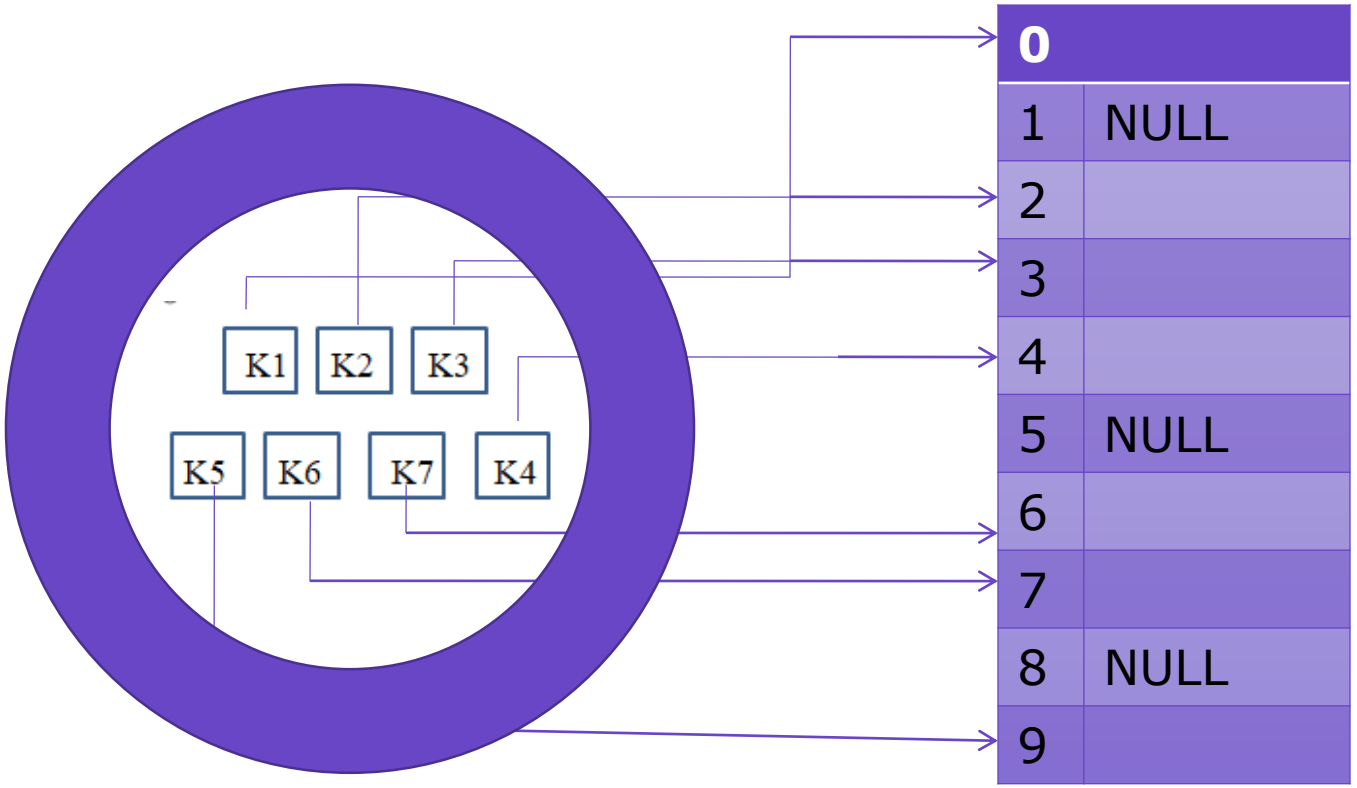
- In a **telephone directory**, how do you know the **contact number of Mr.John is on page no. 377**?





Hash Table

- **Hash table or hash map** is a data structure used to **store key-value pairs**.



Hash Function

- A process converts a key to a hash key is known as a **Hash Function.**
- There are **some ways** of calculating the **hash function**:
    - Division Method
    - Multiplication Method
    - Mid Square Method
    - Folding Method

Hash Function - Division Method

- In the **division method**, the hash function can be defined as:

  h(key) = key % m, where m is the size of the hash table

Example 1:

- If the **key value is 6** and the size of the **hash table is 10**.
- When we apply the hash function to key 6 then the index would be:
  - **h(6) = 6%10 = 6**
- The index is 6 at which the value is stored.

Hash Function - Division Method

Example 2:

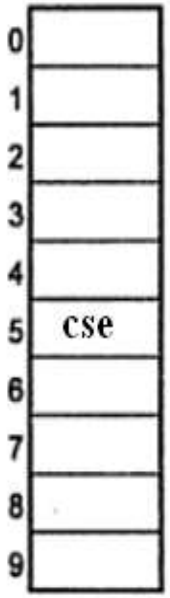
- If the **key is a string ‘cse’** and the size of the **hash table is 10**, add up ASCII values of characters in the string.

```
                cse
c - 99
s - 115
e - 101
  ------
   315
  -------
```

- When we apply the hash function to key ‘cse’ then the index would be:

h(315) = 315%10 = 5

- The index is 5 at which the value ‘cse’ is stored.



Hash Function - Mid Square Method

- Key is multiplied by itself, and address is obtained by **selecting appropriate numbers** of **bits or digit from the middle**

Example:

- Table size is 1000. Table index is from 0 to 999, Key = 123
- **Perform Square:** H(123) = 123 x 123 = 15129
- **Retrieve Mid value:** Mid of (15129) = 512.
- For the key 123, the **Hash Key is 512**.

Hash Function - Multiplication Method

- The hash function used for the multiplication method is:

H (Key) = Floor(m*(Key*A)mod 1), where m is table size and A is Donald Knuth Constant = 0.61803398987

Example:

Key = 107 and table size is 50.

Here m = 50

H(107) = floor(50 * (107 * 0.61803398987 mod 1))

H(107) = floor(50 * (66.1296369 mod 1)

H(107) = floor(50 * (0.1296369)

H(107) = floor (6.481845)

H(107) = 6.

The key 107 is stored in the hash key 6.

Hash Function - Digit Folding Method

- The **folding method** is used for creating hash functions starts with the item being divided into equal-sized pieces i.e., the last piece may not be of equal size.
- The outcome of adding these bits together is the hash value,

H(x) = (a + b + c) mod M, where a, b, and c represent the preconditioned key broken down into three parts and M is the table size.

- In other words, the **sum of three parts of the preconditioned key** is divided by the table size. The remainder is the hash key.

Hash Function - Digit Folding Method

Example :

The task is to fold the key 123456789 into a Hash Table of ten spaces (0 through 9).

It is given that the key, say X is 123456789 and the table size (i.e., M = 10).

Since it can break X into three parts in any order. Let's divide it evenly.

Therefore, a = 123, b = 456, c = 789.

H(x) = (a + b + c) mod M

H(123456789) =(123 + 456 + 789) mod 10

= 1368 mod 10 = 8.

Hence, 123456789 is inserted into the table at address 8.

Hash Function

The **common operations** for hash table are:

Methods	Description
Create	Creates new hash table
Hash search	Searches the key in hash table
Hash insert	Inserts a new key into hash table
Hash delete	Deletes a key from hash table
Delete hash table	Deletes hash table

Collision

- When the two different values have the same value, then the problem occurs between the two values, known as a collision.
- To resolve these collisions, we have some techniques known as collision techniques.
- The following are the collision techniques:
  - **Separate chaining** : It is also known as Open Hashing.
  - **Open addressing** : It is also known as Closed Hashing.

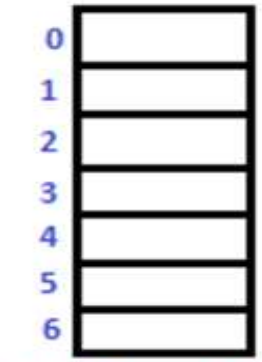
Collision - Separate chaining

To handle the collision,

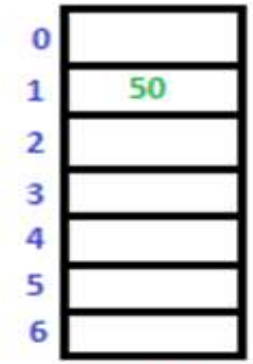
- This technique creates a linked list to the slot for which collision occurs.
- The new key is then inserted in the linked list.
- These linked lists to the slots appear like chains.
- That is why, this technique is called as separate chaining.

Collision - Separate chaining

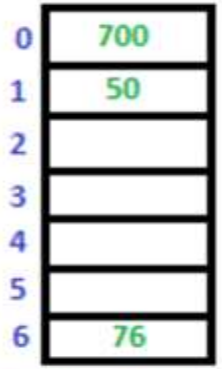
Example: Let's consider a simple hash function as "key mod 7" and sequence of keys as 50, 700, 76, 85, 92, 73, 101.



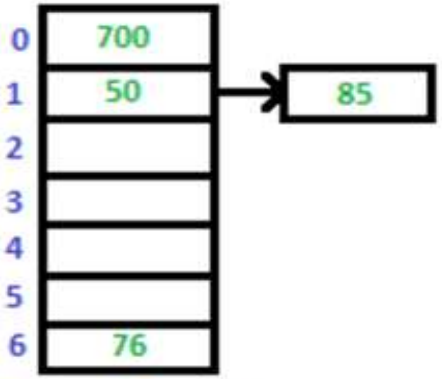
Initial Empty Table



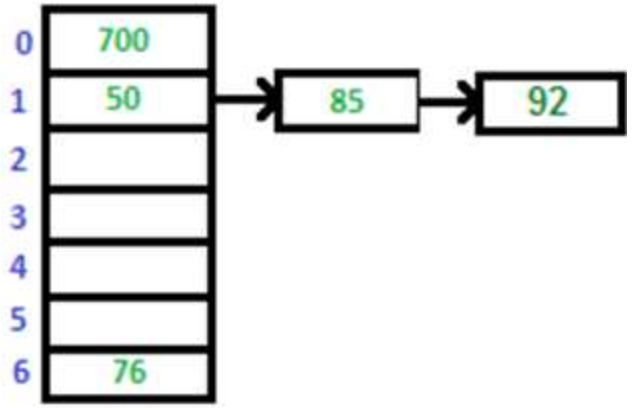
Insert 50



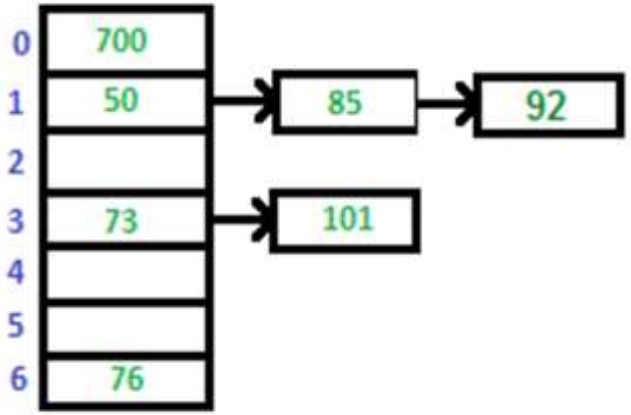
Insert 700 and 76



Insert 85: Collision
Occurs, add to chain



Inser 92 Collision
Occurs, add to chain



Insert 73 and 101

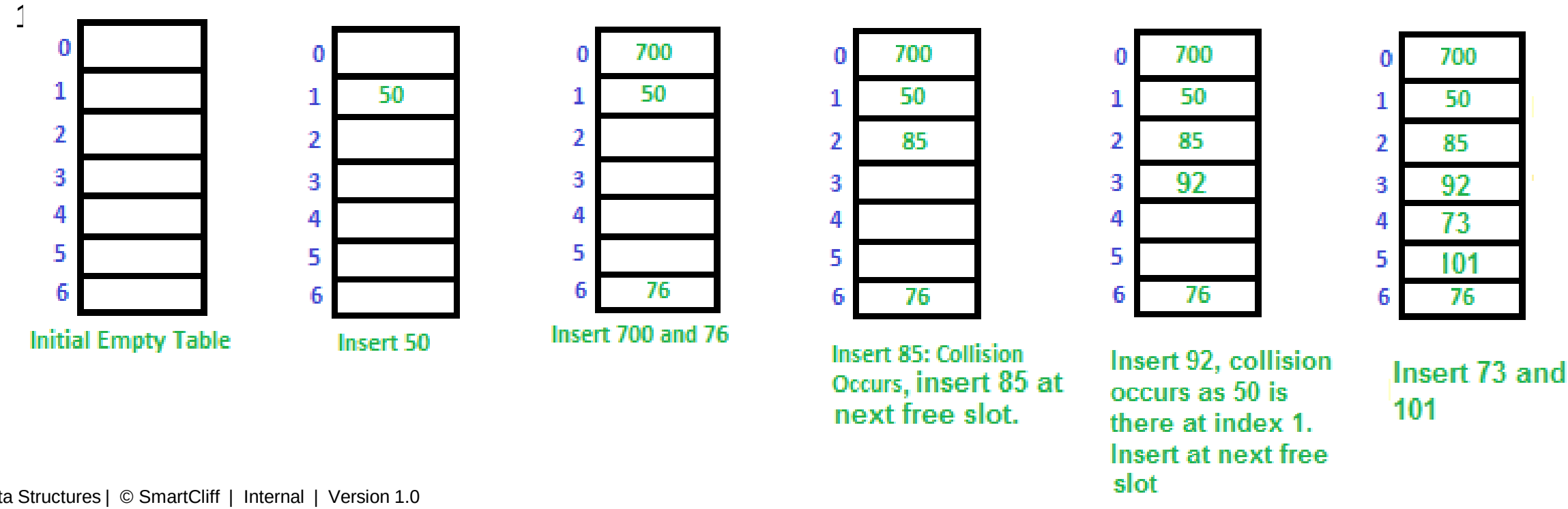
Open Addressing

- In open addressing, all elements are stored in the hash table itself.
- Each table entry contains either a record or NIL.
- When searching for an element, we examine the table slots one by one until the desired element is found or the element is not in the table.
- If collision occurs, alternative cells are tried until an empty cell is found.
- This technique is used when storage space is large.
- **Open Addressing is done in the following ways**:
    a) Linear Probing
    b) Quadratic Probing
    c) Double hashing

Open Addressing

Linear Probing

- In linear probing, we linearly probe for next slot.
- Let's consider a simple hash function as "key mod 7" and a sequence of keys as 50, 700, 76, 85, 92, 73,



Open Addressing

Quadratic Probing

- In quadratic probing,
  - When collision occurs, we probe for i^2'th slot in i^{th} iteration.
  - We keep probing until an empty slot is found.

Double Hashing

- In double hashing,
  - We use another hash function hash2(x) and look for i * hash2(x) slot in i^{th} iteration.
  - It requires more computation time as two hash functions need to be computed.

Applications

- **Widely used applications** are,
  - Web searches
  - Compilers (Symbol table)
  - Spell checkers
  - Database Query processing
  - Password matching
  - Game programs

Quiz



1) Which of the following is true about hash table?

a) A structure that maps values to keys

b) A structure that maps keys to values

c) A structure used for storage

d) A structure used to implement stack and queue

Answer : Option b)

Quiz



2) When the two different values have the same value, then the problem occurs between the two values, known as a

a) Collision

b) Diffusion

c) Replication

d) Duplication

Answer : Option a)

Quiz



3) Hashing is a technique used for performing insertion, deletion & search operations in constant average time of _____.

a) O(n)

b) O(1)

c) O(nlogn)

d) O(logn)

Answer : Option b)

Quiz



4) Open hashing is a collision technique, which also known as

a) Open Addressing

b) Separate Chaining

c) Closed hashing

d) Closed Addressing

Answer : Option b)

Quiz



5) Given the following input (4322, 1334, 1471, 9679, 1989, 6171, 6173, 4199) and the hash function x mod 10, which of the following statements are true?

i. 9679, 1989, 4199 hash to the same value

ii. 1471, 6171 hash to the same value

iii. All elements hash to the same value

iv. Each element hashes to a different value

a) i only

b) ii only

c) i and ii only

d) iii or iv only

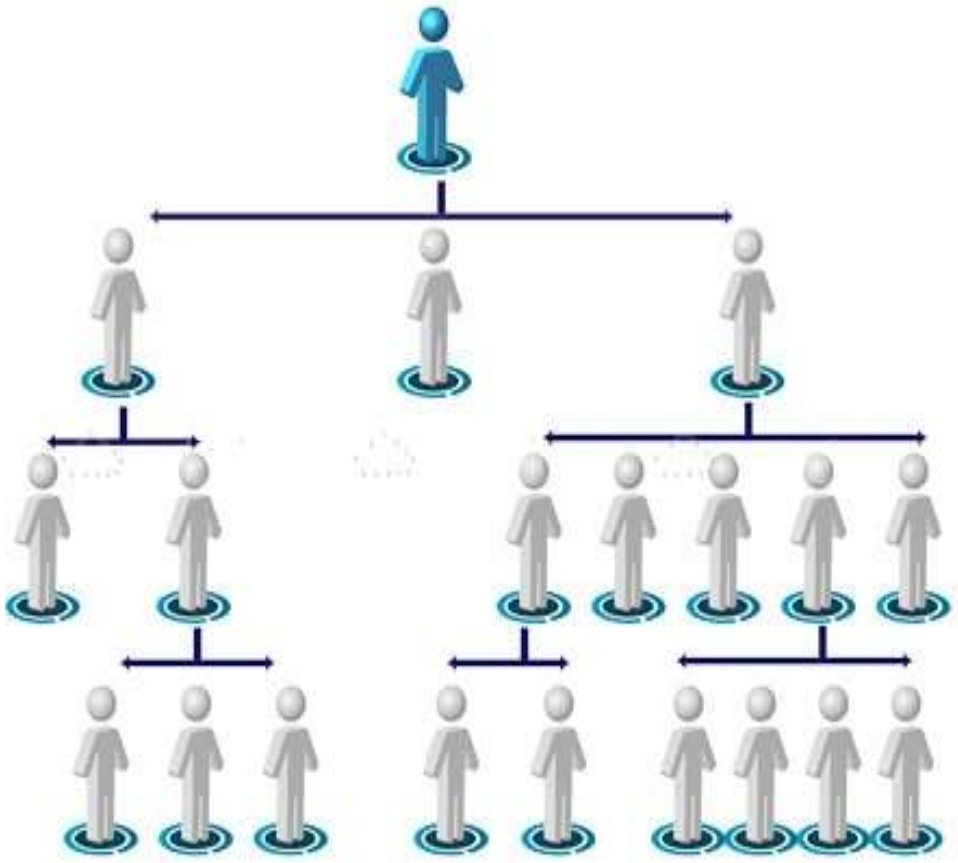
Answer : Option c)

Introduction to Tree and binary tree



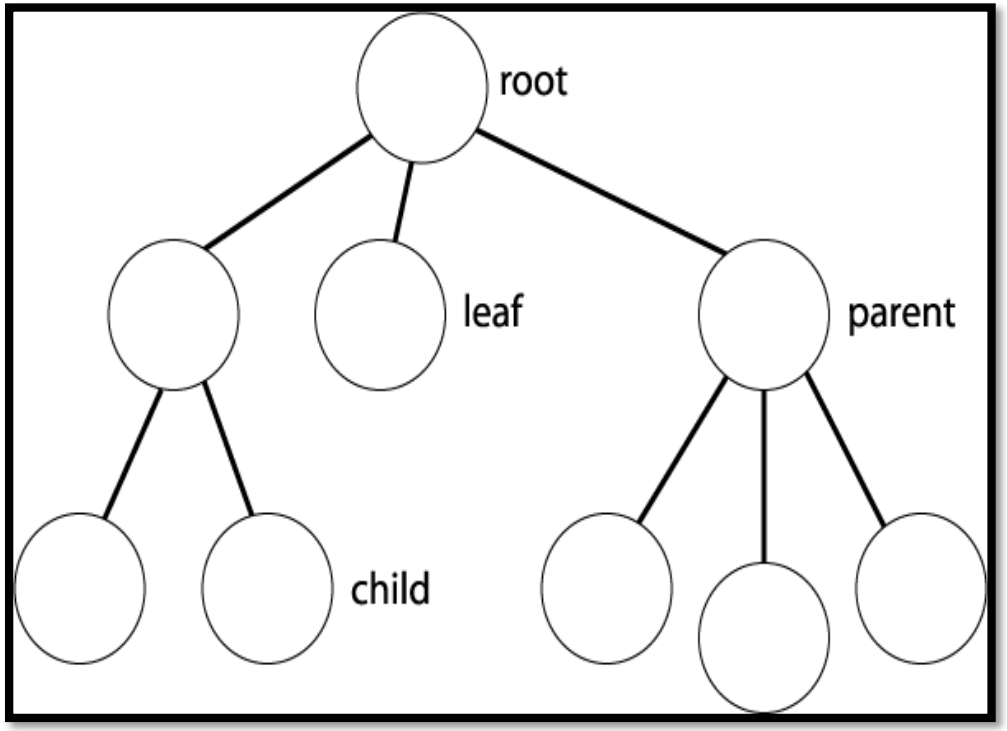
Introduction to Tree

- A tree is a **nonlinear data structure** that is made up of a set of linked nodes, which can be used to represent a hierarchical relationship among data elements.
- In nonlinear data structure, data elements are not organized in a sequential fashion. **Every item is attached with many other items.**



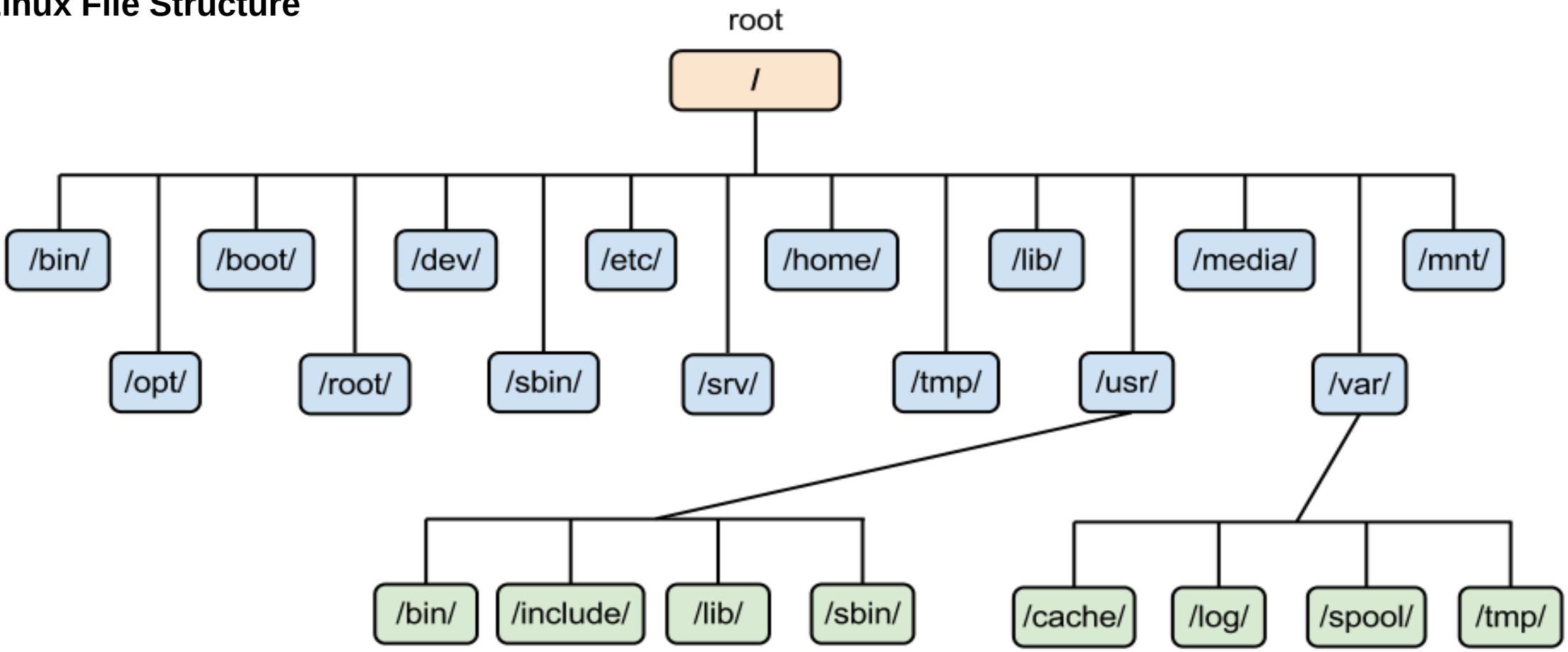
Introduction to Tree

- Tree is a collection of a distinguished node **“root”** with **zero or more sub trees connected** to root by edge.
- The root of each sub tree is said to be child of ‘‘root’’, and ‘‘root’’ is the parent of each sub tree root.

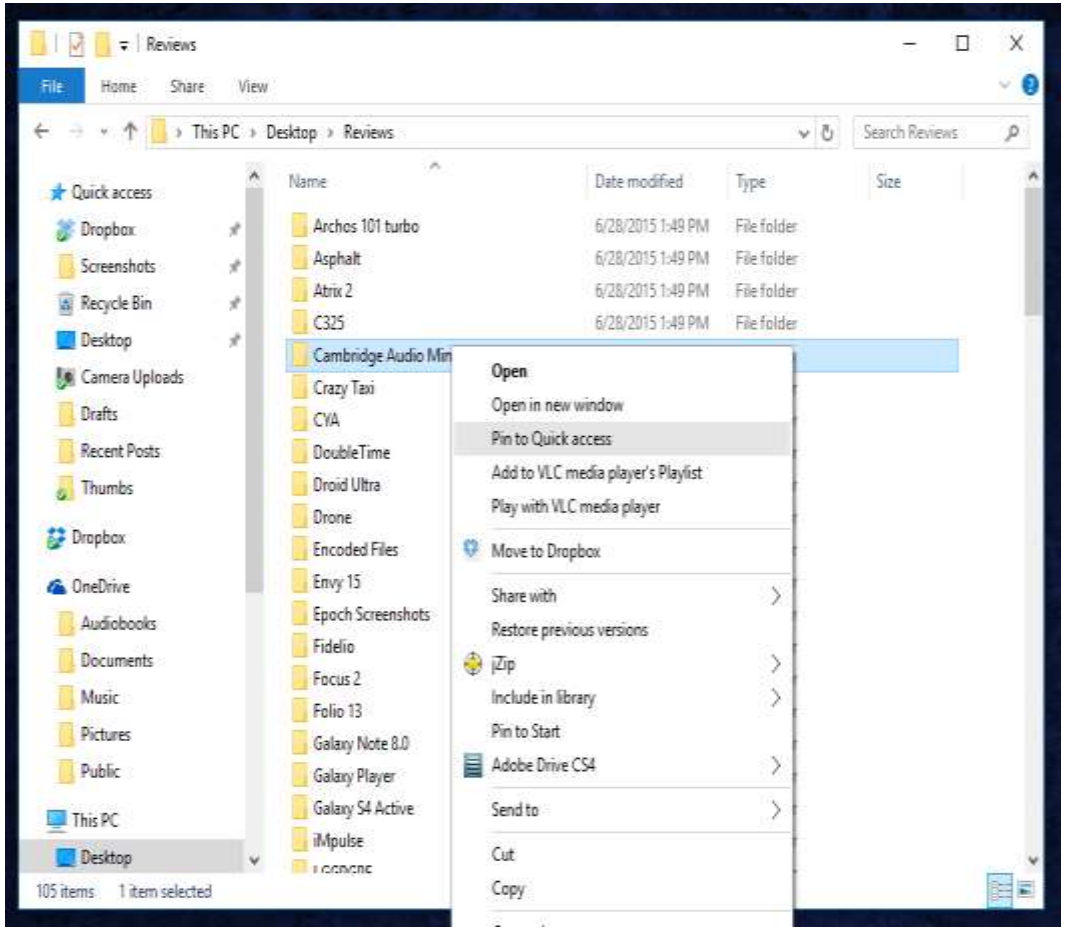


Real Time Applications

Linux File Structure



Real Time Applications



- The **file explorer application** used by operating systems to browse folders and files is called File Explorer.
- It gives the user a **graphical interface** to navigate and access the files on the computer.

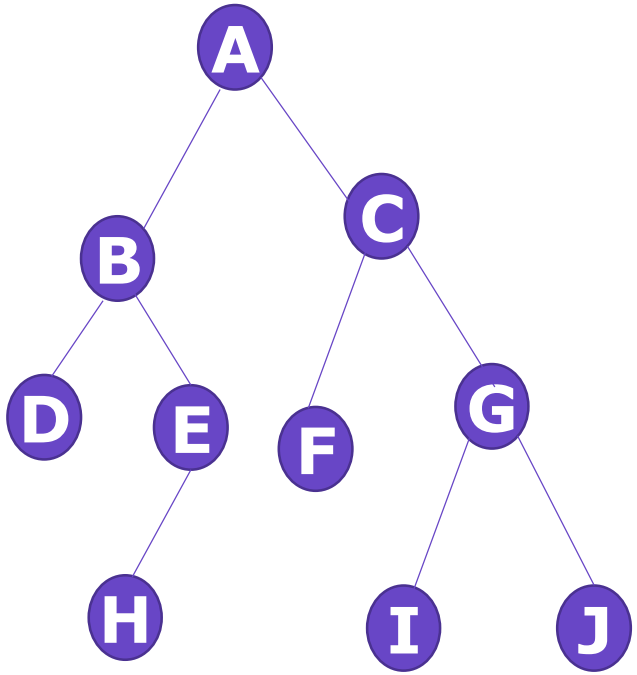
Terminologies

- **Node** - Data item in the tree {A,B,C,D,E,F,G,H,I,J}
- **Root**
  - –A unique node to which further sub trees are attached.
  - –First and Topmost node (node A)
- **Parent Node** - Node having further subbranches
- **Leaves** - Terminal nodes of the tree & Nodes with no children
- **Siblings** - Children nodes of same parent

  Siblings(A)={B,C} Siblings(B)={D,E}

  Siblings(C)={F,G} Siblings(E)={H}

  Siblings(G)={I,J}



Terminologies

- **Degree of the node :** Total no. of sub trees attached to that node.

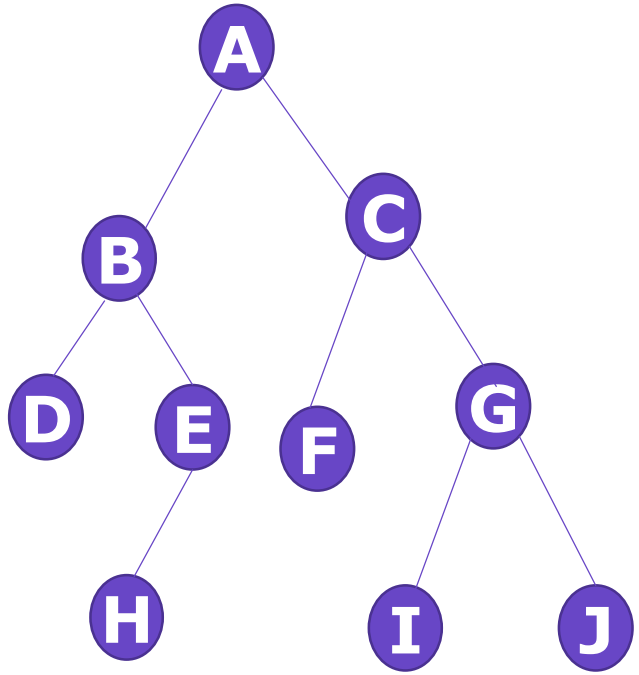
  Degree of (A)= 2 Degree of (B)= 2

  Degree of (G)= 2 Degree of (C)= 2

  Degree of (E)= 1

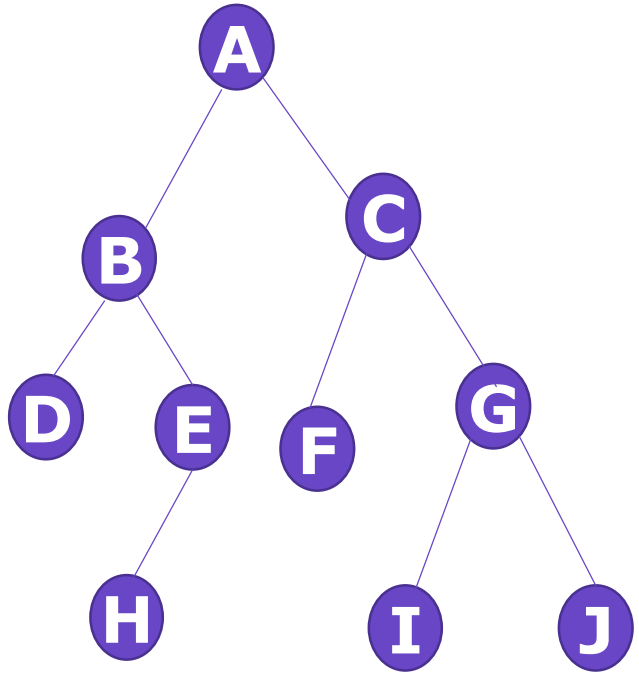
  Degree of (D, H, F, I, J)= 0

- **Degree of Tree:** Maximum degree in the tree
- **Terminal node (External or Leaf node)**
  - node which do not have further links
  - node with Degree 0 (leaf) { D,F,H,I,J }
- **Non-Terminal node (Internal or Non leaf node)**
  - node whose degree is not Zero { A,B,C,E,G }



Terminologies

- **Level**
  - Root node at level 0. Adjacent node to root at level 1 and so on.
- **Edge**
  - A connection line between two adjacent nodes of a tree
- **Path**
  - Sequence of consecutive edges from source to destination node



Example

Path 1: A->B-D

Path 2: A->B->E->H

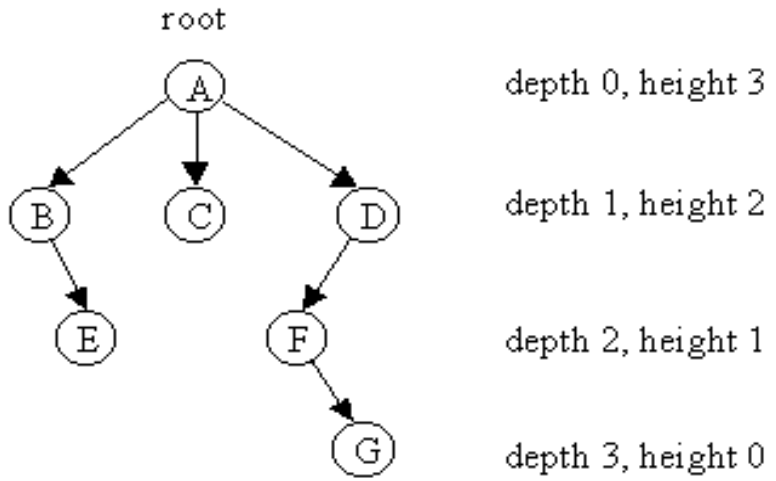
Path 3: A->C->F

Path 4: A->C->G->I

Path 5: A->C->G->J

Terminologies

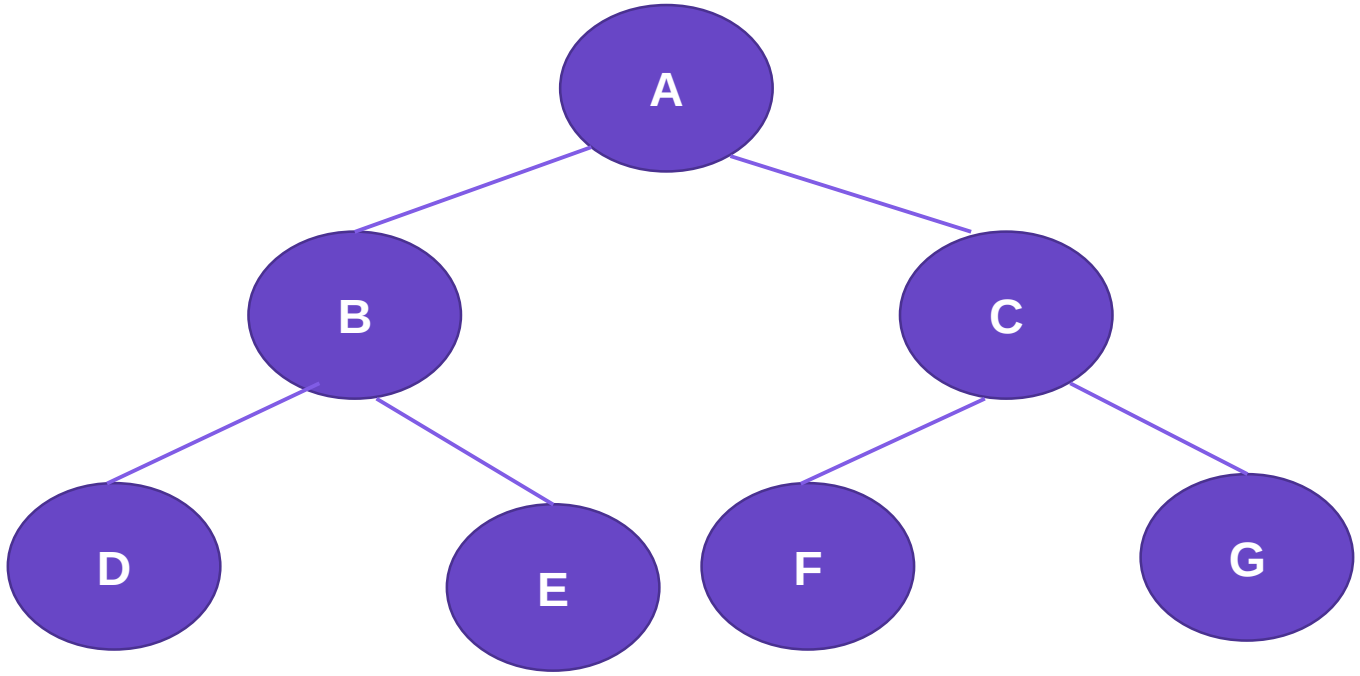
- **Height of a node**
  - Length of the longest path from the node to a leaf.
- **Depth of a node**
  - Length of a unique path from root to the node.
- **Height of Tree** – Maximum level is the height of tree
  - Height of root
- **Depth of Tree**
  - Depth of the deepest leaf
- **Ancestor & Descendant**
  - If there is a path from node n1 to n2, then
  - n1 is ancestor of n2 and n2 is descendant of n1



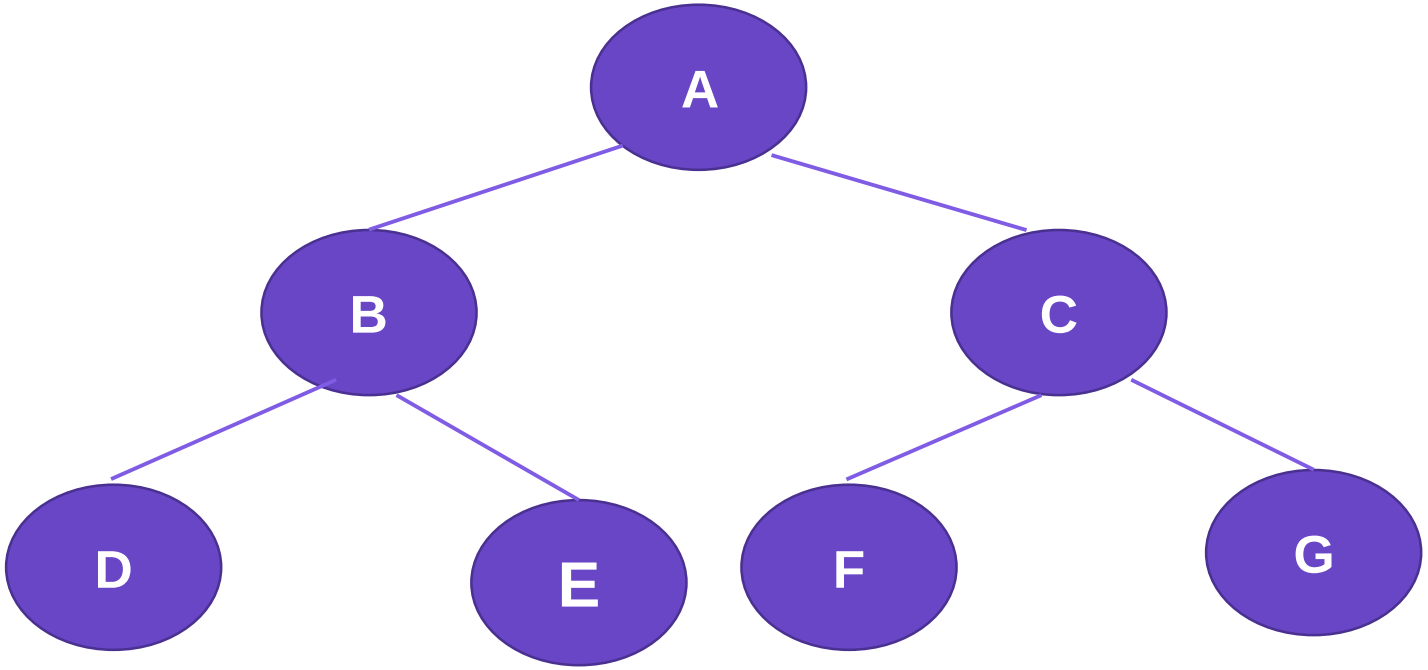
A tree of height 3

Binary Tree

- Binary tree is a tree in which every node **except leaf node** has **at most two child nodes**. In other words, binary tree is a tree in which **no node** can have **more than two children**.

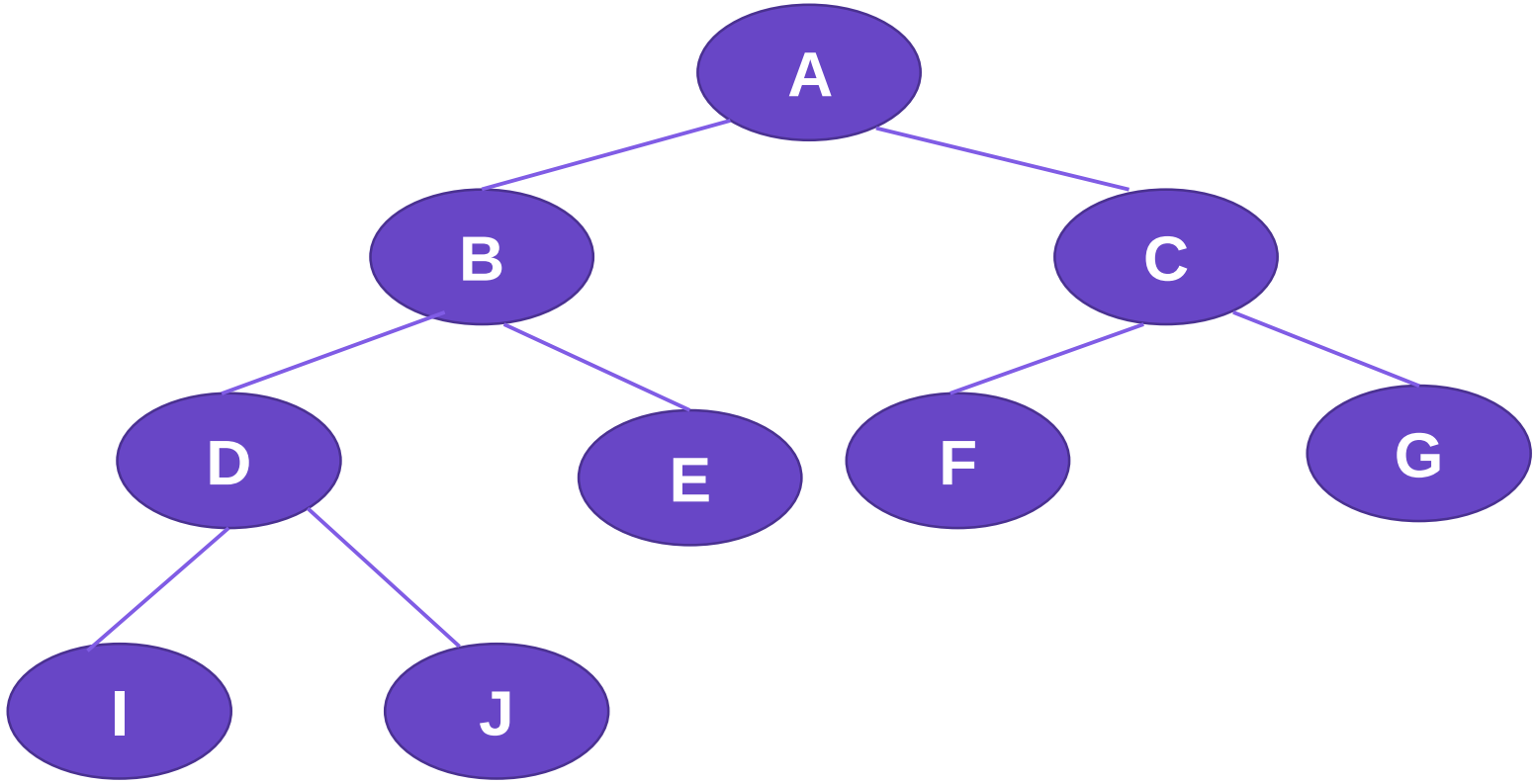


Types of Binary Tree: Full Binary Tree



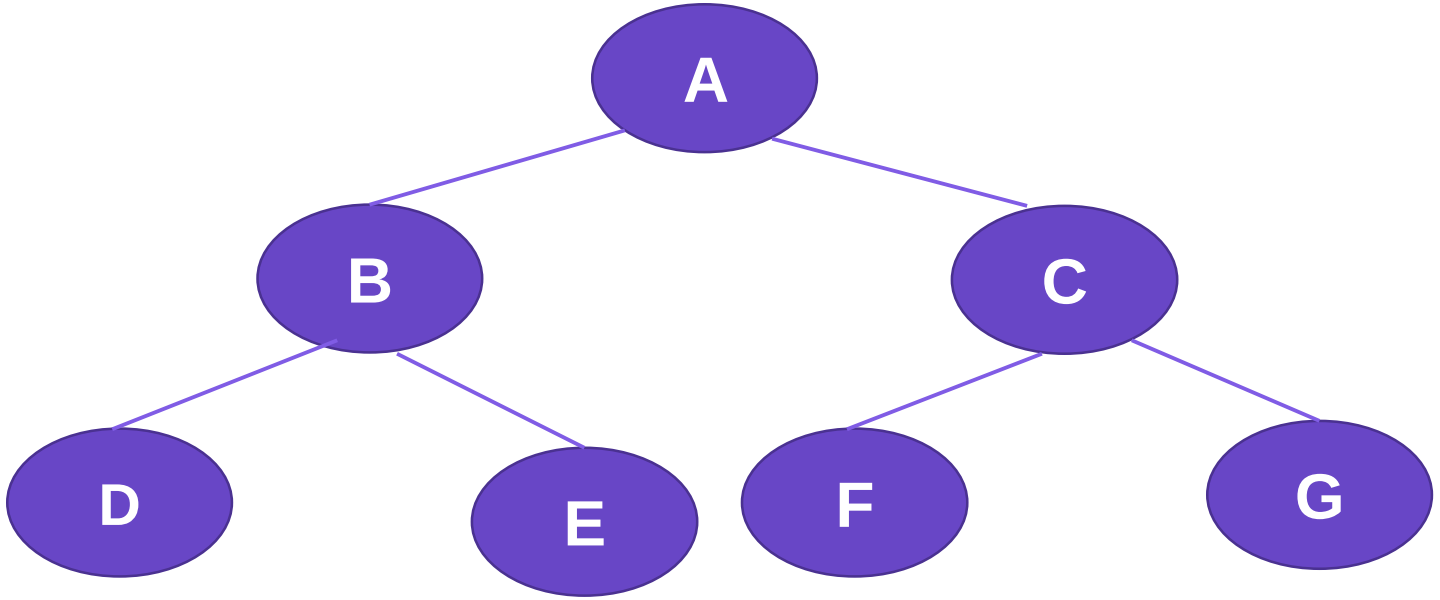
A binary tree in which **every node** has **exactly 0 or 2 children**

Types of Binary Tree: Complete Binary Tree



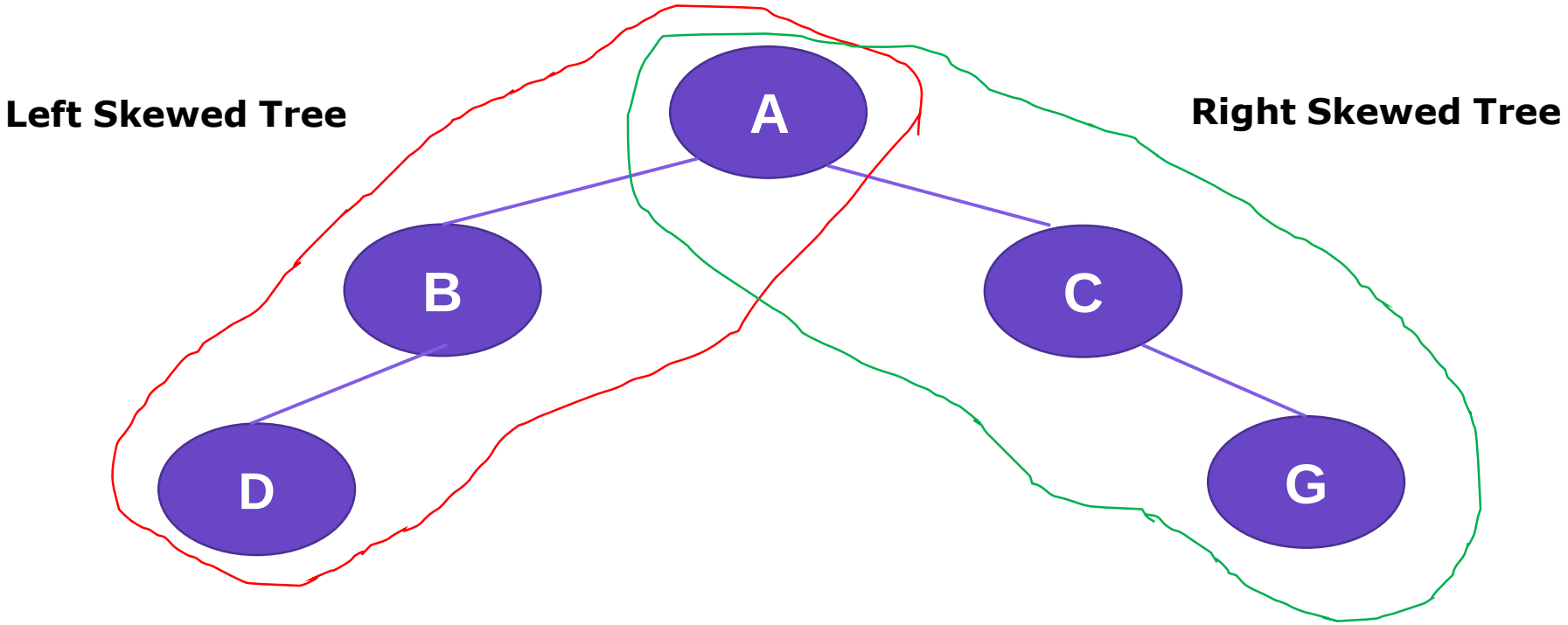
A complete binary tree in which **every level, except possibly the last**, is completely filled, and all nodes are as far left as possible.

Types of Binary Tree: Perfect Binary Tree



A perfect binary tree in which **all leaves are at the same depth or level**

Types of Binary Tree: Left and right Skewed Tree



Left Skewed Tree	**-** The tree in which each node is attached as a left child of parent node.
Right Skewed Tree	**-** The tree in which each node is attached as a right child of parent node.

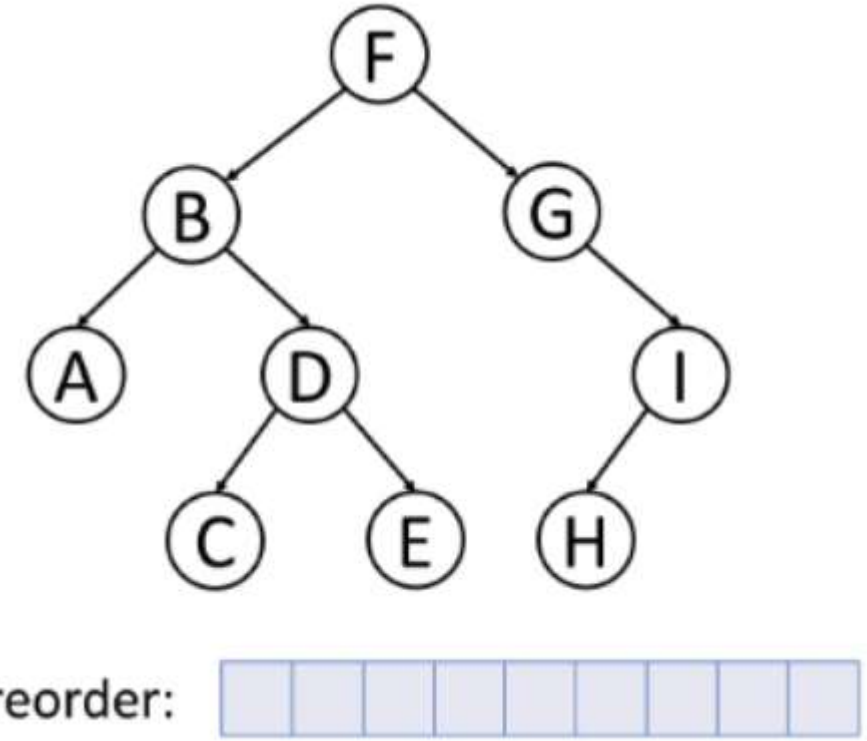
Binary Tree Operations

- The **common operations** for Binary tree are:

Methods	Description
Insert / Delete	Insert / Delete an element in a tree
Search	Searches an element in a tree
Preorder Traversal (Root- Left- Right)	Traverses a tree in preorder fashion
Inorder Traversal (Left- Root- Right)	Traverses a tree in inorder fashion
Postorder Traversal (Left- Right- Root)	Traverses a tree in postorder fashion

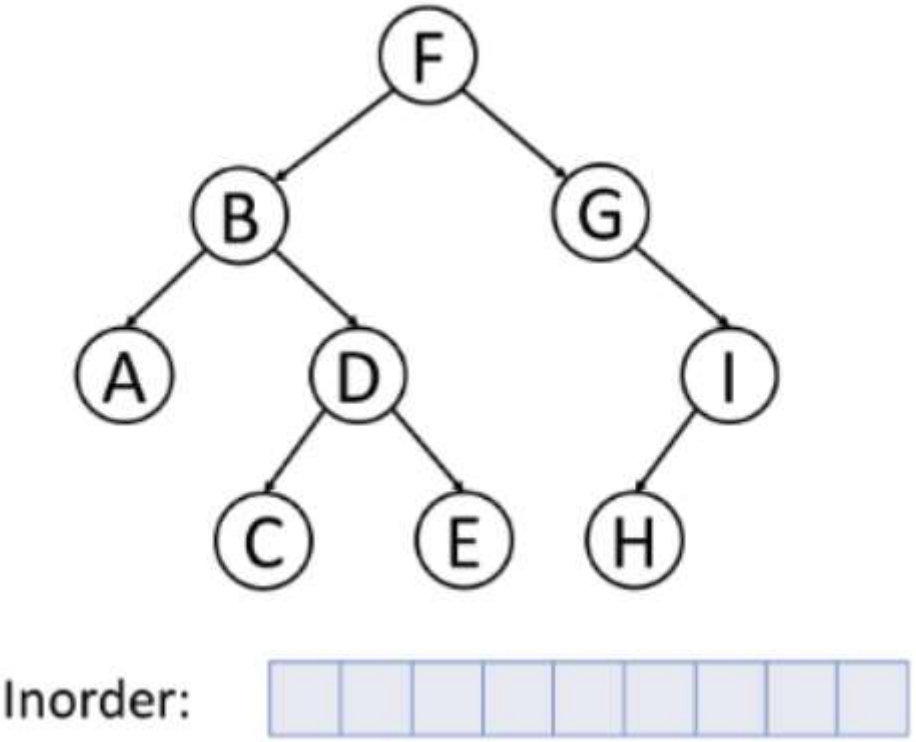
Binary Tree Operations

Preorder (Root- Left- Right)



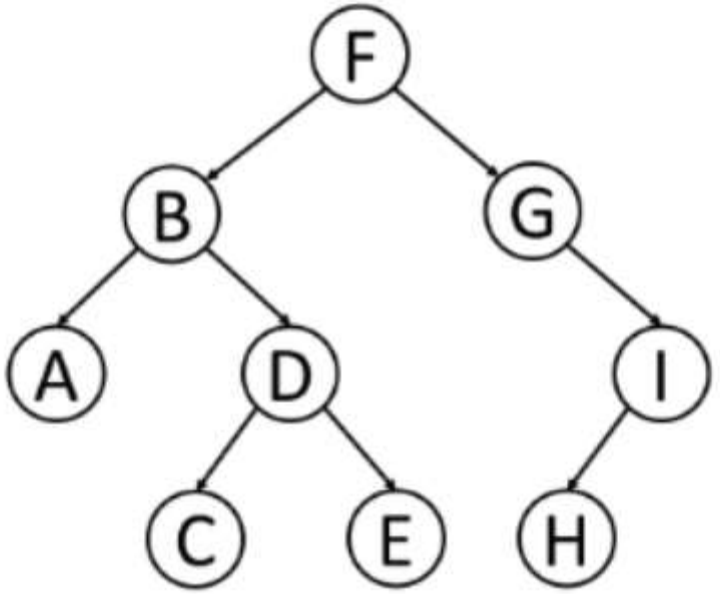
Binary Tree Operations

Inorder (Left- Root- Right)



Binary Tree Operations

Postorder (Left- Right- Root)



Quiz



1) The number of edges from the node to the deepest leaf is called _____ of the tree.

a) Height

b) Depth

c) Level

d) None of the Above

Answer : Option a)

Quiz



2) Which of the following is not an advantage of trees?

a) Hierarchical structure

b) Faster search

c) Routing Algorithm

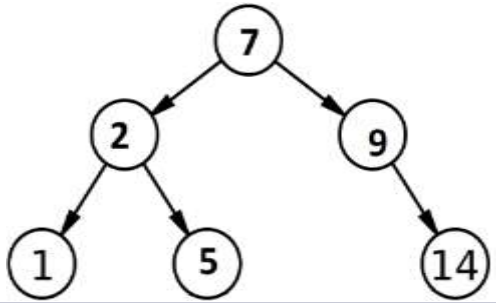
d) Undo/Redo operations

Answer : Option d)

Quiz



3) Consider the following binary tree. Which statement is correct?



a) Tree is neither complete nor full.

b) Tree is complete but not full.

c) Tree is full but not complete.

d) Tree is both full and complete.

Answer : Option a)

Quiz



4) Which of the following represents the Postorder Traversal of a Binary Tree?

a) Left – Right – Root

b) Left – Root – Right

c) Right – Left – Root

d) Right – Root – Left

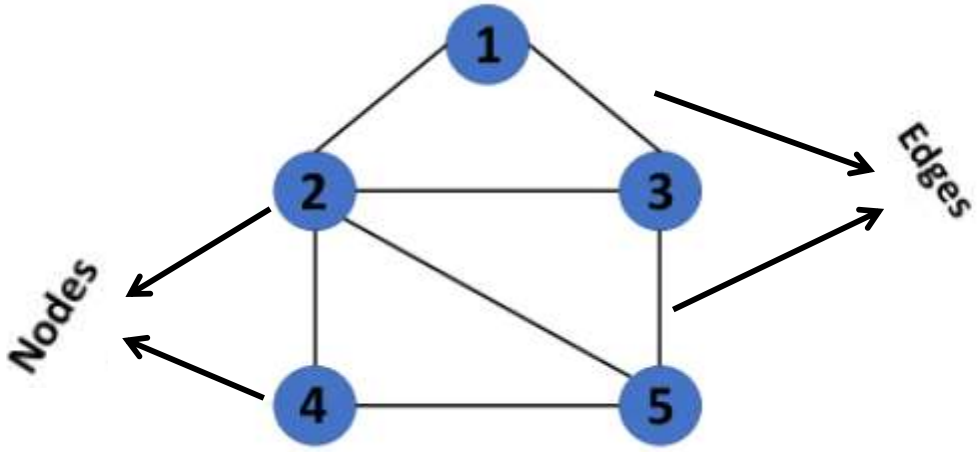
Answer : Option a)

Introduction to Graph



What is Graph?

- A graph is a **non-linear** kind of data structure made up of **nodes or vertices** and **edges**.
- The edges connect any two nodes in the graph, and the **nodes** are also known as **vertices**.

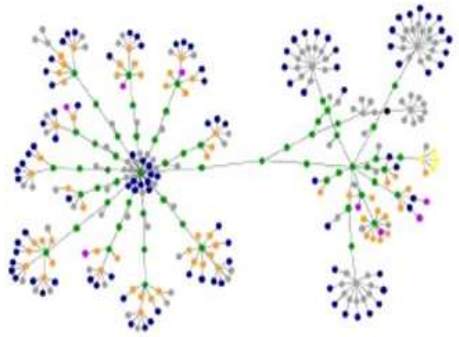


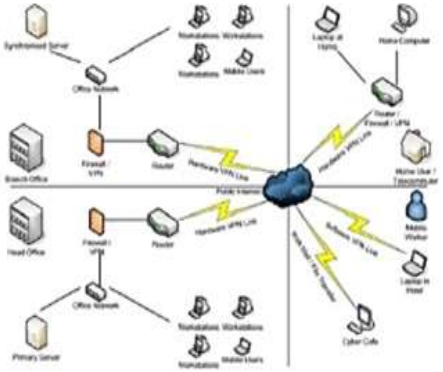
This graph has a

- set of vertices V = { 1,2,3,4,5}
- set of edges E = { (1,2),(1,3), (2,3),(2,4), (2,5), (3,5),(4,50 }

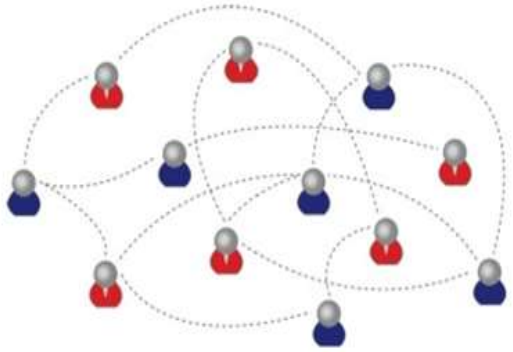
- Graphs in data structures are used to address real-world problems in which it represents the **problem area as a network** like **telephone networks, circuit networks, and social networks**.

Graphs are Everywhere









Driving Map

Edge = Road

Vertex = Intersection

Edge weight = Time required to cover the road

Airline Traffic

Vertex = Cities serviced by the airline

Edge = Flight exists between two cities

Edge weight = Flight time or flight cost or both

Computer networks

Vertex = Server nodes

Edge = Data link

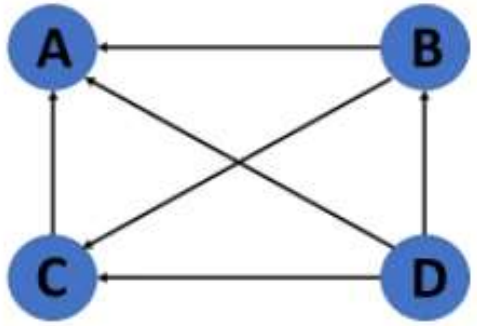
Edge weight = Connection speed

Etc.,

Types of Graph

Directed Graph

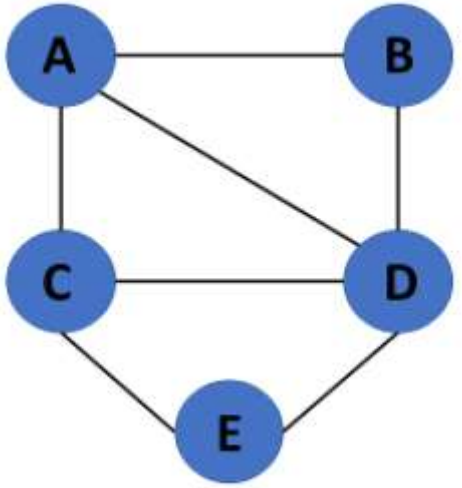
- It is also called as **Digraph**
- Each edge has a direction, which indicates that can move from one vertex to the other through the edge only in the **given direction.**



Directed Graph

Undirected Graph

- Can move in **both directions** between vertices.

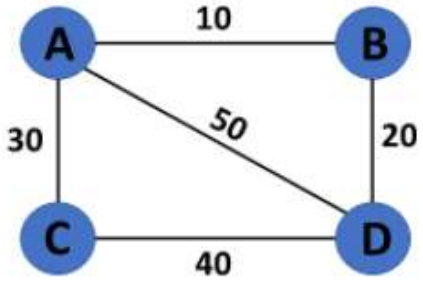


Undirected Graph

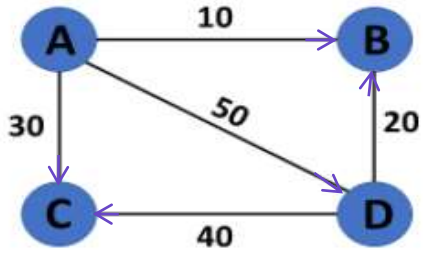
Types of Graph

Weighted Graph

- A graph G= (V, E) is called a **labeled or weighted** graph because each edge has a **value or weight** representing the cost of traversing that edge.



Undirected Graph



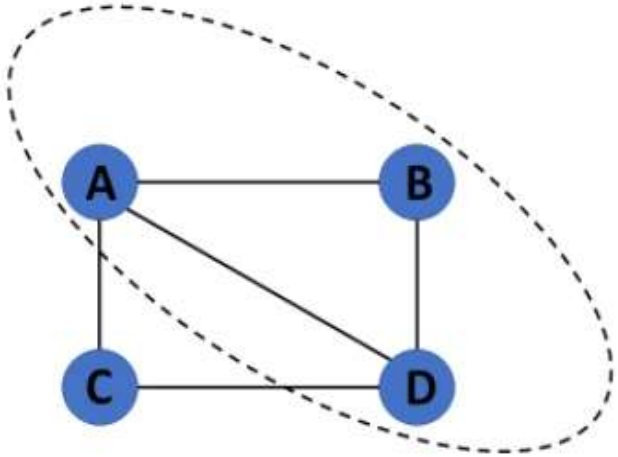
Directed Graph

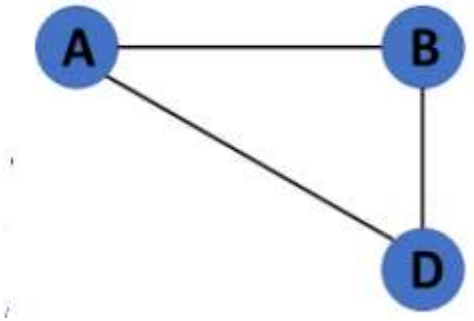
- **Unweighted Graph:** A graph in which there is **no value or weight** associated with the edge. All the graphs are unweighted by default unless there is a value associated.

Basic Graph Terminologies

Subgraph

- A subgraph **S** of a graph **G** is a graph whose set of vertices and set of edges are all subsets of G.





Adjacent Vertices and Edges

- Vertex A is said to be adjacent to vertex B if there is an edge between A and B.



Basic Graph Terminologies

Path

- A path is a **sequence of vertices** with the property that each vertex in the sequence is adjacent to the vertex next to it.
- **Some paths in Graph** :

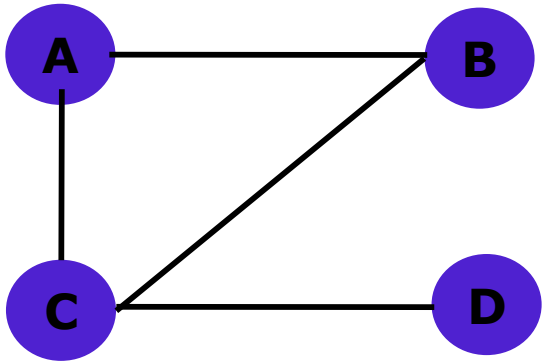
  A B C D

  A C B A C D

  A B

  D C B

  C B A



- A path that **does not repeat vertices** is called a simple path.
- In Graph, D C B A is a **simple path**, while D C B A C is not a simple path

Basic Graph Terminologies

Circuit

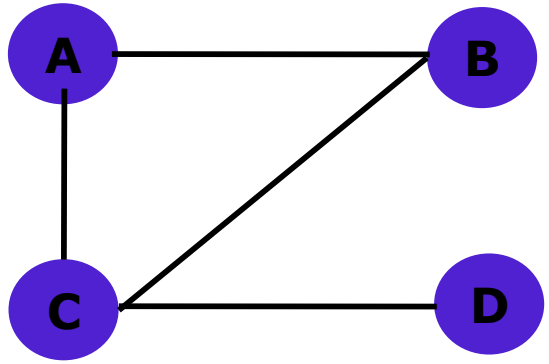
- A circuit is path that **begins and ends** at the **same vertex**.

Cycle

- A circuit that **doesn't repeat vertices** is called a cycle.

Cycles in Graph

- C A B C, C B A C, A B C A, A C B A, B A C B, B C A B
- **A B A** is not a cycle, because the **edge A B** is the **same as B A**



Basic Graph Terminologies

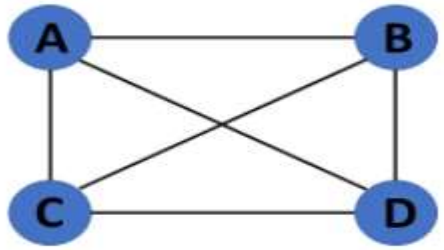
Acyclic Graph

- When there are **no cycles in a graph**, it is called an **acyclic graph**.



Complete Graph

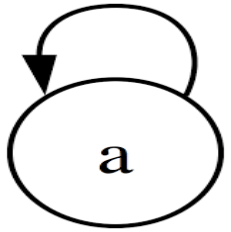
- If a graph G= (V, E) is also a simple graph, it is **complete**. Using the edges, with n number of vertices must be connected. It's also known as a **full graph** because **each vertex's degree must be n-1**.



Basic Graph Terminologies

Loop

- An edge that connects the vertex with itself.

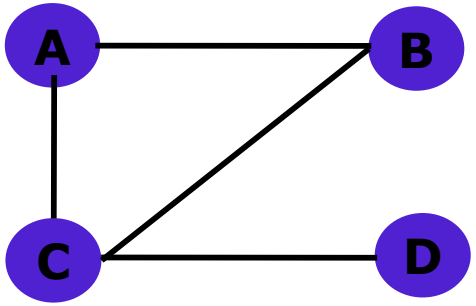


The Size of a Graph:

- The size of a graph is the number of vertices that it has.

Degree

- The number of edges incident on a vertex V determines its degree



Vertex	Degree
A	2
B	2
C	3
D	1

Representation of Graphs

- Graphs in data structures are used to represent the **relationships between objects**.
- Every graph consists of a set of points known as **vertices or nodes** connected by lines known as **edges.**
- The **most frequent graph representations** are the two that follow:
  - Adjacency matrix
  - Adjacency list

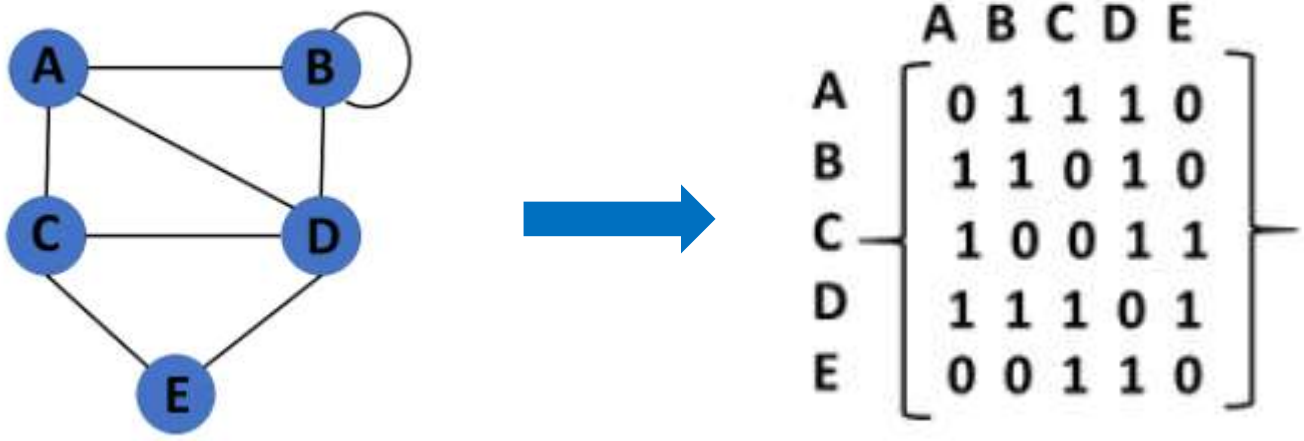
Representation of Graphs

Adjacency Matrix Representation

- Represents a graph as a **2-dimensional array**.
- The adjacency **matrix A** for a graph G=(V,E) with **n vertices**, is an **n*n matrix** such that,
  - A[i, j] = 0 if there is no edge between vertices i and j
  - A[i, j] = 1 if there is an edge between vertices i and j
- **Undirected graphs** have both A[i, j] and A[j, i] = 1 if there is an edge between vertices i and j
  - A[i, j] = weight for weighted graphs

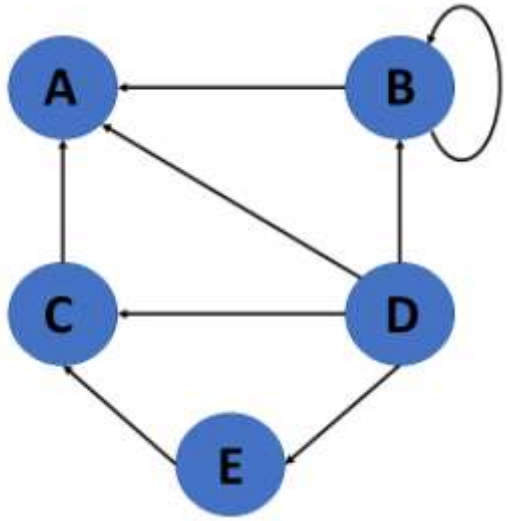
Representation of Graphs

- **Adjacency Matrix Representation:** Undirected Graph Representation



Representation of Graphs

- **Adjacency Matrix Representation:** Directed Graph Representation

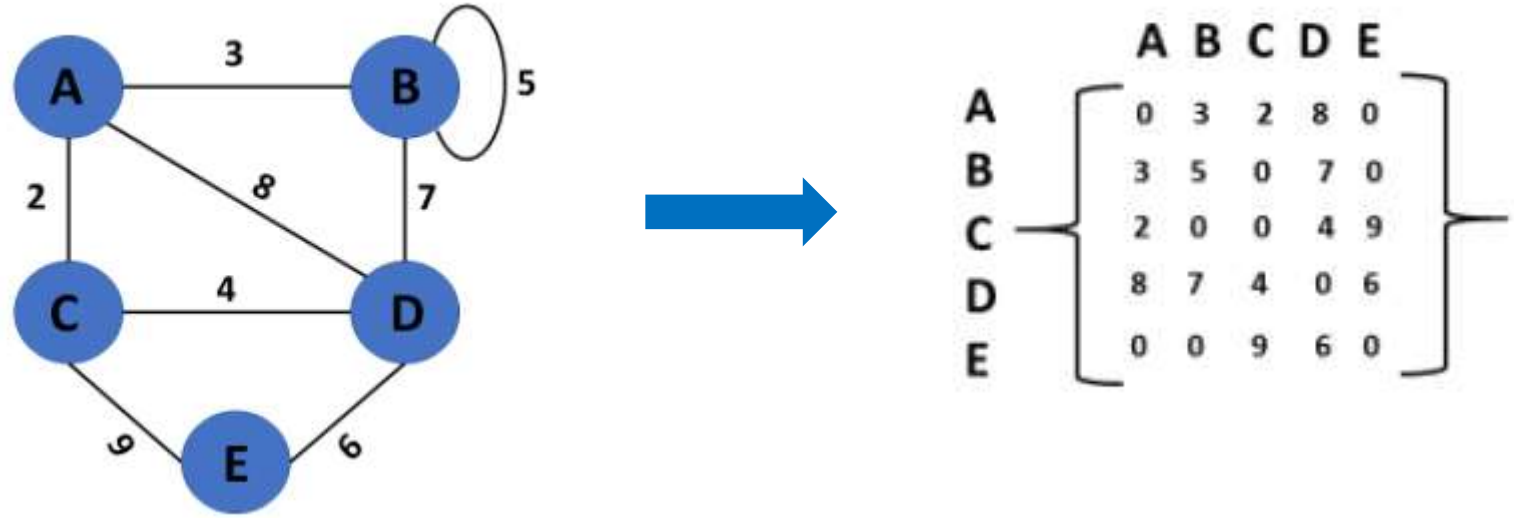




	A	B	C	D	E
A	0	0	0	0	0
B	1	1	0	0	0
C	1	0	0	0	0
D	1	1	1	0	1
E	0	0	1	0	0

Representation of Graphs

- **Adjacency Matrix Representation:** Weighted Undirected Graph Representation
  - **Weight or cost** is indicated at the **graph's edge**, a weighted graph representing these values in the matrix.

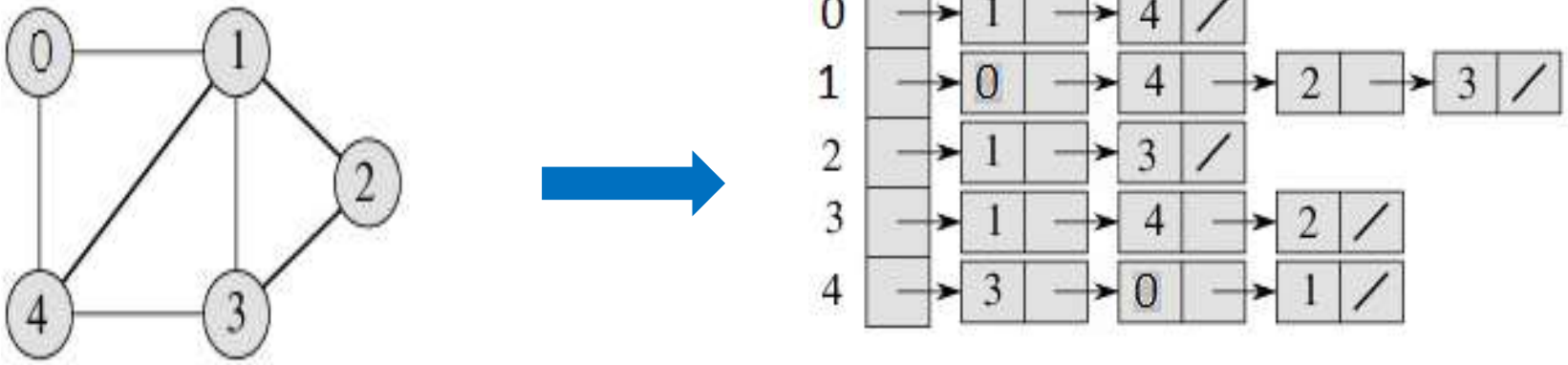


Representation of Graphs

Adjacency List

- A linked representation is an adjacency list.
- A vertex w is inserted in the list for vertex v if there is an outgoing edge from v to w

Undirected Graph Representation



Graph Search (traversal)

- The process of **visiting (checking and/or updating) each vertex** in a graph exactly once.
- Traversals are classified by the **order** in which the vertices are visited.
- **Two different types** of traversal strategies are:
    - Breadth First Search (BFS)
    - Depth First Search (DFS)

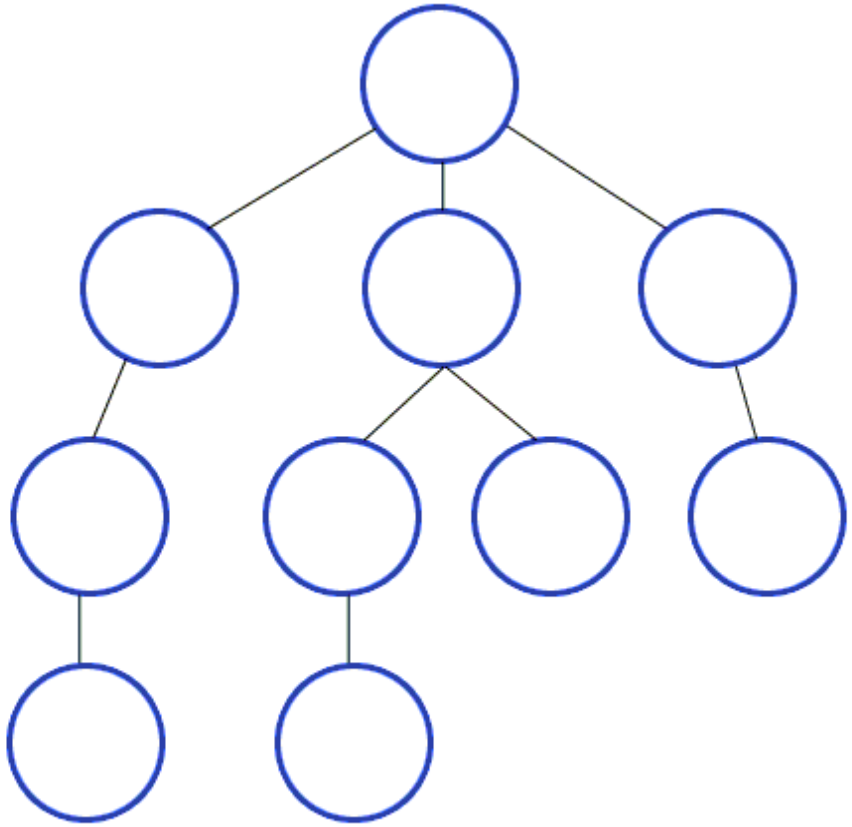
Graph Search (traversal)

Breadth First Search (BFS)

- Visit the sibling before the child.
- Use **Queue data structure**

Steps

1. Enqueue the node into queue
   - Make the front element visited, Enqueue its neighbors into the queue without duplicates (either visited or already in queue)
2. Dequeue the front element.
   - Repeat the above two process again and again until the queue becomes empty & all the nodes are visited.



Graph Search (traversal)

Depth First Search (DFS)

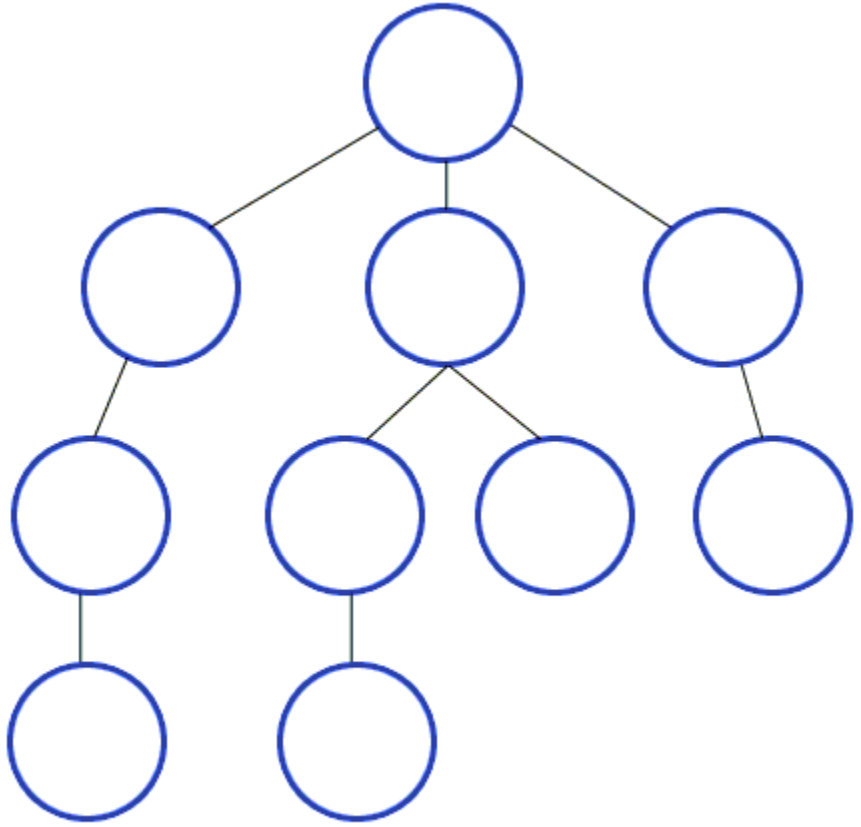
- Visit the Child before the sibling
- Use **Stack data structure**

Steps

1. Push the node into stack

   - Make the top element visited, pop it, make it visited and then push its neighbors into the stack without duplicates (either visited or already in queue)

2. Repeat the above two process again and again until the queue becomes empty & all the nodes are visited.



Quiz



1) Graphs are represented using .....

a) Adjacency tree

b) Adjacency list

c) Adjacency graph

d) Adjacency queue

Answer : Option b)

Quiz



2) Which of the following is useful ın traversıng a gıven graph by breadth first search?

a) Set

b) List

c) Stack

d) Queue

Answer : Option d)

Quiz



3) A graph is a collection of nodes, called ……. and line segments called arcs or …….. that connect pair of nodes.

a) vertices, edges

b) edges, vertices

c) vertices, paths

d) graph node, edges

Answer : Option a)

Set



Introduction to Set

- A set is a **collection of objects** need not to be in any order i.e. A set is an **unordered collection** of items.
- Sets can also be used to perform **mathematical set operations** like union, intersection, symmetric difference, etc.

Rules

- Elements should not be repeated i.e., **no duplication**
- Each elements should have a reason to be in the set

Example

- Assume we are going to store **Indian cricketers in a set**.
  - The names should not be repeated, as well who is not in a team can't be inserted.
  - This is the restriction we found in a set

Null Set (Empty Set)

- The set with no elements is called **null set or empty set**.

Example

- A = { x | x is a prime number, where 24<x<29 }
- This set can't contain an element. We don’t have any prime number within the limit 24 and 29.
- A = { }

Set Operations

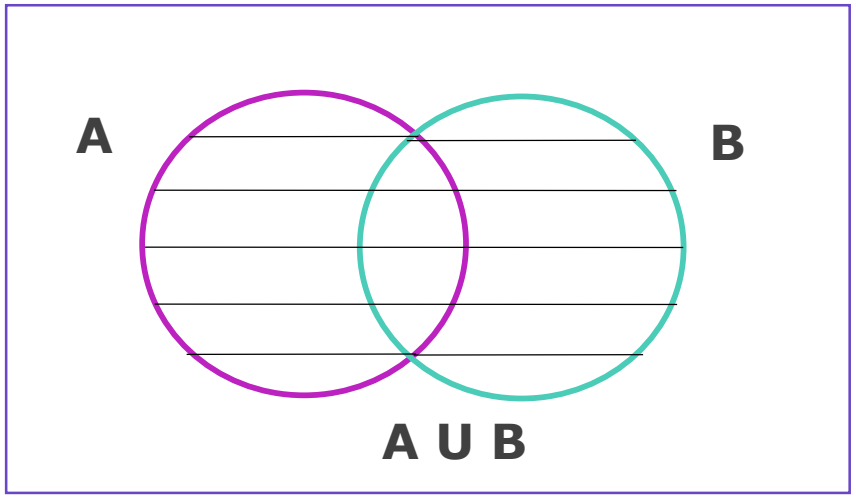
Commonly used **set operations**:

- Union
- Intersection
- Difference
- Symmetric difference

Set Operations

Union

- Union operation can be performed **between the two sets**, and this will **return all the elements** from **both the sets**. This operation can be formed **using U operator**.



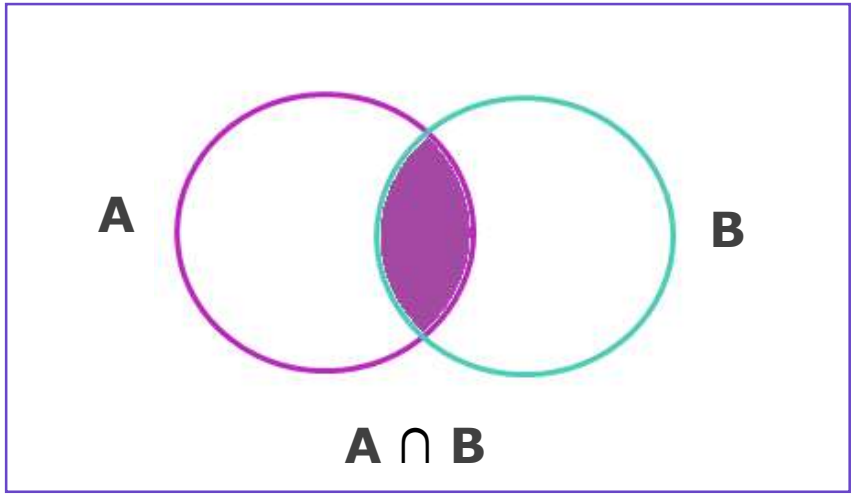
Example:
A = {1, 2, 3, 4, 5}
B = {3, 4, 5, 6, 7, 8}

A U B = {1, 2, 3, 4, 5, 6, 7, 8}

Set Operations

Intersection

- Intersection operation can be performed between the two sets and will return **common elements** from both the sets. This operation can be formed **using ∩ operator.**



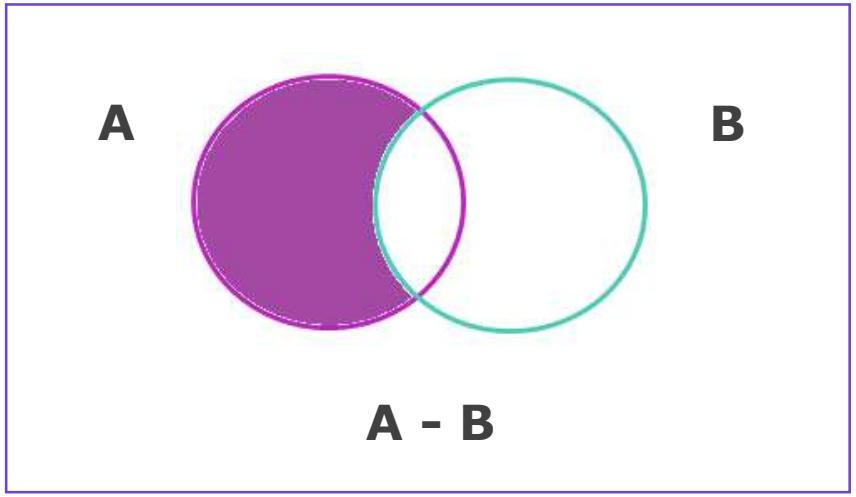
Example:
A = {1, 2, 3, 4, 5}
B = {3, 4, 5, 6, 7, 8}

A ∩ B = {3, 4, 5}

Set Operations

Difference

- Difference operation between set A and Set B (A-B) return the **elements that are only in set A but not in set B**. This operation can also be formed using – operator.



Example:
A = {1, 2, 3, 4, 5}
B = {3, 4, 5, 6, 7, 8}

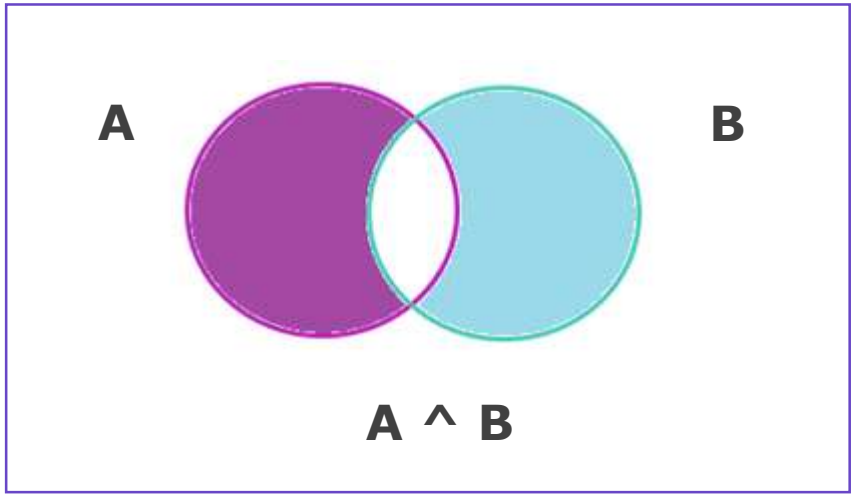
A **-** B = {1, 2}

- A – B = { x | for all x belongs to A but not in B }

Set Operations

Symmetric Difference

- Symmetric difference operation between set A and Set B return the elements the **uncommon elements from both set A and set B**. This operation can also be formed using ^ operator.



Example:
A = {1, 2, 3, 4, 5}
B = {4, 5, 6, 7, 8}

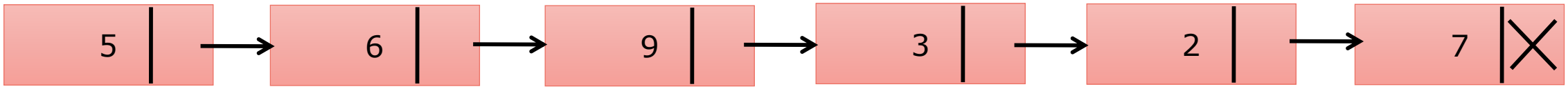
A ^ B = {1, 2, 3, 6, 7, 8}

Representation of Sets

- The set data structure is indirectly **implemented using lists, trees, hash table**, etc by restricting while storing data.
- This is **widely used data structure** when we need **no repetition as well as collection**.

List Representation of Sets

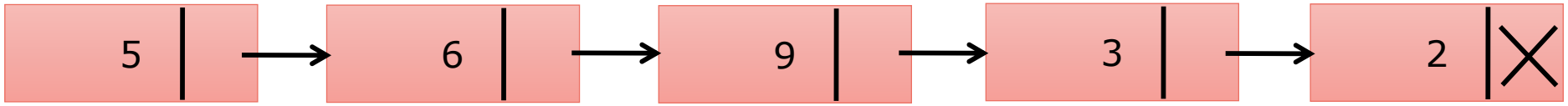
- Simplest and straight forward.
- Best suited for dynamic storage facility.
- This allow multiplicity of elements.
- All operations can be easily implemented, and performance of these operations are as compared to other representations.
- **Example**: set S = { 5,6,9,3,2,7} using linked list structure



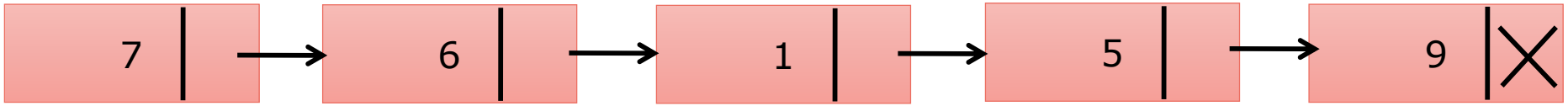
Operations on List Representation of Sets

Union

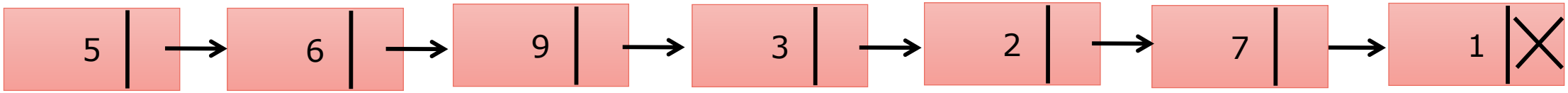
Si:



Sj:



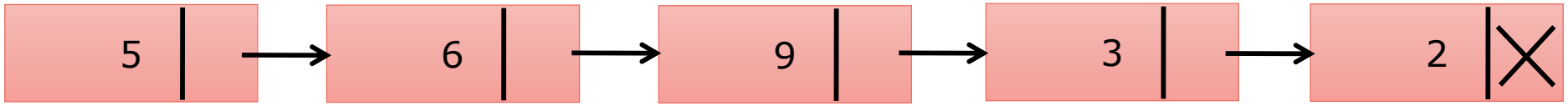
Si U Sj:



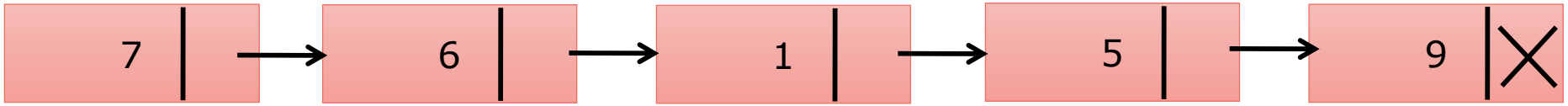
Operations on List Representation of Sets

Intersection

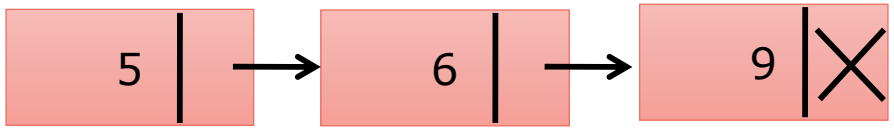
Si:



Sj:



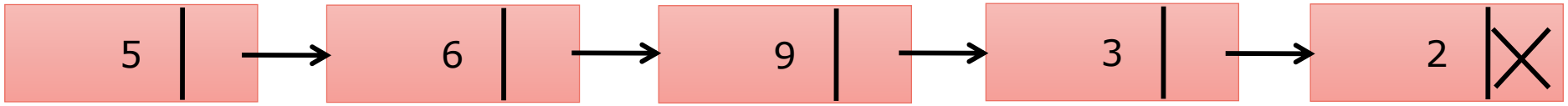
Si ∩ Sj:



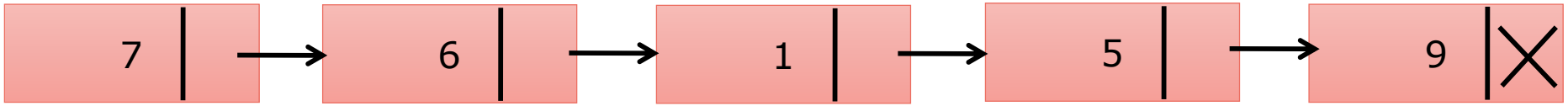
Operations on List Representation of Sets

Difference

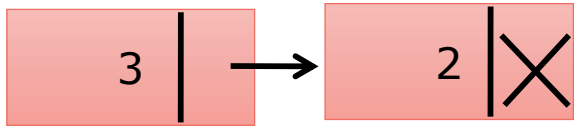
Si:



Sj:



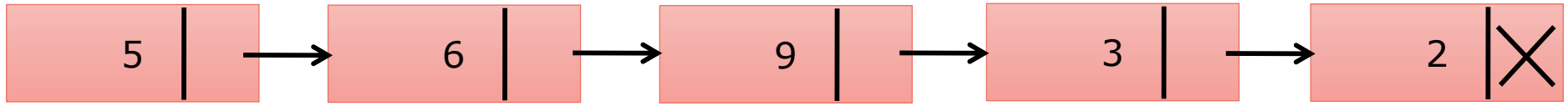
Si - Sj:



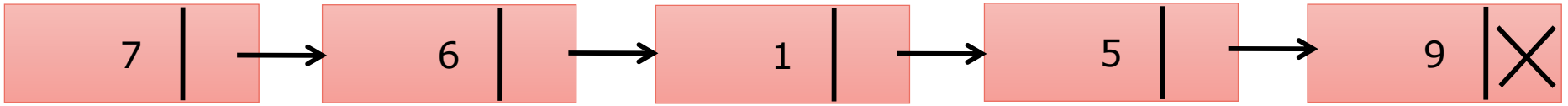
Operations on List Representation of Sets

Symmetric Difference

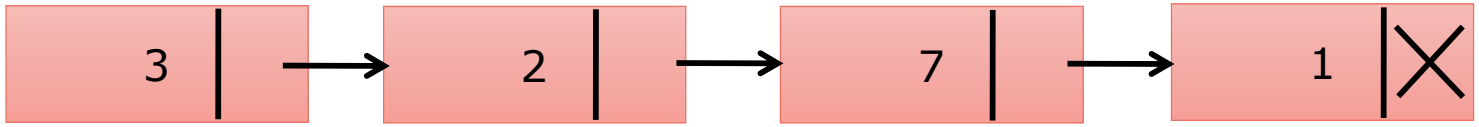
Si:



Sj:



Si ^ Sj:

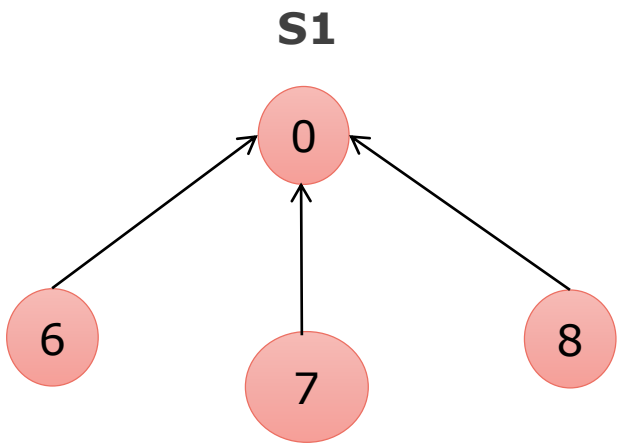


Trees Representation of Sets

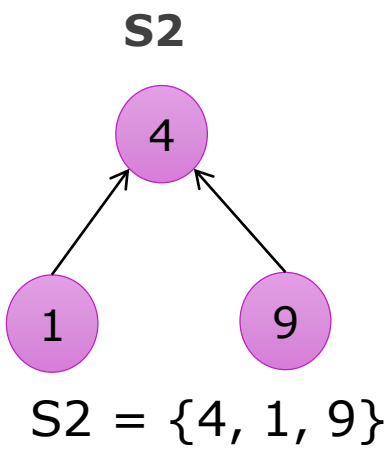
- A tree is used to represent sets.
- If Si and Sj are two disjoint sets, then their union, Si U Sj = { all elements x such that x is in Si or Sj}

Lets consider sets S1 = {0, 6, 7, 8}

S2 = {4, 1, 9}



S1 = {0, 6, 7, 8}

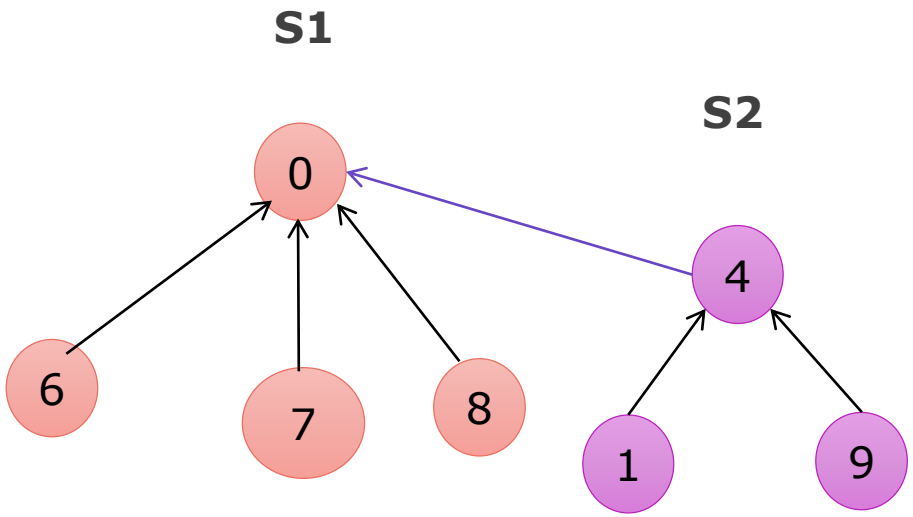


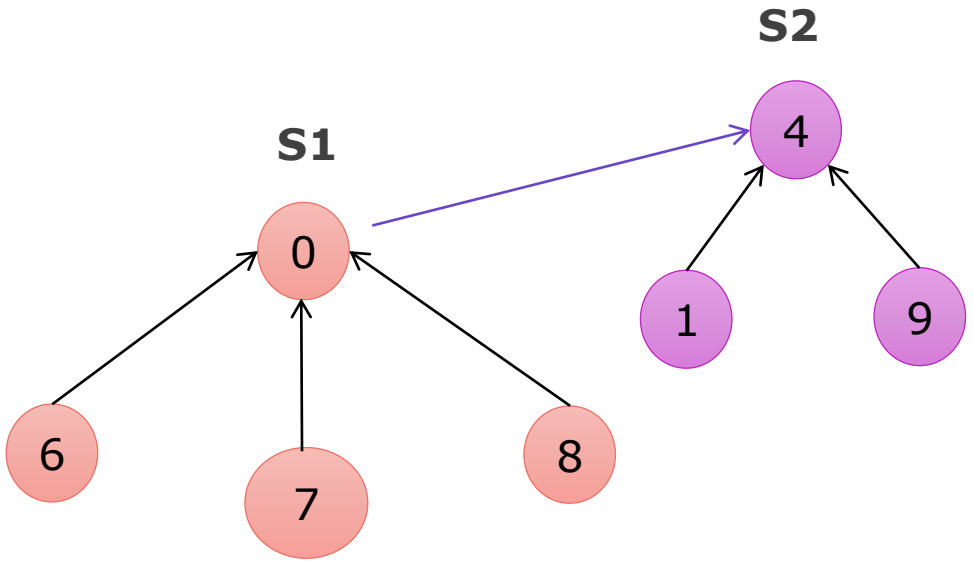
S2 = {4, 1, 9}

Trees Representation of Sets

To obtain the union of two sets, just set the parent field of one of the roots to the other root.

Possible representations of Si U Sj





Quiz



1) A collection of objects need not to be in any order

a) Trees

b) Graph

c) Set

d) None of the mentioned

Answer : Option c)

Quiz



2) Which of the following is true about Set?

a) Elements should be repeated.

b) Each elements need not have a reason to be in the set

c) Elements should not have duplication

d) All the above

Answer : Option c)

Quiz



3) The set with no elements is called _____.

a) Empty set

b) Duplication set

c) Disjoint set

d) None of the above

Answer : Option a)

Quiz



4) The operation that return all the elements from both the sets.

a) Intersection

b) Union

c) Difference

d) Symmetric Difference

Answer : Option b)

Quiz



5) The operation that returns uncommon elements from both the sets.

a) Intersection

b) Union

c) Difference

d) Symmetric Difference

Answer : Option d)


THANK YOU
u·n·i·q technologies
Services | technology | development
SmartCliff
Career Mobility Solutions